NPU COMPILER · V09 BACKEND · TVM 0.19 RELAX / TIR

TVM 기반 LLM NPU Compiler

모델의 계산이 NPU 명령과 다음 token 이 되기까지 :
전체 compile pipeline 과 runtime 을 예제로 따라가기

계산

Relax · semantic TIR · 수치 계약

배치

Schedule · SRAM/DRAM 계획

실행

ISA · linker · C-model runtime

예제 : Llama 3.2 3B (28 layers, hidden 3072, MLP 8192, Q 24 / KV 8 heads, head 128, vocab 128256)
작은 예제 : SiLU(X[64,128]·W^T+bias) 와 두 matmul X·W1·W2 (SRAM 44 KiB) · 설정 : FP16 저장 / FP32 누산 · SRAM 8 MiB · tile 64×64

2026-09-29 탐색 개선 반영 · 근거 : d_compiler/docs (13 장 , examples, evidence)

개요

발표 구성 : 계산 → 배치 → 실행 , 그리고 검증

각 파트는 " 입력 → 해결할 문제 → 실제 동작 → 결과 " 순서로 , 같은 예제를 단계마다 다시 봅니다 .

1 **전체 구조**
장치 계약 , compile/load/run, 세 수준의 IR, 전체 pipeline

2 **모델 → Relax**
relax.frontend.nn Llama, 입력 명세 , 작은 예제의 authored IR

3 **Graph 정리와 host**
CanonicalizeAttention, LiftTransformParams, host transform 실행

4 **LegalizeOps 와 수치**
call_tir + PrimFunc, FP16 저장 /FP32 누산 , softmax·RMSNorm 정의

5 **Fusion 후보 선택**
op_pattern, FuseOps/FuseTIR, 영역 probe 와 전체 module 비교

6 **Matmul scheduling**
64×64 누산 체인 , panel 탐색 공간 , Schedule API, tensorize

7 **Vector· 정규화 ·attention**
SiLU 전개 , VLEN, softmax, RMSNorm, RoPE, query-blocked attention

8 **메모리 계획**
DRAM pool, 수명 , USMP, 커널 사이 SRAM 보관 , 직접 저장 , 주소 압축

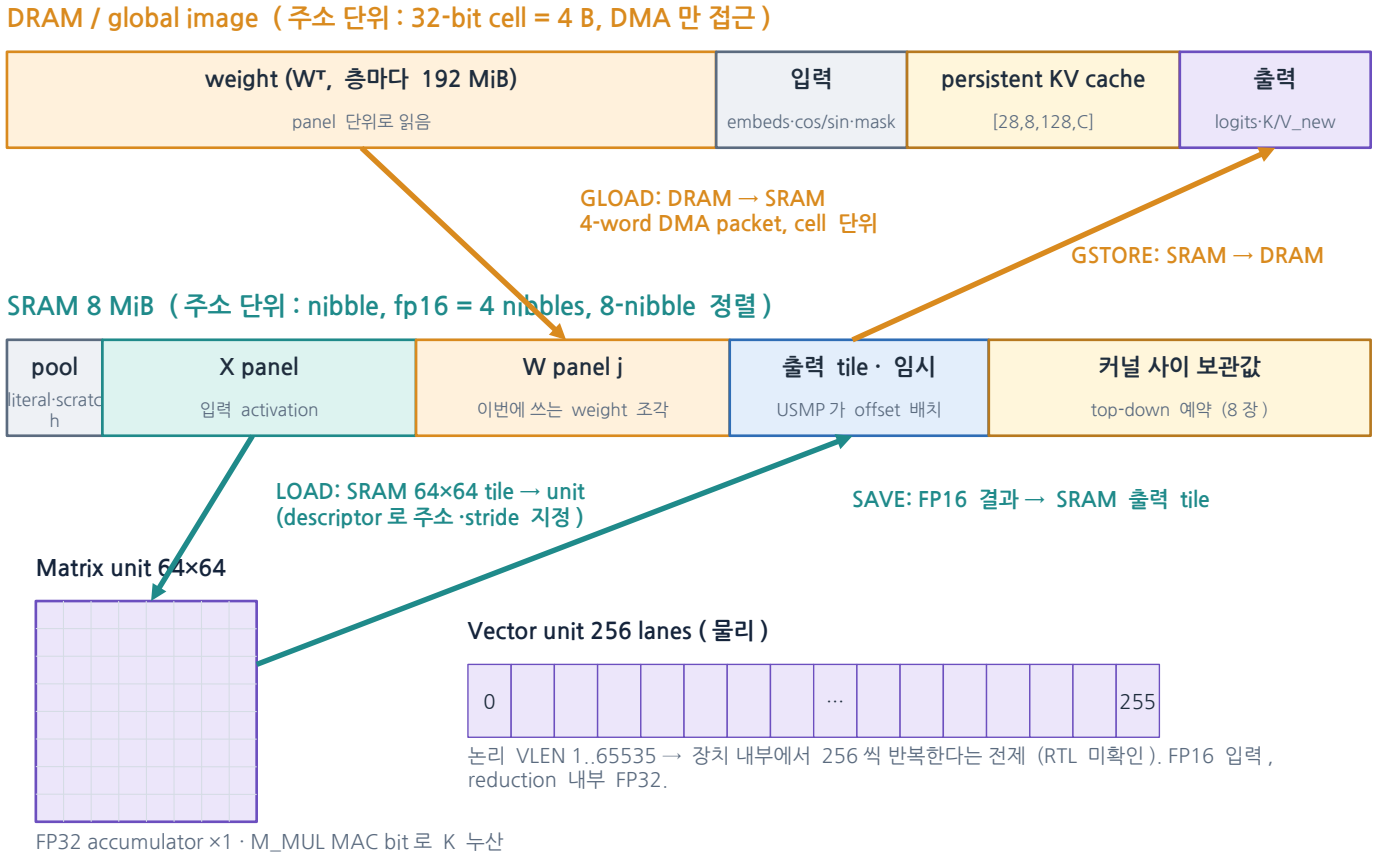
9 **ISA·linker·runtime**
명령 인코딩 해독 , linker, bundle, prefill/decode, KV append

10 **실험 · 한계 · 정리**
E1~E9 측정 ·E10 진행 , 지원 경계 , 남은 검토 목록 , 부록 (main, 양자화 , API)

세 예제를 끝까지 사용 : ① SiLU(X·Wᵀ+bias) [64,128]→[64,64] ② X·W1·W2 두 matmul (SRAM 44 KiB) ③ Llama 3.2 3B, prompt 7 tokens, capacity 9

대상 장치 : 세 층의 메모리와 고정된 수치 계약

데이터가 실제로 어떻게 움직이는가 : DRAM 의 weight 를 panel 단위로 SRAM 에 올리고 (GLOAD), SRAM 의 64×64 tile 을 연산장치에 넣고 (LOAD), 결과를 SRAM 에 내린 뒤 (SAVE) DRAM 으로 보냅니다 (GSTORE).



항목	현재 계약
tile / lanes	64×64 matrix, 256 lanes (VLEN 1..65535)
SRAM	8 MiB = 2^{24} nibbles
DRAM	plan unit 2 B, DMA 주소 32-bit cell
수치	FP16 저장 , K=64 FP32 부분합 누산
누산기	1 개 , partial spill 금지 , K tile 64
Activation	ISA 에 있으나 자동 matmul 경로는 OFF
DMA	GLOAD 0xA0 / GSTORE 0xA8, 4 words
종료	HALT 0xFF, SNAPSHOT 0xF0
실행기	_poc/mysim_v09.cpp (cycle 없음)

연산장치 내부의 FP32 accumulator 는 SRAM buffer 가 아닙니다 . TIR 의 FP32 buffer 가 그대로 SRAM 에 materialize 된다고 읽으면 안 됩니다 .

성능 지표는 DMA bytes → ISA words → SRAM peak 순서의 정적 비교입니다 . C-model 통과가 RTL 검증을 대신하지 않습니다 .

Llama 3.2 3B: 발표 내내 사용할 shape 와 크기

Decode 는 S=1, prefill 은 prompt 길이 S. Weight 는 checkpoint 의 [N,K] 배치이며 계산에는 [K,N] 이 필요합니다 .

설정	값
hidden / intermediate	3072 / 8192
layers	28
Q heads / KV heads / head_dim	24 / 8 / 128 (GQA: Q 3 개가 KV 1 개 공유)
vocabulary	128256
RoPE	llama3, θ=500000, factor 32, low 1, high 4, orig 8192
RMSNorm ε	1e-5
예제 shape	prefill S=7 tokens, decode capacity C=9

Weight (FP16)	shape [N,K]	크기
q_proj / o_proj	3072 × 3072	18 MiB 씩
k_proj / v_proj	1024 × 3072	6 MiB 씩
gate / up	8192 × 3072	48 MiB 씩
down	3072 × 8192	48 MiB
layer 합계	9 tensors	192 MiB × 28 = 5.25 GiB
lm_head	128256 × 3072	751.5 MiB
activation X (S=64)	64 × 3072	384 KiB

Llama 한 layer 에서 계산하는 것 (입력 X[S,3072])

RMSNorm	Q proj (24 heads)	K / V proj (8 heads)	RoPE	Attention	O proj + residual	RMSNorm	gate / up	SiLU(gate) × up	down + residual
[S,3072]	[S,3072]	[S,1024] 각각	Q, K 회전	QK^T → scale → mask → softmax → PV	[S,3072]	[S,3072]	[S,8192] 각각	[S,8192]	[S,3072]

어떤 projection weight 도 8 MiB SRAM 에 통째로 들어가지 않습니다 → panel 단위 streaming 과 재사용 순서가 compiler 의 핵심 결정입니다 .

근거 : docs/examples/llama_ir/summary.json, docs/current/01_overview.md

Compile, load(prepare), run 은 서로 다른 시점의 작업

Compile 은 shape/dtype 만 보고 프로그램을 확정하고 , load 는 weight 를 변환 · 배치하며 , run 은 입력만 갱신해 같은 명령을 재실행합니다 .

Compile

- 입력 : Relax IRModule + 정적 shape/dtype
- graph 후보 선택 · schedule · SRAM/DRAM 주소 · 명령 생성
- Llama: prompt 길이 S 의 prefill_cache 1 개 + capacity C 의 decode 1 개
- 출력 : NpuExecutable = words + plan + lowered module + 입력 signature

Load / prepare

- checkpoint 배열 읽기 (embedding table 은 host 에 유지)
- host transform 실행 : W transpose, (양자화 시) scale/quantize
- 변환된 signature 순서대로 DRAM image 에 배치 (npu_link.build_image)
- 결과를 params-cache 에 저장 (IR+weight 내용 hash 키)

Run

- host: embedding 행 , cos/sin, mask 준비 → image 의 정해진 주소에 patch
- C-model: program_memory.bin + global_memory.bin 실행
- plan.outputs 의 shape/dtype/offset 으로 logits, K/V 읽기
- decode: argmax → 다음 token, KV append DMA tail

```
from npu_compiler import npu_target
with compilation(progress=print):
    exe = npu_target.build(module, func_name="prefill", fusion="auto")
output, counters = exe.run(values)        # values: 최종 signature 이름 → 배열
exe.export_bundle("/tmp/bundle", values)  # program_memory.bin, program.json, IR
```

재사용 조건
weight 가 parameter 로 남아 있고 shape·ABI 가 같으면 값만 바꿔 재사용 . constant folding 한 weight, 다른 S/C 는 다른 프로그램 .

" 모델을 compile 한다 " 에는 graph 선택 · schedule · 주소 배치 · 명령 생성이 모두 들어 있고 , 모델 실행은 그 뒤의 별도 단계입니다 .

근거 : docs/current/01_overview.md, npu_compiler/npu_target.py, npu_generate.py

같은 계산을 네 수준에서 읽는다 : Relax → TIR → scheduled TIR → ISA

작은 예제 P = X[64,128] @ B[128,64] 의 matmul 이 각 수준에서 어떻게 보이는지 . 아래 코드는 모두 실제 dump 에서 발췌했습니다 .

Relax
무엇을 계산하는가

```
lv1 = R.matmul(x, lv)
lv2 = R.add(lv1, bias)
gv  = R.nn.silu(lv2)

# 호출 관계 , shape/dtype
# TensorStructInfo
```

Semantic TIR
어떤 원소를 읽어 더하는가

```
for i0,i1,k in T.grid(64,64,128):
  with T.block("matmul"):
    T.init(): C32[i,j] = 0.0
    C32[i,j] += f32(x[i,k])
                * f32(lv[k,j])
compute[i,j] = f16(C32[i,j])
```

Scheduled TIR
조각 · 위치 · 순서

```
x_global_sram = alloc((64,128),
   "float16", scope="global.sram")
for k_0 in range(2):   # K tile
  npu_gemm_acc(C, A[:,k_0*64:], B)
# cache_read → compute_at →
# tensorize (64×64 intrinsic)
```

ISA words
실제 주소와 장치 명령

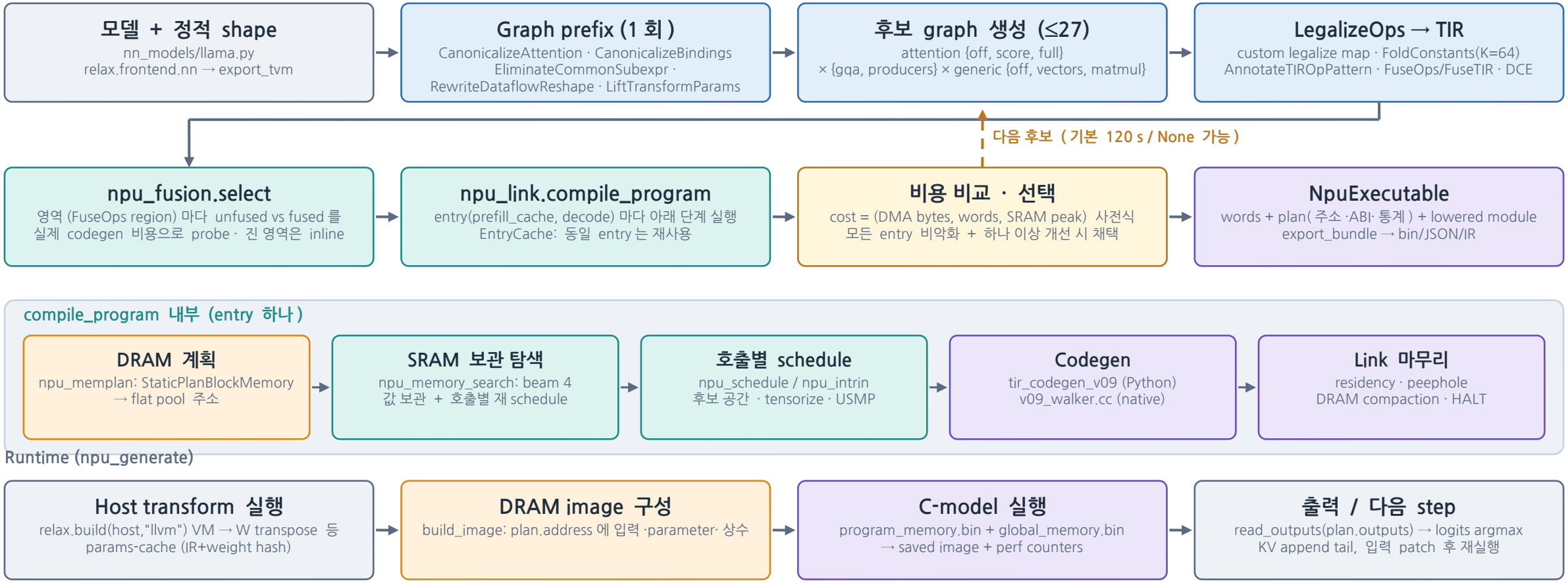
```
000c18a0 00000000  # GLOAD X
00000800 00010800  #  → SRAM 1548
404c1880 ...       # ADDR SRC2
c0000090           # LOAD matrix
80000042           # M_MUL MAC=0
88000042           # M_MUL MAC=1
```

수준	정하는 것	아직 정하지 않는 것	만드는 단계
Relax	연산 호출 순서 , tensor shape/dtype, parameter 경계 (num_input)	원소 계산 방법 , 저장 위치	nn_models export, graph prefix
Semantic TIR	원소 수식 , reduction 축 , FP32→FP16 cast 위치	tile 크기 , SRAM 주소 , DMA 순서	LegalizeOps(custom map), FuseTIR
Scheduled TIR	loop 분할 · 순서 , cache buffer 와 복사 위치 , intrinsic 치환	실제 nibble/cell 주소 , 명령 word	npu_schedule + npu_intrin + USMP
ISA	descriptor 상태 , LOAD/SAVE/M_MUL/vector op, DMA packet 주소	실장치 cycle	tir_codegen_v09 / v09_walker.cc + linker

LegalizeOps 뒤에도 Relax 함수는 남아 호출 순서를 유지하고 , 각 호출의 원소 계산이 TIR PrimFunc 로 정의됩니다 . 하나의 IRModule 에 두 종류의 함수가 공존합니다 .

전체 pipeline: 한 번 지나가는 직선이 아니라 비용으로 되돌아오는 탐색

실제 호출 구조 . 후보 graph 마다 schedule· 메모리 ·codegen 까지 내려가 (DMA bytes, words, SRAM peak) 를 얻고 그 비용으로 선택합니다 .



TVM 표준 기능 위에 NPU backend 의 결정을 얹는다

TVM 이 제공하는 것 (표현 · 변환 · 분석) 과 backend 가 정의하는 것 (장치 계약 · 후보 · 비용 · 명령) 을 구분합니다 . DLight 는 미사용이며 MetaSchedule trace 재사용 ·ReplayFunc 탐색은 선택형입니다 .

TVM 0.19 이 담당 (표준 API 그대로 사용)

영역	사용하는 API
모델 표현	relax.frontend.nn, export_tvm, BlockBuilder, call_te / call_tir
Graph pass	CanonicalizeBindings, EliminateCommonSubexpr, RewriteDataflowReshape, LiftTransformParams, LegalizeOps, FoldConstant, AnnotateTIROpPattern, FuseOps(ByPattern), FuseTIR, DCE
Loop 변환	tir.Schedule: split, reorder, cache_read/write, compute_at, reverse_compute_at, set_scope, pad_einsum, decompose_reduction, tensorize, rfactor
TIR 후처리	LowerInitBlock, PlanAndUpdateBufferAllocationLocation, ConvertBlocksToOpaque, CompactBufferAllocation, LowerMatchBuffer/LowerOpaqueBlock(분석용)
메모리 분석	ToNonDataflow, CallTIRRewrite, StaticPlanBlockMemory, USMP extract_buffer_info + greedy 배치
검증	structural_hash/equal, relax.build(…,"llvm") + VirtualMachine (host transform, CPU reference)
탐색 · 기록 (선택형)	meta_schedule: TuneContext, ScheduleFn, ReplayFunc, sample_categorical, JSONDatabase, TuningRecord, Schedule.trace, relax_integration.extract_tasks

NPU backend 가 정의 (npu_compiler/)

영역	구현
장치 계약	device.NpuProfile, v09_contract.json (dtype/nibble/opcode/ 수치 정책)
수치 정의	npu_legalize: matmul(FP32 누산), softmax, RMSNorm(scaled absmax), sum/mean, split view; npu_numeric: K=64 rfactor reference
후보 생성 · 선택	tvm_pipeline 패턴 , npu_attention, npu_graph_opt, npu_fusion.select, npu_build.lower_and_select, npu_compilation 예산 /cache
Schedule	npu_schedule_space(정규화 · 용량 필터), npu_schedule(_fit_matmul/_fit_attention), npu_intrin(TensorIntrin, schedule 함수), npu_accumulator, npu_meta_schedule(선택형 ReplayFunc·trace DB)
메모리	npu_memplan(flat 주소), npu_sram(USMP 연결), npu_memory_search(커널 사이 보관), npu_residency, npu_abi
명령 · 실행	tir_codegen_v09 / v09_walker.cc, isa_v09, isa_compute, npu_link, peephole, npu_cache, npu_generate, v09_runtime

표준 pass 를 호출했다고 장치 정책이 저절로 생기지는 않습니다 . 반대로 backend 코드가 있다고 TVM 기본 pass 인 것처럼 부르지도 않습니다 .

근거 : docs/current/01_overview.md, reference/tvm_api.md

코드 지도 : 어느 현상이 어느 단계의 결정인지 먼저 정한다

d_compiler/npu_compiler (약 12.7k lines, Python + C++). 파일 순서로 읽기보다 관심 있는 결정을 만드는 단계부터 봅니다 .

책임	파일 (lines)	핵심 함수 / 클래스
공개 build · 결과물	npu_target.py (178)	npu_target(), build() → NpuExecutable{words, plan, module}, export_bundle()
모델 · checkpoint	nn_models/llama.py (435), hf.py (437), assets/llama.py	LlamaModel(prefill/prefill_cache/decode), get_default_spec, rope_inputs, causal_mask, Llama32Assets
Graph · 후보	tvm_pipeline.py (169), npu_build.py (124), npu_fusion.py (161), npu_attention.py, npu_graph_opt.py	graph_prefix, graph_pipeline, lower_and_select, select(_probe), CanonicalizeAttention, FoldRepeatedGemv
수치 · schedule	npu_legalize.py (190), npu_numeric.py, npu_schedule_space.py (78), npu_schedule.py (616), npu_intrin.py (621), npu_accumulator.py (201)	legalize_map, tile_order_reference, candidates(), _fit_matmul / _fit_attention / _measure_trial, schedule_matmul_sram, lower_stores
메모리 · ABI	npu_memplan.py (427), npu_sram.py (210), npu_memory_search.py (379), npu_residency.py (179), npu_abi.py (173)	assign_addresses, compact_unused_storage, plan_sram(USMP), discover/plan_program/reserve, forward, output_tree/validate_entry
ISA · link · 실행	npu_link.py (472), tir_codegen_v09.py (1533), npu_codegen/v09_walker.cc (1696), isa_v09.py, isa_compute.py, peephole.py, npu_cache.py, npu_generate.py (398), v09_runtime.py	compile_program, Walker.emit_gemm/flush, DMA packet 인코더 , AppendPlan, generate/DecodeSession, PersistentTransport
Compile 제어	npu_compilation.py (373), v09_stats.py, npu_provenance.py	compilation() context: recipe/trial/probe/region/Entry cache, optimization_budget, phase; analyze/cost

검증 도구 : tests/ 17 개 suite (run_gate.sh) · docs/examples/reproduce.py (작은 예제 실제 실행) · run_nn_generate.py (전체 Llama 생성) · run_nn_npu.py (prefill + HF/LLVM cosine)

근거 : docs/current/12_operations_limits.md, wc -l npu_compiler/*.py npu_codegen/*.cc

Llama 는 relax.frontend.nn 으로 정의하고 정적 명세로 export 한다

Compiler 가 보는 것은 Python 의 for-loop 가 아니라 export 된 계산 그래프입니다 . Embedding lookup 은 host 에 남기고 input_embeds 를 device 입력으로 받습니다 .

```
class Attention(nn.Module):        # llama.py
  q/k/v/o = nn.Linear(..., bias=False)   # k,v out = 8*128
  def forward(self, x, cos, sin, mask):
    q,k,v = reshape → [S,24,128], [S,8,128]
    q,k   = x*cos + rotate_half(x)*sin   # split/neg/concat
    k,v   = op.repeat(k, 3, axis=1)      # GQA: 24 Q ← 8 KV
    scores = matmul(q_h, kᵀ_h) * (1/√128) + mask
    probs  = softmax(scores)             # FP16 (FP32 sum 내부 )
    return o_proj(matmul(probs, v_h))
class MLP:  down(silu(gate(x)) * up(x))
class DecoderLayer:  x + attn(norm1(x)); r + mlp(norm2(r))

spec = get_default_spec(seq=7, decode_context=9)
mod, params = LlamaModel(cfg).export_tvm(spec)   # packed params
mod["decode"] = npu_cache.declare(mod["decode"],
     [("k_cache", 1, 3), ("v_cache", 2, 2)])  # npu.cache_append
```

K cache 는 [L, kv, hd, C] 로 transposed 저장 , V 는 [L, kv, C, hd] 입니다 . decode 는 cache 와 현재 token 의 K/V 를 concat 하여 C+1 길이로 attention 하고 , 현재 token 은 임시로 slot C 에 놓입니다 .

Entry	Device 입력 (FP16)	반환
prefill	input_embeds [S,3072], cos/sin [S,1,128], mask [1,S,S], packed_params	logits [S,128256]
prefill_cache	위와 같음 (num_input = 4)	(logits [1,V]*, K [L,8,128,S], V [L,8,S,128])
decode	input_embeds [1,3072], cos/sin [1,1,128], k_cache [L,8,128,C], v_cache [L,8,C,128], mask [1,1,C+1], packed_params (num_input = 6)	(logits [1,V], K_new [L,8,128,1], V_new [L,8,1,128])

packed_params: 9L+2 개 tensor 를 담은 tuple 인자 하나 (ABI 표현). 물리적으로 합쳐진 배열이 아님 . weight 는 HF [N,K] 그대로

num_input: 앞 N 개 인자만 실행 입력 , 나머지는 parameter 라는 경계 attribute → LiftTransformParams 가 사용

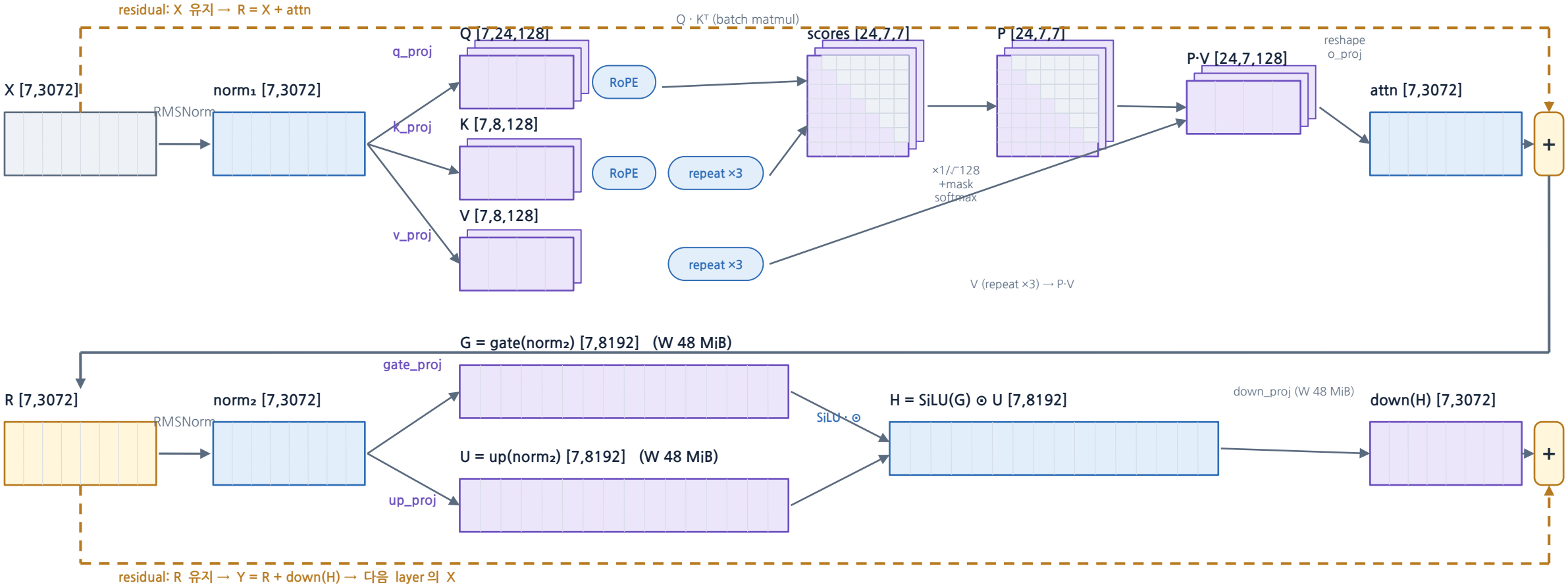
***last_logits:** generation 경로는 마지막 hidden 행만 norm/LM head 에 넘겨 [1,V] logits (TE last_hidden)

Cache 효과는 함수 attribute(npu.cache_append) 로 선언되어 순수 tensor 계산 밖에 있습니다 . layer 수나 cache 이름을 codegen 에 하드코딩하지 않습니다 .

근거 : npu_compiler/nn_models/llama.py, docs/current/02_model_relax.md

Llama 한 layer 의 계산 그래프 (prefill, S=7): tensor 의 모양으로 보기

각 glyph 는 tensor 하나이고 화살표 위 글자가 연산입니다. 겹친 glyph 는 head 축 (24 또는 8), 회색 삼각형은 causal mask. 두 residual 과 GQA repeat, RoPE 의 위치를 봅니다. 이 그래프가 28 번 반복됩니다.



읽는 법 : 보라 = matrix unit(projection·QKᵀ·PV), 파랑 = vector/ 정규화, 주황 = residual. glyph 의 칸 수는 축약 (실제 [7,3072] 는 폭 48 tiles). Decode 는 S=1 이며 K/V 대신 cache 와 현재 token 을 concat 한 [8,128,C+1]/[8,C+1,128] 을 씁니다.

실제 authored Relax (decode, 1 layer, C=9) 발췌

실제 3B 폭 , layer 수만 28→1 로 줄인 export 결과 . 변수 이름은 자동 생성이지만 계산은 앞 슬라이드의 그래프와 1:1 대응합니다 .

```
@R.function
def decode(input_embeds: R.Tensor((1, 3072), "float16"), cos: (1,1,128), sin: (1,1,128),
           k_cache: (1, 8, 128, 9), v_cache: (1, 8, 9, 128), mask: (1, 1, 10),
           packed_params: R.Tuple(11 tensors)) -> (logits (1,128256), K_new (1,8,128,1), V_new (1,8,1,128)):
    R.func_attr({"npu.cache_append": '{"entries": [{"axis": 3, "output": 1, "parameter": "k_cache"}, ...]}',
"num_input": 6})
    rms_norm6 = R.nn.rms_norm(input_embeds, layers_0_input_layernorm_weight2, axes=[-1], epsilon=1e-05)
    permute_dims28 = R.permute_dims(layers_0_self_attn_q_proj_weight2)      # [3072,3072] Wᵀ → host 로 이동 예정
    matmul20 = R.matmul(rms_norm6, permute_dims28)            # [1,3072]  Q
    reshape14 = R.reshape(matmul20, [1, 24, 128])
    mul12 = R.multiply(reshape14, cos);  split4 = R.split(reshape14, [64], axis=-1)
    concat6 = R.concat((R.negative(split4[1]), split4[0]), axis=-1)   # rotate_half
    add10 = R.add(mul12, R.multiply(concat6, sin))            # RoPE(Q)  (K 도 동일 )
    concat8 = R.concat((reshape12 /*k_cache [8,128,9]*/, permute_dims31 /*[8,128,1]*/), axis=2)   # [8,128,10]
    repeat4 = R.repeat(concat8, repeats=3, axis=0)            # [24,128,10]  GQA
    matmul23 = R.matmul(permute_dims33 /*[24,1,128]*/, repeat4)                # scores [24,1,10]
    add12 = R.add(R.multiply(matmul23, R.const(0.08837890625, "float16")), mask)  # scale + mask
    softmax2 = R.nn.softmax(add12, axis=-1);  matmul24 = R.matmul(softmax2, repeat5)   # PV [24,1,128]
    matmul25 = R.matmul(reshape17, permute_dims35);  add13 = R.add(input_embeds, matmul25)   # O proj + residual
    rms_norm7 → matmul26 (gate) → silu2 → mul17 = silu2 * matmul27 (up) → matmul28 (down) → add14 = add13 + matmul28
    rms_norm8 = R.nn.rms_norm(add14, norm_weight2);  matmul29 = R.matmul(rms_norm8, permute_dims39)   # LM head
[1,128256]
    gv2 = (matmul29, concat10 /*K_new*/, concat11 /*V_new*/)
    return gv2
```

num_input = 6: input_embeds, cos, sin, k_cache, v_cache, mask 가 실행 입력 . packed_params 는 parameter.

permute_dims(weight): parameter 에만 의존 → 3 장에서 host 함수로 분리 . token 마다 반복하지 않음 .

reshape/split/concat: 일부는 view(zero-copy) 로 처리되고 일부는 복사 . 실제 여부는 lowering 이 결정 (npu_views).

scale 상수 : 1/√128 = 0.0884 를 FP16 상수로 곱함 . Gemma 는 query 에 scale 을 접어 곱셈이 없음 .

mask [1,1,10]: capacity 9 + 현재 token 1. 유효 길이 밖 slot 은 -65504 (FP16 floor).

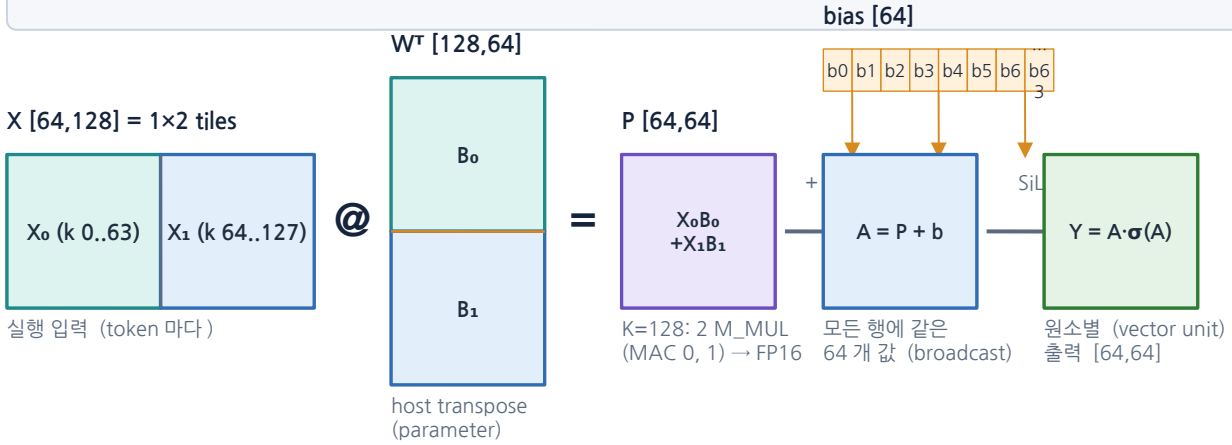
출력 tuple: 3 개 leaf. plan.outputs 가 leaf 별 shape/dtype/offset 을 기록하고 runtime 이 그 주소에서 읽음 .

근거 : docs/examples/llama_ir/01_authored_relax.py (IR export 만 , checkpoint/ 실행 없음)

작은 예제 ① : SiLU(X @ Wᵀ + bias) — 끝까지 따라갈 4 개 연산

X[64,128], checkpoint weight[64,128], bias[64]. 64×64 tile 로 보면 X 는 1×2, Wᵀ 는 2×1, 출력은 1 tile 입니다 . bias 는 broadcast 와 fusion 을 설명하기 위해 넣었고 실제 Llama projection 에는 없습니다 .

```
@R.function
def prefill(x: R.Tensor((64, 128), "float16"), weight: R.Tensor((64, 128), "float16"),
            bias: R.Tensor((64,), "float16")) -> R.Tensor((64, 64), "float16"):
    R.func_attr({"num_input": 1})      # x 만 실행 입력
    with R.dataflow():
        lv:  (128, 64) = R.permute_dims(weight, axes=[1, 0])   # B = Wᵀ
        lv1: (64, 64)  = R.matmul(x, lv, out_dtype="void")     # P = X @ B
        lv2: (64, 64)  = R.add(lv1, bias)                      # A = P + bias ( 행마다 broadcast)
        gv:  (64, 64)  = R.nn.silu(lv2)                        # Y = SiLU(A)
        R.output(gv)
```



tile = 64×64 (matrix unit 단위). 같은 출력 tile P 에 X₀B₀ 와 X₁B₁ 을 누적하고 (6 장), bias/SiLU 는 vector 명령으로 이어집니다 (7 장). 6 장부터 이 그림의 tile 이 SRAM 주소와 명령으로 바뀝니다 .

단계	이 예제에서 볼 파일
authored Relax	01_authored_relax.py
graph prefix (host 분리)	02_graph_prefix.py, 05_host_transform.py
semantic TIR (unfused)	03_unfused_semantic_tir.py
선택된 fusion	04_selected_semantic_tir.py
기본 DRAM plan	06_initial_dram_plan.py
scheduled TIR (실제 recipe)	07_scheduled_fused_matmul_add_silu.py
LLVM reference (K=64 rfactor)	08_llvm_reference.py
ISA packets / bundle	09_packets.json, bundle/program.json
입출력 · 검증	inputs.npz, outputs.npz, stage_summary.json

검증 수준 : 이번 문서 작업에서 fusion off/auto 를 실제 C-model 로 실행 . 두 구성의 FP16 출력이 bit-exact 이고 같은 부분합 grouping 의 LLVM 출력과 최대 절대차 0.

작은 예제 ② (두 matmul X[64,64]·W1[64,128]·W2[128,64]) 는 8 장 메모리 계획에서 , 실제 Llama 는 각 장의 " 실제 폭 " 기록에서 사용합니다 .

Relax 를 읽는 데 필요한 최소 용어

Dataflow block 은 순수 데이터 의존성을 표현할 뿐 하드웨어 동시 실행을 약속하지 않습니다 . 실행 순서와 storage 재사용은 후속 단계가 정합니다 .

표현	역할	작은 예제에서
IRModule	이름 있는 함수들의 집합	prefill, prefill_transform_params, 나중의 matmul/add/silu PrimFunc
relax.Function	tensor 계산의 호출 관계	transpose → matmul → add → SiLU
TensorStructInfo	shape 와 dtype 정보 (값 없이 추론)	x 는 [64,128] FP16, lv1 은 [64,64]
Var / binding	값에 이름 부여	lv1 = matmul 결과
DataflowBlock	순수 tensor 계산 영역	분석 · 변환 범위 (with R.dataflow())
num_input	실행 입력 / parameter 경계 attribute	1 → x 만 입력 , weight·bias 는 parameter
call_tir / GlobalVar	TIR 함수 호출 + 결과 sinfo	R.call_tir(cls.matmul, (x, lv), out_sinfo=…)
BlockBuilder / call_te	type·shape 를 가진 graph 구성 · TE→PrimFunc	legalizer 가 matmul TE 정의를 PrimFunc 로 변환
structural_hash / equal	IR 구조 동일성	후보 dedupe, recipe/entry cache 키

두 frontend 경로

authored (본문): nn_models/llama.py, gemma.py, qwen3.py. 정적 S/C 로 export. generation CLI 는 Llama 대상 .

HF importer (prefill 만): torch.export → HFImporter. weight·position 을 상수로 fold, RMSNorm 패턴 복원 , FP32 구간을 FP16 으로 demote(정밀도 보존 아님), unit batch 축 제거 .

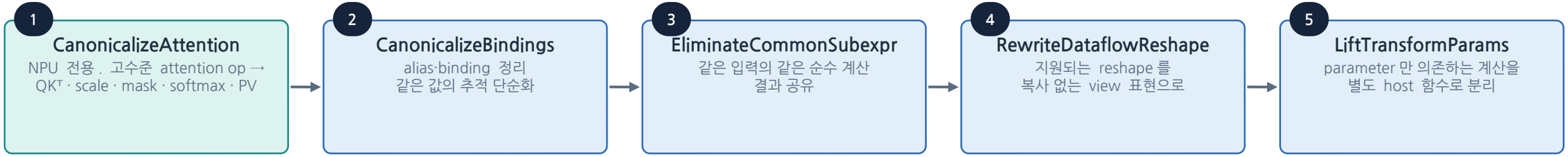
공통 : 임의 동적 control flow 나 모든 HF 모델을 자동 지원하는 compiler 가 아님 . 지원 subset 을 검사하고 밖이면 거절 .

28 층 export
호출 그래프에 모든 층의 계산이 나타납니다 . 같은 shape 의 계산은 schedule 판단 (recipe) 을 공유하지만 , layer 마다 다른 weight 주소와 residual 관계는 최종 프로그램에 그대로 반영됩니다 .

근거 : docs/current/02_model_relax.md, reference/tvm_api.md

Graph prefix: 후속 lowering 이 이해하기 쉬운 graph 로 만드는 5 개 pass

한 번만 적용됩니다 (prepared=True). 1 번만 NPU 전용이고 나머지는 TVM 표준 pass 입니다 .




```python
def graph_prefix(lift_params=True):        # tvm_pipeline.py
    stages = [CanonicalizeAttention(),
relax.transform.CanonicalizeBindings(),
              relax.transform.EliminateCommonSubexpr(),
              relax.transform.RewriteDataflowReshape()]
    if lift_params: stages.append(relax.transform.LiftTransformParams())
    return tvm.transform.Sequential(stages, name="npu_graph_prefix")
```

선택적 후보 (prefix 아님)	무엇을 바꾸는가	언제 채택되는가
FoldRepeatedGemv	matmul(A[24,1,K], repeat(B[8,K,N],3)) → reshape+expand_dims broadcast matmul (m=1, decode 전용)	비용 비교에서 이길 때
matrix_producer 패턴	matmul(x, concat(…)) (+repeat/expand_dims) 을 하나의 composite 로	decode KV concat 등 , 비용 비교
attention / masked_softmax 패턴	FuseOpsByPattern 으로 score chain 또는 전체 attention 그룹	5 장 fusion 후보

- 이 단계는 tile 크기 , SRAM 주소 , DMA 순서를 정하지 않음 . Wᵀ 를 host 에서 준비해도 Wᵀ 전체가 SRAM 에 들어가는 것은 아님
- 사용하지 않는 함수 · 값은 표준 DeadCodeElimination 이 정리
- Authoring 한 Llama 는 이미 세부 연산으로 쓰여 CanonicalizeAttention 이 새로 할 일이 없을 수 있음

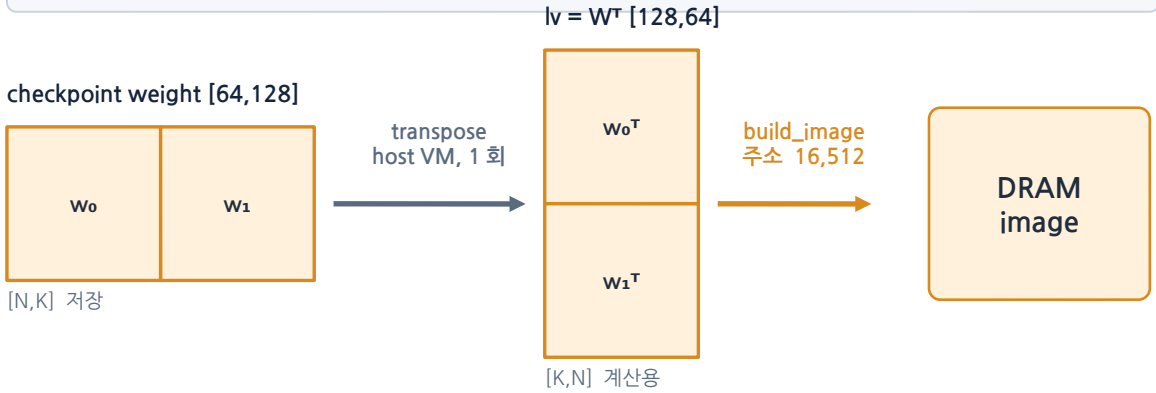
Prefix 의 목적 : 실행마다 필요한 계산과 모델 준비 시 한 번만 할 계산을 구분하고 , 같은 값을 추적하기 쉬운 형태로 만드는 것 .

LiftTransformParams: weight transpose 는 언제 , 어디서 실행되는가

작은 예제의 실제 IR. device 함수에서 permute_dims 가 사라지고 반환 tuple 의 순서가 (bias, lv) 로 바뀝니다 . 변환된 W^T 는 DRAM image 의 정해진 주소에 한 번 놓입니다 .

변환 전 (01_authored_relax)

```
def prefill(x, weight, bias):          # num_input = 1
    lv  = R.permute_dims(weight, [1,0]) # 매 호출마다 transpose?
    lv1 = R.matmul(x, lv)
    lv2 = R.add(lv1, bias)
    gv  = R.nn.silu(lv2)
```



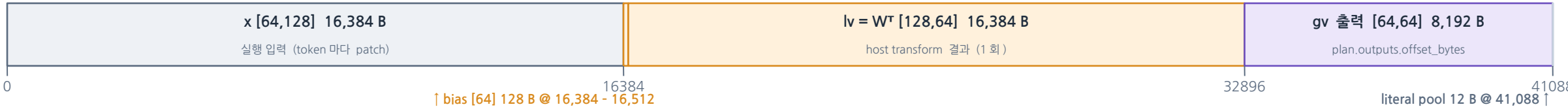
변환 후 (02_graph_prefix) — device 함수

```
def prefill(x, bias, lv: (128,64)):    # num_input = 1
    lv1 = R.matmul(x, lv)               # transpose 없음
    lv2 = R.add(lv1, bias)
    gv  = R.nn.silu(lv2)
    # parameter 순서 : (bias, lv) ← 원래 (weight, bias) 와 다름
```

변환 후 — host 함수 (runtime 준비 단계에서 1 회 실행)

```
def prefill_transform_params(model_params: R.Tuple((64,128), (64,))):
    R.func_attr({"num_input": 0})
    weight = model_params[0]
    lv = R.permute_dims(weight, axes=[1, 0])     # → call_tir(transpose)
    bias = model_params[1]
    return (bias, lv)                            # 변환된 parameter 묶음
```

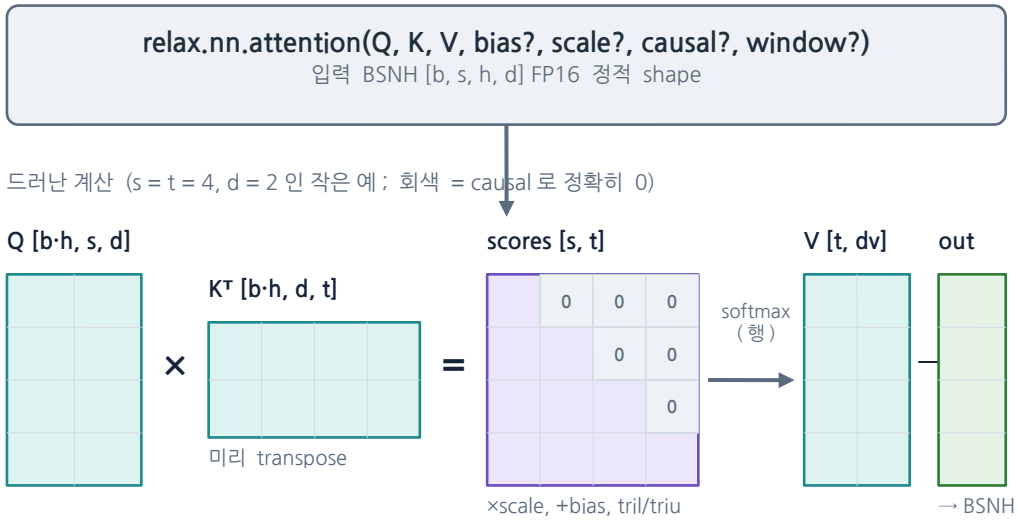
DRAM image (bundle/program.json 실제 주소 , bytes): 변환된 signature 순서 (x, bias, lv) 로 배치 · 총 41,100 B



근거 : docs/examples/pipeline/02_graph_prefix.py, 05_host_transform.py, bundle/program.json (memory.addresses), docs/current/03_graph_host.md

CanonicalizeAttention: attention 을 세부 연산으로 드러내는 이유

고수준 relax.nn.attention 을 그대로 LegalizeOps 에 넘기면 표준 attention legalizer 가 자체 TE 계산을 만들어 , 우리가 등록한 matmul·softmax 의 NPU 정의를 거치지 않습니다 .



IR 비교 기록 (examples/attention_canonicalization, [B,S,H,D]=[1,4,2,8])

경로	결과 IR 의 특징
01 입력	R.nn.attention(q, k, v) 한 호출
02 TVM DecomposeOpsForInference	attention 을 그대로 둠 (이 예제에서 분해하지 않음)
03 NPU CanonicalizeAttention	permute_dims → matmul → multiply(scale) → softmax(max/exp/sum/divide) → matmul → permute_dims
04 표준 attention legalize	attention legalizer 의 자체 TE: NPU matmul/softmax 정의 미사용
05 canonical → legalize	NPU legalize map 의 matmul(FP32 누산), softmax(FP32 sum) PrimFunc 사용

- 단순히 연산 이름을 바꾸는 pass 가 아님 : layout·scale·bias·causal 정렬 ·window 의미를 유지해야 함
- 분해 후 각 연산에 NPU 수치 계약 적용 → scale+mask+softmax 만 묶을지 , 전체를 묶을지 5 장에서 비용으로 비교
- " 분해했으므로 영원히 분리 실행 " 이 아님 . FuseOpsByPattern 이 다시 합칠 수 있음 . window without causal 은 LinkError
- Authoring 한 Llama 는 이미 matmul/softmax 를 쓰므로 이 pass 가 할 일이 없을 수 있음

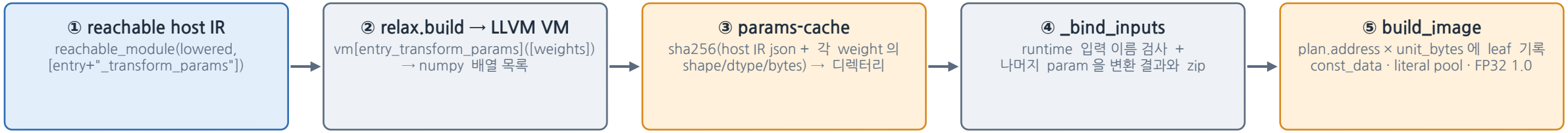
검증 수준 : IR/compile 검사이며 수치 · 성능 benchmark 가 아닙니다 . 표준 attention legalization 과 NPU canonicalization 뒤 legalization 의 차이는 같은 폴더의 dump 에서 확인합니다 .

Relax 에서 먼저 계산을 드러내야 NPU 의 수치 계약 (FP32 누산 ·FP16 경계) 과 fusion 후보 비교를 적용할 수 있습니다 .

근거 : npu_compiler/npu_attention.py, docs/examples/attention_canonicalization/comparison.json

Runtime 의 host transform 실행과 parameter cache

IR 로 표현된 변환을 그대로 실행해야 linked program 의 image 가 기대하는 값과 정확히 같습니다 . NumPy 로 재구현하면 같은 규칙이 두 곳에 생깁니다 .




```
def _run_transform(lowered, entry, weights):
    host = _host_transform_module(lowered, entry)
    vm = relax.VirtualMachine(relax.build(host, "llvm"), tvm.cpu())
    return [t.numpy() for t in vm[entry + "_transform_params"]([weights])]

def _bind_inputs(func, inputs, transformed):
    runtime_count = len(func.params) - len(transformed)
    names = [p.name_hint for p in func.params[:runtime_count]]
    if set(inputs) != set(names): raise ValueError(...)
    return {**inputs, **dict(zip(names_of_rest, transformed))}
```

cache 안전장치	이유
키에 weight 내용 포함	파일명만 같은 다른 checkpoint 재사용 방지
키에 host IR 포함	변환 본문 · 양자화 모드가 바뀌면 miss
flock + COMPLETE marker	동시 실행 시 반쯤 쓴 cache 사용 방지
tmp → os.replace	원자적 commit
배열별 sha256 재검증	손상된 .npy 감지 ("cache is corrupt")
np.load(mmap_mode="r")	build_image 가 복사만 하므로 상주 불필요

준비 비용과 실행 비용의 분리 : 변환 ·image 구성은 모델을 준비할 때의 host 작업이고 , token 마다 실행되는 것은 NPU 계산과 작은 입력 patch(decode step 당 6,676 bytes) 뿐입니다 . 이 둘을 한 표에 더해 "NPU bandwidth" 로 보고하지 않습니다 .

근거 : npu_compiler/npu_generate.py (_run_transform, _transform, _bind_inputs), npu_link.build_image

LegalizeOps 뒤에도 Relax 는 남는다 : call_tir + PrimFunc

작은 예제의 실제 unfused IR. Relax 함수는 호출 순서를 , 각 PrimFunc 는 원소 계산을 정의합니다 . op_pattern attribute 가 다음 단계 FuseOps 의 입력입니다 .

Relax 함수 (호출 순서 유지)

```
@R.function
def prefill(x: (64,128), bias: (64,), lv: (128,64)) -> (64,64):
    R.func_attr({"num_input": 1});  cls = Module
    with R.dataflow():
        lv1 = R.call_tir(cls.matmul, (x, lv),  out_sinfo=R.Tensor((64,64),
"float16"))
        lv2 = R.call_tir(cls.add,    (lv1, bias), out_sinfo=...)
        gv  = R.call_tir(cls.silu,   (lv2,),      out_sinfo=...)
        R.output(gv)
```

PrimFunc 목록 (같은 IRModule 안)

PrimFunc	op_pattern	계산
matmul	4 kOutEWiseFusable	FP32 누산 + FP16 cast, npu.numeric_policy attr
add	0 kElemWise	T_add[i,j] = lv1[i,j] + bias[j] (broadcast)
silu	0 kElemWise	compute = sigmoid(x); T_multiply = x * compute
transpose	2 kInjective	host 함수용 (prefill_transform_params 가 호출)

matmul PrimFunc (TVM Script)

```
@T.prim_func(private=True)
def matmul(x: T.Buffer((64, 128), "float16"), lv: T.Buffer((128, 64), "float16"),
           compute: T.Buffer((64, 64), "float16")):
    T.func_attr({"npu.numeric_policy": "fp16_storage_fp32_accum", "op_pattern": 4})
    matmul = T.alloc_buffer((64, 64))              # FP32 논리 buffer
    for i0, i1, k in T.grid(64, 64, 128):
        with T.block("matmul"):
            v_i0, v_i1, v_k = T.axis.remap("SSR", [i0, i1, k])   # S,S = spatial, R =
reduction
            T.reads(x[v_i0, v_k], lv[v_k, v_i1]);  T.writes(matmul[v_i0, v_i1])
            with T.init():
                matmul[v_i0, v_i1] = T.float32(0.0)
            matmul[v_i0, v_i1] = matmul[v_i0, v_i1] \
                + T.Cast("float32", x[v_i0, v_k]) * T.Cast("float32", lv[v_k, v_i1])
    for i0, i1 in T.grid(64, 64):
        with T.block("compute"):
            compute[v_i0, v_i1] = T.Cast("float16", matmul[v_i0, v_i1])   # FP16 저장
경계
```

TIR 의 FP32 buffer "matmul" 이 반드시 FP32 SRAM 배열로 materialize 되는 것은 아닙니다 . tensorize 뒤에는 장치 accumulator 가 이 상태를 맡습니다 .

Semantic TIR 에서 확인할 것 : 입출력 shape/dtype, 더하는 축과 초기값 , FP32→FP16 cast 위치 , 아직 global 인 buffer, schedule 뒤에도 같은 계산 의미인지 .

근거 : docs/examples/pipeline/03_unfused_semantic_tir.py, docs/current/04_semantic_tir.md

TIR 읽기 : block, 축의 종류 , reads/writes, TE 와 call_te

TE 의 placeholder/compute/reduce_axis 는 " 이 원소의 값은 어떤 수식인가 " 를 선언하고 , bb.call_te 가 그 정의에서 PrimFunc 와 call_tir 를 만듭니다 .

TIR 요소	뜻	이 예제
T.block("matmul")	읽기 · 쓰기 영역과 iteration 관계를 가진 계산 단위	C32[i,j] += f32(x[i,k])·f32(lv[k,j])
T.axis.remap("SSR")	S = spatial(출력 위치), R = reduction(더하는 축)	i0, i1 spatial / k reduction
T.init()	reduction 초기값	C32 = 0.0 (FP32)
T.reads / T.writes	block 접근 영역 → schedule 합법성 분석	x[i,k], lv[k,j] / matmul[i,j]
T.alloc_buffer	PrimFunc 내부 논리 buffer (scope 기본 global)	matmul (64,64) FP32
T.Cast	dtype 변환 위치 = 수치 경계	FP16→FP32 곱 , FP32→FP16 저장
scope	buffer 저장 계층 (global / global.sram / local.accumulator)	schedule 이 global.sram 으로 변경

TE 정의 → PrimFunc (npu_legalize._matmul_te)

```
def _matmul_te(a, b):
    ...  # rank>=2 검사 , batch broadcast shape 계산
    axis = te.reduce_axis((0, ashape[-1]), name="k")
    result = te.compute(shape, lambda *idx: te.sum(
        a[..., idx[-2], axis].astype("float32")
        * b[..., axis, idx[-1]].astype("float32"), axis=axis),
        name="matmul")
    return topi.cast(result, "float16")

def _legalize_matmul(bb, call):   # FP16×FP16→FP16 만 NPU 정의
    return bb.call_te(_matmul_te, *call.args, primfunc_name_hint="matmul",
        primfunc_attrs={"npu.numeric_policy": NUMERIC_POLICY})
```

matmul_te 는 실행기가 아님 : 수식 선언일 뿐 , 손으로 쓴 ISA 생성기도 아님

확장 지점 : LegalizeOps(custom_legalize_map) 에 matmul, sum, mean, rms_norm, softmax, split 만 등록 . 나머지는 TVM 기본 legalization

다른 dtype 조합 : FP16 이 아니면 TVM 기본 _matmul 로 위임 (범용 FP32 vector 실행은 별도 범위)

이 파일에는 아직 최종 DMA 량이 없습니다 . DMA 량은 cache 위치 , 보관 선택 , 실제 주소와 후단 명령 정리까지 적용한 뒤에야 판단합니다 .

NPU legalize map: 기본 lowering 과 다른 dtype· 저장 경계가 필요한 연산만 바꾼다

TVM 기본 RMSNorm 은 FP32 중간값을 유지하지만 우리 vector unit 은 FP16 을 저장하고 내부 reduction 만 FP32 입니다 . 그래서 per-op recipe 만 교체합니다 .

Relax op	NPU 정의 (TE)	지원 조건 검사	PrimFunc attr / 이름
relax.matmul	FP16×FP16 → FP32 sum → FP16 cast. rank≥2, batch broadcast	FP16 입력 ·out_dtype void/float16. 그 외는 TVM 기본 _matmul	npu.numeric_policy = fp16_storage_fp32_accum, "matmul"
relax.nn.softmax	max → exp(x−max) → FP32 sum → FP16 sum → divide	FP16, 마지막 축 , 정적 길이 1..65504 (exp≤1 → FP16 합 범위)	"softmax"
relax.nn.rms_norm	scaled absmax: a=max(absmax,√ε), u=x/a, z=u/√D, s=FP32 Σz²→FP16, y=u·(1/√(s+(√ε/a)²))·w	마지막 축 , 1-D FP16 weight, $2^{-48} \le \varepsilon \le 65504^2$, 정적 D	npu.rms_algorithm = scaled_absmax_v1
relax.sum / mean	FP32 sum → FP16 cast (mean 은 ×1/count FP16)	FP16 입력 . 아니면 TVM 기본	"sum", "mean"
relax.split	axis-0 정적 split → memory.view + ensure_zero_offset (zero-copy)	연속 view 가능한 경우만	decode cache 의 layer 별 분리
그 외 (add, silu, permute_dims, concat, repeat, reshape …)	TVM 기본 legalization	—	op_pattern 은 AnnotateTIROpPattern 이 부여

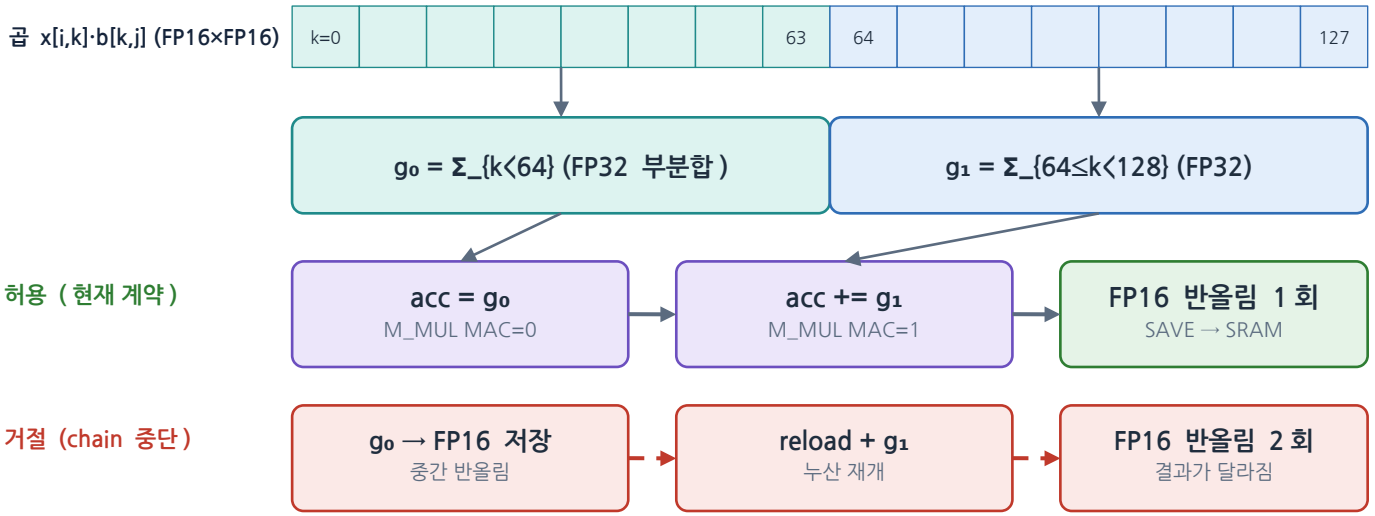
```
def legalize_map(numeric_policy=NUMERIC_POLICY):
    return {"relax.matmul": _legalize_matmul, "relax.split": legalize_split,
            "relax.sum": _legalize_sum, "relax.mean": _legalize_mean,
            "relax.nn.rms_norm": _legalize_rms_norm, "relax.nn.softmax": _legalize_softmax}
stages += [relax.transform.LegalizeOps(custom_legalize), FoldConstants(), AnnotateTIROpPattern()]
```

조용히 통과시키지 않는다 : 지원 subset 밖 (예 : 길이 70000 softmax, FP32 RMSNorm) 은 ValueError 로 거절 . 임의로 근사하거나 fallback 하지 않음 .

근거 : npu_compiler/npu_legalize.py legalize_map(), npu_views.legalize_split

FP16 저장 / FP32 누산 : K 전체 동안 accumulator 를 끊지 않는다

K=128 출력 tile 하나의 128 개 곱은 64 개씩 두 그룹의 FP32 부분합이 되어 같은 accumulator 에 더해지고 , 마지막에 한 번만 FP16 으로 반올림됩니다 . 중간에 FP16 으로 저장했다 다시 더하면 반올림 위치가 달라집니다 .



수치 정책이 적용되는 곳	방법
장치 matmul	tensorize 64×64 intrinsic, MAC chain 1 개 (matrix_partial_spill forbidden)
CPU / LLVM reference	tile_order_reference → relax.build(…,"llvm") VM
Constant folding	FoldConstants: reference module 로 fold 후 원래 body 복원
검증	validate_primitive: sum 은 FP32 zero init, max identity 는 – 65504
Fusion 뒤	matmul→add→SiLU 의 FP16 저장 경계 유지 (SRAM 에 둔다고 FP32 가 되지 않음)

CPU reference: 같은 K=64 grouping (rfactor)

```
# npu_numeric.tile_order_reference: split(k,[None,64]) + rfactor(factor_axis=0)
matmul_rf = T.alloc_buffer((2, 64, 64))            # 64 개 부분합 × 2 (g₀, g₁)
for i0, i1, k_0, k_1 in T.grid(64, 64, 2, 64):
    matmul_rf[vk_0, i, j] += f32(x[i, vk_0*64+vk_1]) * f32(lv[vk_0*64+vk_1, j])
for i0, i1, k_0 in T.grid(64, 64, 2):           # 부분합 2 개를 FP32 로 결합
    matmul[i, j] += matmul_rf[vk_0, i, j]
compute[i, j] = T.Cast("float16", matmul[i, j])    # 마지막에 한 번 반올림
```

정밀도 정책은 고정 : 중간 FP16 반올림을 생략하면 수학적 기준에 더 가까워질 수도 있지만 다른 결과를 만드는 변환이므로 별도 정책이 필요합니다 . 현재 compiler 는 오차 예산을 받아 FP16/FP32 경계를 자동 탐색하지 않습니다 . E1: 같은 grouping 의 LLVM 과 최대 절대차 0.

근거 : docs/current/04_semantic_tir.md, npu_numeric.tile_order_reference, docs/examples/pipeline/08_llvm_reference.py

Softmax 와 RMSNorm 의 실제 정의 : reduction 과 FP16 범위

"FP32 reduction" 은 softmax 전체가 FP32 라는 뜻이 아닙니다 . 입력 ·exp· 저장된 합 · 출력은 FP16 이고 합산 내부만 FP32 입니다 .

Softmax 한 행 [2, 1, 0]

단계	값	장치 계산	dtype
최댓값	2	REDUCE_MAX (첫 원소로 시작)	FP16
최댓값 빼기	[0, −1, −2]	BROADCAST + SUB	FP16
exp	[1, 0.3679, 0.1353]	vector EXP	FP16 저장
합계	1.5032	REDUCE_SUM (내부 FP32) → FP16 저장	FP32→FP16
나누기	[0.6652, 0.2447, 0.0900]	BROADCAST + DIV	FP16

```
def _softmax_te(data):                          # npu_legalize.py
    maximum  = te.compute(shape[:-1], lambda *i: te.max(data[i+(k,)], axis=k))
    exponent = te.compute(shape, lambda *i: te.exp(data[i] - maximum[i[:-1]]))
    total32  = te.compute(shape[:-1], lambda *i: te.sum(exponent[i+
(k,)].astype("float32"), axis=k))
    total    = te.compute(shape[:-1], lambda *i: total32[i].astype("float16"))
    return te.compute(shape, lambda *i: exponent[i] / total[i[:-1]])
```

RMSNorm: scaled_absmax_v1

단계	수식	이유
a	max(absmax(x), √ε)	|x/a| ≤ 1: 제곱 overflow 방지
u	x / a	FP16 저장
z	u / √D	합이 D 에 무관하게 ≤ 1
s	Σ z·z (FP32) → FP16	FP32 reduction, FP16 저장 경계
inv	1 / √(s + (√ε/a)²)	ε 를 같은 scale 로
y	u · inv · weight	원래 x/√(mean(x²)+ε)·w 와 실수 수학에서 동일

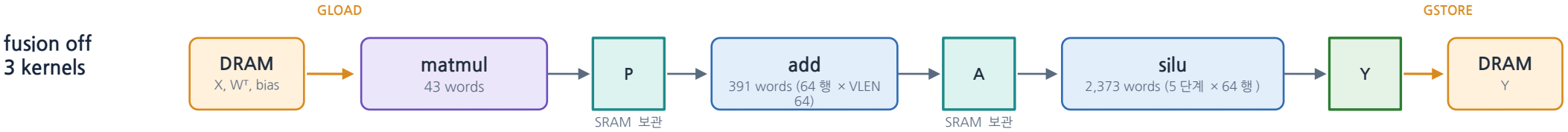
- 원래 식 y = x/√(mean(x²)+ε)·w 를 그대로 쓰면 x=1000 일 때 $x^2=10^6 > 65504$ (FP16 overflow)
- 지원 조건 : 마지막 축 , 1-D FP16 weight, 정적 D, $2^{-48} \le \varepsilon \le 65504^2$
- LegalizeOps 에서 정의하는 명시적 target 근사 . schedule 이 임의로 발명하는 변환이 아님

행 수를 나누는 것과 한 행의 reduction 길이를 나누는 것은 다릅니다 . 임의로 긴 softmax(>65504) 는 현재 계약 밖입니다 .

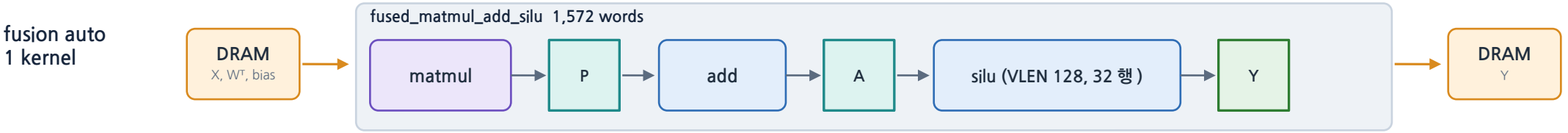
근거 : npu_compiler/npu_legalize.py (_softmax_te, _rms_norm_te), docs/current/07_operators.md

Fusion: 커널 수가 줄어드는 것만으로 이득을 판단하지 않는다

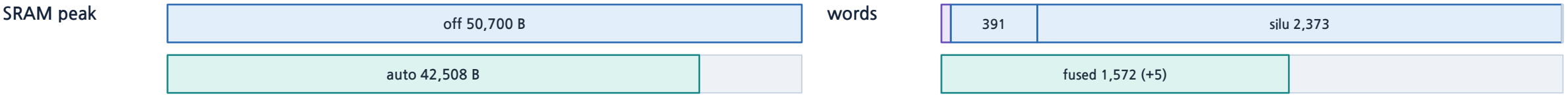
작은 예제에서 분리 (off) 와 결합 (auto) 의 실제 데이터 이동 . 두 구성 모두 중간 tile P·A 가 SRAM 에 머물러 DMA 는 같고 , 커널 경계의 descriptor·LOAD/SAVE 반복과 짧은 vector 행이 words 와 SRAM 작업 공간을 바꿉니다 .



커널 경계마다 : 새 descriptor 설정 + P/A 를 SRAM 에서 다시 LOAD/SAVE (DRAM 왕복은 없음)



한 커널 안에서 이어짐 : descriptor 반복 감소 , 축 결합으로 vector 행 64 → 128 (명령 수 절반), USMP offset 공유



설정	커널 수	DMA bytes	ISA words	SRAM peak	DRAM image	출력	Fusion 이 바꾸지 않는 것
off	3	41,100	2,812	50,700	41,100	bit-exact	FP16 저장 경계 (P, A 각 FP16) · vector 명령은 activation bit 로 바뀌지 않음 (OFF)
auto	1	41,100	1,577	42,508	41,100	bit-exact	수치 결과 (fused/unfused bit-exact) · 입력 · 출력 DMA

비교 기준은 커널 개수가 아니라 실제 프로그램의 (DMA bytes, ISA words, SRAM peak) 입니다 . E1 은 전송량이 같아도 명령과 작업 공간이 줄어든 사례입니다 .

근거 : docs/current/05_fusion.md, docs/examples/pipeline/stage_summary.json (E1)

TVM 은 무엇을 보고 묶는가 : op_pattern, FuseOps, FuseTIR

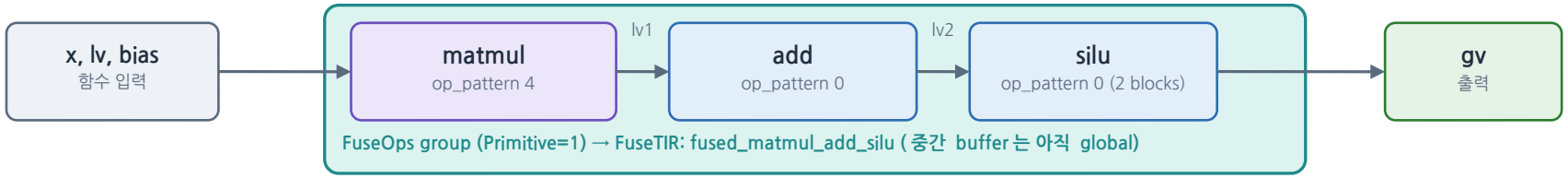
AnnotateTIROpPattern 이 TIR 의 연산 성격을 분류하고 , FuseOps 가 의존 그래프 ·post-dominator· 경로 pattern 으로 영역을 만들며 , FuseTIR 가 TIR 계산을 한 PrimFunc 로 결합합니다 .

값	이름	대표 의미
0	kElemWise	같은 위치끼리 원소별 (add, silu)
1	kBroadcast	작은 입력을 여러 위치에서 사용
2	kInjective	transpose 등 원소 대응이 명확한 변환
3	kCommReduce	합 / 최댓값 축소 (sum, softmax 내부)
4	kOutEWiseFusable	matmul 처럼 출력 뒤 원소별 계산을 붙일 수 있음
7	kTuple	tuple 구성 관련
8	kOpaque	일반 fusion 의 경계 (NPU 가 W8A8· 복합 matrix 영역에 부여)

Partitioner 가 검사하는 연결 (주요 분기)

출발 group	검사하는 경로 / 도착점	예
OutEWiseFusable	도착이 elemwise, 사이가 elemwise/broadcast 인가	matmul 출력에 add/activation
Elemwise/Broadcast	중간이 injective 이하 , 도착이 허용 pattern 인가	add → SiLU
Injective/Tuple	후속 차례에서 injective 이하 경로	reshape/transpose + 후속 계산
CommReduce	다른 복합 연산에 임의로 붙이지 않음	reduction 둘을 합치지 않음
Opaque	여기서 fusion 진행 없음	NPU 가 보호한 경계

작은 예제의 call graph 와 op_pattern → FuseOps 영역 → FuseTIR 결과



4(OutEWiseFusable) → 0 → 0: 출력 뒤 원소별 경로 → 한 영역 . transpose(2) 는 host 함수 (prefill_transform_params) 라 device 그래프에 없음 . 영역 채택 여부는 다음 슬라이드의 비용 검사가 결정 .

- FuseOps/FuseTIR 만으로 중간 buffer 가 SRAM 에 배치되지는 않음 → 뒤의 schedule 과 메모리 계획이 필요
- Attention 은 Relax pattern 이 보일 때 FuseOpsByPattern 으로 먼저 그룹을 정하고 LegalizeOps/FuseTIR 를 적용 . 두 경로는 같은 후보에 공존 가능

장치에서 빠른지 , SRAM 에 들어가는지는 이 표준 분류만으로 알 수 없습니다 . 영역 생성은 TVM, 채택 판단은 NPU 비용 검사입니다 .

근거 : docs/current/05_fusion.md, TVM 0.19 src/relax/transform/fuse_ops.cc, docs/examples/pipeline/03_unfused_semantic_tir.py

FuseOpsByPattern: attention 전체 또는 score chain 을 composite 로

패턴은 pattern 언어 (is_op/wildcard) 로 쓰고 , supported 검사 함수가 정적 rank-3 FP16 64 배수 조건을 확인합니다 . 조건 밖이면 더 작은 score fusion 으로 물러납니다 .

```
def _attention_patterns(tile=64):                       # tvm_pipeline.py
    def supported(context):
        q, k, v = (shape(name) for name in ("q", "k", "v"))   # FP16, rank 3, static
        if int(exprs["softmax"].attrs.axis) not in (-1, 2): return False
        h, m, d = q
        return (h > 0 and k[0] == h and v[0] == h and k[1] == d and v[1] == k[2]
                and all(x > 0 and x % tile == 0 for x in (m, d, k[2], v[2])))
    for scaled in (True, False):
        scores = is_op("relax.matmul")(q, k)
        if scaled: scores = is_op("relax.multiply")(scores, wildcard())
        masked  = is_op("relax.add")(scores, wildcard())
        softmax = is_op("relax.nn.softmax")(masked)
        output  = is_op("relax.matmul")(softmax, v)
        patterns.append(FusionPattern("npu.attention" if scaled else "npu.attention_unscaled",
                        output, {"q": q, "k": k, "v": v, "softmax": softmax}, supported))
```

패턴 이름	모양 / 조건
npu.attention(_unscaled)	matmul→(mul)→add→softmax→matmul. rank-3 FP16, m·d·t·dv 64 배수
npu.masked_softmax(_unscaled)	softmax(add(mul(scores,scale),mask)). 전체 패턴이 안 될 때의 fallback
npu.matrix_producer_{concat,repeat,expand_dims}	matmul(x, concat(…)) (+wrapper). decode KV concat 후보

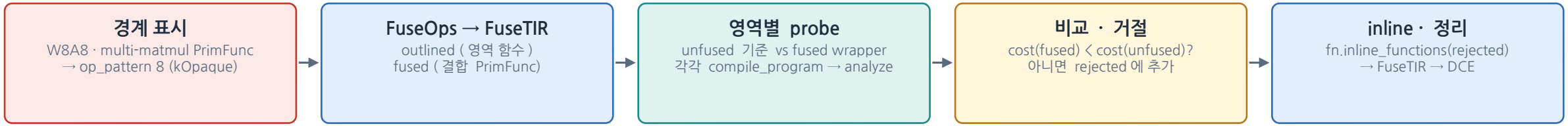
시도 순서 : 전체 attention 패턴을 먼저 , 안 되면 score chain 패턴
decode (m=1): 64 배수 조건 불만족 → full 패턴 대상 아님
옵션 : fuse_attention ∈ {False, "score", "full"}

왜 score chain 만이라도 묶는가 (코드 주석의 설계 근거): scale·mask·softmax 를 분리하면 [heads, seq, seq] score tensor 를 op 마다 한 번씩 훑습니다 . 전체를 묶고 query 행으로 block 하면 score matrix 가 SRAM 을 떠나지 않습니다 . 코드 주석의 seq 2048 관측 (layer 당 3.7 GB 중 1.6 GB, attention 1,057 MB → 235 MB) 은 과거 설계 근거이며 이번 실험 표의 수치가 아닙니다 .

구현된 후보와 실제 선택된 후보는 별개입니다 . 전체 Llama 검증 기록에서는 시간 한도 내에 unfused 기준 module 이 선택되었습니다 .

npu_fusion.select: 영역 하나를 실제 codegen 비용으로 검사한다

TVM 이 만든 FuseOps 영역마다 unfused(기준) 와 fused 를 schedule· 메모리 · 명령 생성까지 내려 비교하고 , 진 영역은 inline_functions 로 다시 펼칩니다 .




```
def _probe(module, function, profile):          # 영역 하나를 프로그램으로
    mod["__npu_fusion_probe"] = function.without_attr("Primitive")
    asm, plan = npu_link.compile_program(mod, "__npu_fusion_probe", ...)
    return analyze(asm.words, plan, detail=False)   # DMA/words/SRAM

reference = _probe(outlined, fn, profile)         # unfused: 필수 기준
candidate = _probe(fused, _wrapper(gv, fn), profile)
selected  = cost(candidate) < cost(reference)      # (dma, words, sram) 사전식
if not selected: rejected[gv] = fn
...  outlined[gv] = fn.inline_functions(rejected)   # 진 영역만 펼침
```

작은 예제의 실제 기록 (program.json → fusion.regions)

항목	unfused	fused
region	fused_matmul_add_silu	
dma_bytes	41,100	41,100
words	2,812	1,577
sram_peak_bytes	50,700	42,508
status		selected ("lexicographic DMA bytes, words, SRAM peak")

- unfused probe 실패 = unsupported 입력 또는 compiler 오류 → 오류를 숨기지 않음

부분 검사의 한계 : 한 영역의 fused/unfused 만 봅니다 . 주변 그래프의 모든 수명 · 다른 선택과의 조합은 모릅니다 . 그래서 다음 슬라이드의 전체 module 비교가 따로 필요하고 , 버린 영역을 모든 조합과 함께 재탐색하는 완전 탐색은 하지 않습니다 .

근거 : npu_compiler/npu_fusion.py select(), _probe(), docs/examples/pipeline/bundle/program.json fusion.regions

npu_build.lower_and_select: 같은 IR 인 source 를 먼저 제거하고 , 전체 entry 의 비용으로 module 하나를 고른다

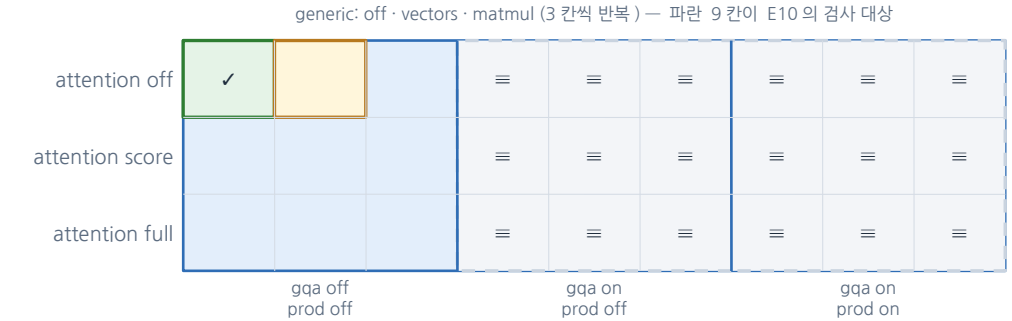
최대 27 개 조합 중 structural_equal 인 source 는 generic fusion 검사 전에 버립니다 (E10 prefill: source 9 → 3, 후보 9). 남은 후보는 entry 마다 컴파일한 (DMA, words, SRAM) 을 사전식으로 비교하며 , 첫 합법 module 뒤 추가 탐색은 기본 120 초 , None 이면 제한 없음입니다 .

```
for attention in (False, "score", "full"):                    # 3
  for gqa, producers in ((F,F), (T,F), (T,T)):                # × 3
    source = lower(attention, gqa, producers)                 # graph_pipeline
    if any(structural_equal(source, old) for old in sources): continue
    for generic in (False, "vectors", "matmul"):              # × 3 = 27
      candidate, regions = select(source, ...) if generic else (source, None)
      if duplicate(candidate): continue
      programs, stats, regressing = compile_all(candidate, best_costs)
      costs = {entry: cost(stats[entry]) for entry in entries}
      selected = regressing is None and (best_costs is None or
          (all(costs[e] <= best_costs[e] for e in entries)
           and any(costs[e] <  best_costs[e] for e in entries)))
      if selected: winner, best_costs = candidate, costs; arm_budget()

# E10 (prefill_cache, S=256): source 9 → 3 (GQA/producer 변환이 같은 IR) → 후보 9
# compile_all: 완료된 entry 가 incumbent 보다 나쁘면 즉시 중단 (noncompetitive)
# cost(stats) = (dma_bytes, words, sram_peak_bytes)   # v09_stats.cost
```

후보 기록 status	의미
legal	모든 entry 컴파일 완료 , 비용 기록 (Pareto 후보)
noncompetitive	한 entry 가 이미 더 나쁨 → 나머지 entry 생략
unsupported	LinkError / V09TirError (지원 밖 schedule 등)
duplicate	동일 IR 후보 (structural_equal). source 중복은 기록 없이 건너뜀
budget_exhausted	추가 탐색 예산 초과 (기본 120 s, None 이면 발생하지 않음)

27 조합 격자 — 회색 18 칸 = 같은 IR 인 source (E10 prefill)



✓ 후보 0 legal·selected (모두 off) 후보 1 (generic vectors) 검사 중 120 s 소진 : 과거 7-token 기록 (compile 189.7 s). ≡ E10 prefill 에서 source 가 같은 조합 . 예산은 첫 합법 module 뒤 시작하며 phase 경계에서 검사됩니다 .

두 단계가 다른 질문에 답합니다 : " 이 영역을 합칠 가치가 있는가 ?"(npu_fusion.select) 와 " 주변을 포함한 프로그램을 이렇게 선택해도 좋아지는가 ?"(lower_and_select). 중복 계산은 region/Entry cache 로 줄이고 , 시간 제한을 풀어도 (None) 부분 검사에서 버린 영역의 다른 조합까지 전수 탐색하지는 않습니다 .

근거 : npu_compiler/npu_build.py lower_and_select(), npu_compilation.optimization_budget, evidence/current/llama_selection.json (7-token 기록), llama_prefill256_full_search_progress.json (E10)

과거 7-token Llama 선택 기록 : 120 초 안에서 unfused 기준이 선택되었다

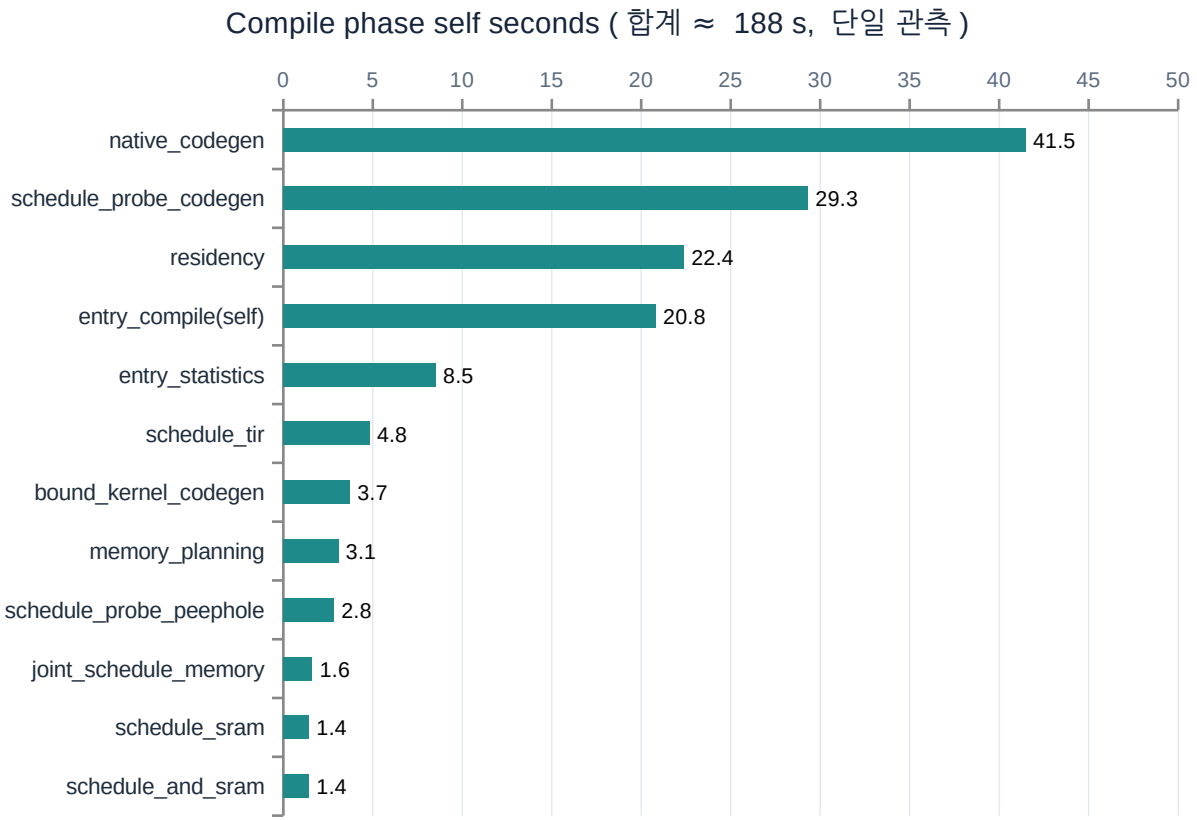
compile 189.7 s. 후보 0(모두 off) 이 legal·selected, 후보 1(generic vectors) 에서 추가 탐색 예산 120 s 가 소진 . " 모든 fusion 이 더 나쁘다 " 는 증거가 아닙니다 .

후보	attention	gqa/producers	generic	status
0	off	off / off	off	legal → selected
1	off	off / off	vectors	budget_exhausted
2~26	(미검사)			예산 소진

entry (후보 0)	words	DMA bytes	SRAM peak	image bytes
prefill_cache (S=7)	8,582,352	6,457,527,556	8,374,292	6,429,587,576
decode (C=9)	7,310,801	6,438,980,676	7,342,100	6,427,349,032

Cache 통계 (compilation.describe)

cache	hit	miss
schedule recipe (PrimFunc 구조별)	272	383
schedule trial (mapping 별)	184	248
cost probe	294	270
region (fusion 영역)	0	40
native_rejections	FP32 matrix chain interrupted	5

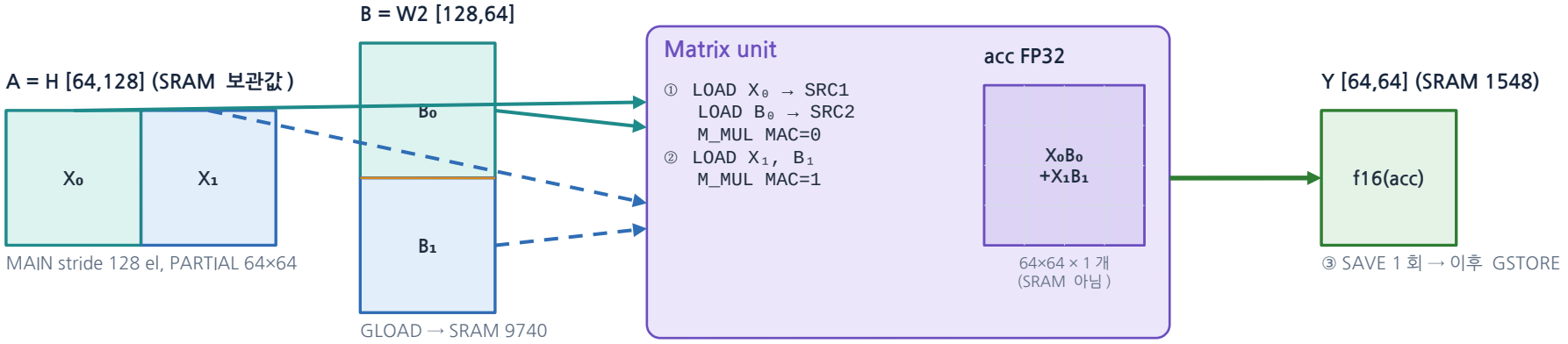


이 검증에서 채택된 것은 unfused 기준 module 입니다 . 비용이 큰 phase(native codegen·probe·residency) 가 곧 탐색 폭을 제한하는 요인입니다 .

근거 : docs/evidence/current/llama_selection.json (과거 7-token 보존 기록 ; 새 256-token E10 과 구분)

64×64 장치로 K=128 을 계산하기 : 출력 tile 하나의 K 누산은 끊지 않는다

작은 예제 ② 두 번째 matmul H[64,128] @ W2[128,64] 의 실제 명령 20 words. $X_0 \cdot B_0$ 로 accumulator 를 시작하고 (MAC=0), $X_1 \cdot B_1$ 을 같은 accumulator 에 더한 뒤 (MAC=1), 한 번만 FP16 으로 SAVE 합니다 .



K 반복이 출력 tile 반복 안에 있음 (K 최내측). M/N 이 크면 출력 tile 마다 ①②③를 반복하고 panel 내부 순서는 inner_order 가 정합니다 . npu_fill_zero 는 명령을 내지 않고 " 새 누산 시작 " 만 기록해 첫 M_MUL 이 MAC=0 이 됩니다 . activation bit 는 자동 경로에서 OFF 입니다 .

실제 명령 (program_memory.bin word 52-71, 20 words = 이 matmul 전체)

① X_0 → SRC1				B_0 → SRC2				② X_1 (+256 nib)				B_1 (+16384 nib)				③ Y 저장			
ADDR SRC1 M	COLS 128	ADDR SRC1 P	**LOAD SRC1**	ADDR SRC2 M	COLS 64	ADDR SRC2 P	**LOAD SRC2**	**M_MUL MAC=0**	ADDR SRC1 M	ADDR SRC1 P	**LOAD SRC1**	ADDR SRC2 M	ADDR SRC2 P	**LOAD SRC2**	**M_MUL MAC=1**	ADDR DST M	COLS 64	ADDR DST P	SAVE
52								60							67				71

두 번째 K tile 에서 COLS(0x89) 가 없는 이유 : rows/cols 가 바뀌지 않아 peephole 이 생략 . SRC1 MAIN 0xCC18 → 0xCD18 (+64 열), SRC2 MAIN 0x4C18 → 0x8C18 (+64 행 × 64 열). DST 0x0C18 = byte 1548 (X 자리 재사용).

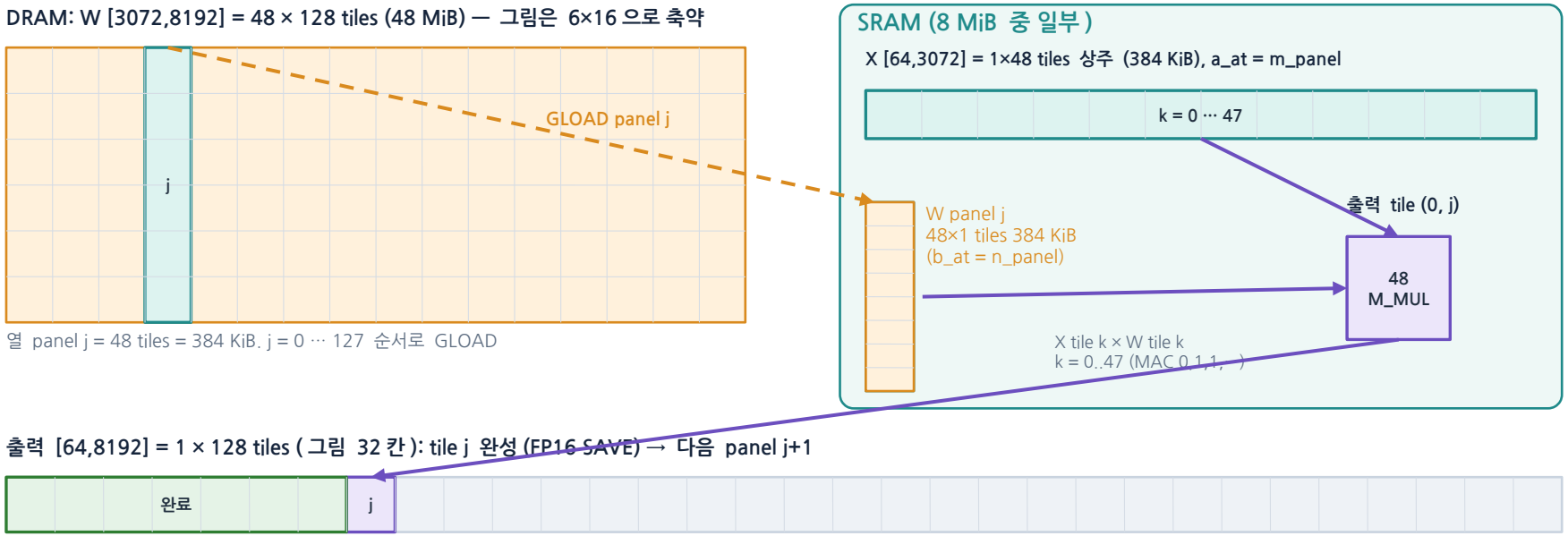
질문	답
왜 K 가 최내측인가 ?	accumulator 가 1 개이고 partial spill 금지 → 출력 tile 하나의 K chain 을 끝내고 다음 tile. M, N 이 크면 tile 마다 ①②③ 반복

Schedule 의 자유도는 tile 안이 아니라 tile 바깥에 있습니다 : 어떤 panel 을 SRAM 에 얼마나 오래 두고 , 어떤 순서로 출력 tile 을 돌 것인가 .

근거 : docs/examples/two_matmuls_recorded/program.json (words 52-71), scheduled_0001.py, docs/current/06_matmul_schedule.md

SRAM 에 올리는 panel 은 matrix tile 보다 크다 : 실제 폭 gate projection

X[64,3072] @ W[3072,8192]. W 는 48 MiB = 48×128 tiles 라 SRAM 에 들어가지 않으므로 X(1×48 tiles, 384 KiB) 를 상주시키고 W 의 열 panel j(48×1 tiles, 384 KiB) 를 128 번 교체합니다 .



출력 tile 하나 : M_MUL 48 개 (K tile 48)
출력 열 tile: 8192/64 = 128 → 총 6,144 M_MUL
W 전송 : 128 panel × 384 KiB = 48 MiB, 각 1 회
X 전송 : 384 KiB 1 회
SRAM: X 384 KiB + W panel 384 KiB + 출력 tile + 임시 (나머지는 보관값 · 작업 공간)

Projection	계산	보존된 선택 (M=64)	입력 배치	M_MUL
gate / up	[64,3072] × [3072,8192]	("A", 64): input-stationary	X 384 KiB 유지 , W [3072,64] 384 KiB × 128 회 교체	6,144 씩
down	[64,8192] × [8192,3072]	("B", 256): weight-stationary	W [8192,256] 4 MiB 유지 , 출력 64 열 tile 4 개 계산 . H 는 앞 커널 SRAM 보관값	12 panel × 4 tile × 128 K = 6,144

K chunk 숫자만 보고 DMA 크기를 판단하면 안 됩니다 . cache stage 가 어느 loop 에 있는지 (a_at/b_at) 가 실제 전송 영역과 재읽기 횟수를 정합니다 .

탐색 공간 : m/n panel, K chunk, loop 순서 , 그리고 두 operand 의 cache 위치

npu_schedule_space.candidates: 64×64 명령과 끊기지 않는 K chain 은 제약이지 mapping 템플릿이 아닙니다 . 출력 tile 을 도는 순서와 각 operand 의 cache 위치가 재읽기 횟수를 정합니다 .

설정	의미
m_panel / n_panel	바깥에서 묶는 출력 행 / 열 수 . 그 안은 64 단위 tile
k_chunk	K 를 묶는 크기 . 그 안에서 64 개씩 M_MUL
outer_order	M panel 과 N panel 중 무엇을 먼저 반복
inner_order	panel 내부 M/N tile 순서
a_at / b_at	X/W 복사 stage 를 넣을 loop. 그 안에서 반복 사용하는 영역이 SRAM 에 올라감
resident	이미 SRAM 에 있는 operand: cache stage 없음 , 위치 선택 1 개로 축소

출력 [128,256] = 2×4 tiles (panel 64) 를 도는 순서 — 무엇이 SRAM 에 남아 재사용되는가

1	2	3	4
5	6	7	8

outer_order = mn: X 행 panel 고정 , W 열 panel 교체 → X 재사용 , W 는 M panel 수만큼 재읽기

1	3	5	7
2	4	6	8

outer_order = nm: W 열 panel 고정 , X 행 panel 교체 → W 재사용 , X 는 N panel 수만큼 재읽기

→

추정 `DMA = Σ 2·K·(M|N)·repeats (operand` 가 쓰지 않는 축의 바깥 `loop` 반복 곱) → 검사 순서만 결정

loop_spec: 항상 이 구조 (K 최내측)

```
for m_panel / n_panel:                          outer_order
    for n_panel / m_panel:
        for m_tile (64):                        inner_order
            for n_tile (64):
                for k_group (k_chunk):
                    for k_tile (64) → M_MUL:
```

a_at = "m_panel" 이면 X 복사 stage 가 m_panel loop 안 , 즉 그 panel 의 X 행 전체 (K 전체 포함) 를 한 번 올림 . b_at = "k_group" 이면 W 는 K chunk 마다 다시 올림 .

실제 폭 예	개선 전 후보 (기록)	현재 정규화 공간 (계산)	기록의 선택
gate/up [64,3072]×[3072,8192]	1,433	203 → 증명 제외 41 → 162	합법 2 개 뒤 word 예산 → incumbent ("A",64)
down [64,8192]×[8192,3072], H resident	299	17 → 제외 5 → 12	("B",256) 선택 , 43,410 words
작은 예제 [64,128]×[128,64]	4	4 (모두 추정 32,768 B)	a_at = b_at = m_panel 선택

" 개선 전 " 은 09-28 compile 기록 (refinement.json), " 현재 " 는 candidates()/prioritize() 계산값 (새 compile 기록 아님).

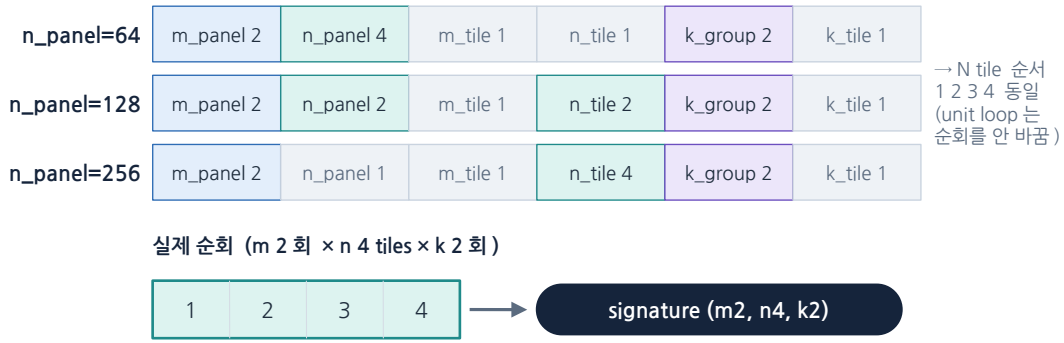
Factor 는 padding 한 축 길이의 64 배수 약수에서 만듭니다 (N=256 → 64/128/256). 같은 순회의 다른 표기는 합치고 , 증명된 용량 초과만 미리 빼며 , 나머지는 예산으로 제한합니다 (다음 두 슬라이드).

근거 : npu_compiler/npu_schedule_space.py (factors, loop_spec, candidates), evidence/current/refinement.json (09-28 기록)

후보 정규화와 용량 필터 : 같은 tile 순회 · 복사 주기는 한 후보로 , 증명 가능한 SRAM 초과만 미리 제외

npu_schedule_space.candidates / prioritize (09-29 개선). 512-token down [512,8192]×[8192,3072]: 22,236 → 5,034 후보 , 그중 1,596 은 두 입력 cache 의 합만으로 8 MiB 를 넘어 제외 , 3,438 을 검사 목록에 남김 (E9 기록).

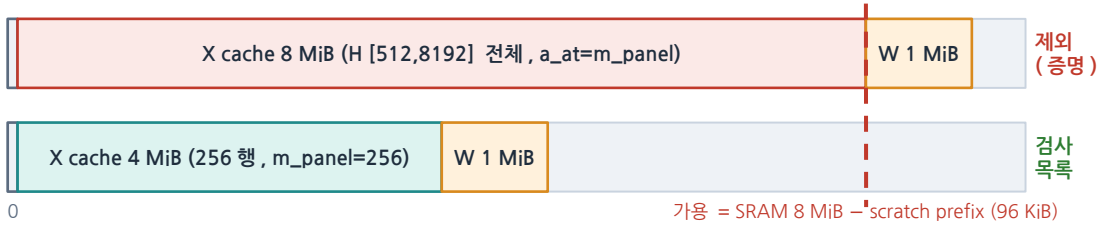
① 같은 순회의 다른 표기는 signature 하나로 — toy [128,256]×K128 (2×4 tiles), N 축의 세 표기



cache 위치는 그 loop 까지의 prefix signature 로 구분 : b_at = n_panel 이면 n_panel=64 는 tile 마다 , 128 은 2 tile 마다 , 256 은 한 번 복사 → 복사 주기가 다르므로 서로 다른 후보로 유지 . 순회와 복사 주기가 모두 같은 표기끼리만 합칩니다 .

toy 전체 : loop 위치 조합 1,728 → 후보 73 (현재 코드 계산). resident operand 는 cache stage 가 없어 위치 1 개로 축소 .

② 용량 하한 증명 : 두 입력 cache 는 matrix 명령 시점에 동시에 살아 있다



증명 조건 (ordinary): 두 operand 가 서로 다른 함수 parameter, FP16, scope global/global.sram, shape 정확히 (M,K)/(K,N), M·N·K 모두 64 배수 → 두 cache bytes 합 > 가용이면 제외

해당 안 되면 (padding, computed producer, W8A8): footprint 추정으로 순서만 뒤로 (deprioritize), 제거하지 않음

출력 ·scratch·epilogue 추정은 제외 근거가 아님 : 남은 후보는 실제 TIR compaction·SRAM 배치 · 명령 생성으로 검사

계산 (M,K)×(K,N)	개선 전 후보	정규화 후	용량 증명 제외	검사 목록	출처
512-token down (512,8192)×(8192,3072)	22,236	5,034	1,596	3,438	E9 기록
gate M=64 (64,3072)×(3072,8192)	1,433	203	41	162	현재 코드 계산
down M=64, H resident (64,8192)×(8192,3072)	299	17	5	12	현재 코드 계산
작은 예제 (64,128)×(128,64)	4	4	0	4	기록 = 계산

" 개선 전 " 은 정규화 전 candidates() 기록 (E8), " 계산 " 은 현재 코드를 scratch 96 KiB 가정으로 실행한 값이며 새 compile 기록이 아닙니다 .

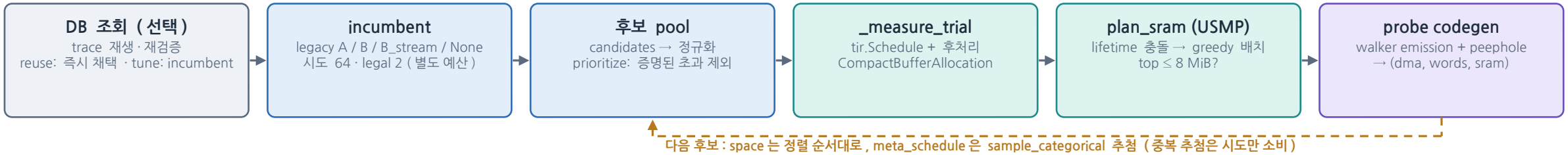
E8 에서 본 문제 : 추정 DMA 만으로 정렬하면 SRAM 에 안 들어가는 후보가 앞을 차지 → 512-down 확장 탐색 16 회가 모두 용량 초과 . 개선 후 같은 커널 : 4 회 검사 = 합법 4 개 (E9).

정규화는 표기 중복을 없애고 , 용량 필터는 증명된 경우만 제외합니다 . 남은 후보는 여전히 실제 TIR compaction·SRAM 배치 · 명령 생성으로 검사합니다 .

근거 : npu_compiler/npu_schedule_space.py (_iteration_signature, candidates, prioritize), npu_schedule.py (_fit_matmul 의 ordinary 판정), docs/current/06_matmul_schedule.md, 11_experiments.md E8/E9

후보 검사 흐름 : DB 재생 (선택) → incumbent → 정규화된 후보를 실제 schedule·SRAM 배치 · 명령 생성으로 검사

_fit_matmul. 실패 · 용량 초과도 시도로 세고 , words 한도는 기준 후보 크기에 맞춰 늘어나며 , 합법 후보 2 개를 보기 전에는 words 로 끝내지 않습니다 . 검사 순서만 space(추정 DMA 순) 와 meta_schedule(ReplayFunc 추첨) 이 다릅니다 .



종료 사유 (diagnostics.stop_reason) — 먼저 오는 조건에서 멈춤

database_hit	legal_budget	word_budget	attempt_budget	exhausted
reuse 모드 DB 적중 → 새 후보 0 회	legal ≥ 8 (schedule_space_trials)	legal ≥ 2 ∧ Σwords ≥ max(100k, 기준 words×8)	검사 수 ≥ 64 (실패 · 용량 초과 포함)	후보 목록 끝 (attempts=None 이면 여기까지)

후보 상태	원인	예
schedule_rejected	Schedule API 실패 , lower_stores 규칙 위반 , ReplayFunc 중복 추첨	FP32 buffer 의 비정형 store · "duplicate sample"
capacity_rejected	USMP top > SRAM 용량 (보관 예약 제외한 cap)	큰 panel 조합
codegen_rejected	walker 가 MAC chain 중단 감지 (LinkError)	"FP32 matrix chain interrupted"
legal → selected	명령 생성 성공 → (dma, words, sram) 사전식 최소	incumbent 보다 작을 때만 교체

예산 (NpuProfile 기본값)	값	뜻
schedule_search	"space"	"meta_schedule"=ReplayFunc 추첨 · "legacy"= 기준만
schedule_max_attempts	64	실패 · 용량 초과 포함 . incumbent/ 확장 각각 . None= 무제한
schedule_space_trials	8	확장 탐색의 legal 목표 (E10 실행기 16)
schedule_candidates	2	legacy incumbent 탐색의 legal 목표
schedule_probe_word_budget	100,000	max(설정 , 기준 words×legal 목표), 후보 완료 후 검사
schedule_seed	0	ReplayFunc 추첨 seed (rand_state)

" 예산 내 합법 후보 없음 "(attempt budget exhausted … within budget) 과 " 원리상 불가능 "(no legal matmul schedule fits) 은 다른 메시지입니다 . trial cache 는 mapping 키로 결과 (실패 포함) 를 재사용하고 용량은 키 밖이라 다른 보관 예약에서도 재사용됩니다 .

추정 DMA 와 추첨은 검사 순서만 정합니다 . 합법성과 비용은 실제 scheduled TIR, buffer compaction, SRAM 배치 , 명령 생성으로 확인합니다 .

근거 : npu_compiler/npu_schedule.py (_fit_matmul, _measure_trial, _probe_accumulator_chain), npu_schedule_space.prioritize, device.NpuProfile

표준 Schedule API 가 실제로 바꾸는 것과 그 자체로 하지 않는 일

tir.Schedule 은 TIR 를 받아 block/loop 변환을 적용합니다 . 변환 뒤 loop 구조가 달라지므로 이전 handle 을 그대로 쓰지 않습니다 .

API	실제 변화	그 자체로 하지 않는 일
split	loop 하나를 바깥 / 안쪽 loop 로 분할 (predicate 포함)	SRAM 복사 생성
reorder	의존성이 허용하는 loop 순서 변경	DMA 비용 자동 최소화
cache_read	입력을 새 cache buffer 에 복사하는 stage 생성	복사의 적절한 반복 위치 결정
compute_at	생산 block 을 소비 loop 의 특정 위치 안으로 이동	ISA 명령으로 남기기
reverse_compute_at	소비 block 을 생산 loop 위치로 이동	무조건 in-place 실행
cache_write	출력 계산이 cache 를 쓰고 원래 출력으로 복사	최종 출력 ABI 제거
set_scope	기존 buffer 의 storage scope 변경	별개 복사 stage 추가
pad_einsum	einsum 축을 tile 배수로 확장 (ragged shape)	결과 의미 변경 (zero padding 증명 유지)
decompose_reduction	init 과 update 분리 → 어느 loop 에서 누산 시작인지	수치 순서 변경
tensorize	일치하는 조각을 등록한 intrinsic 으로 치환	임의 수식의 자동 ISA 구현

schedule_matmul_sram 의 실제 호출 순서

- 계약 검사 : A/B FP16, C FP32, 소비자는 FP16 cast 하나 , epilogue 는 shape/FP16 유지
- pad 등 다른 producer: cache_read + set_scope(SRAM); ragged 면 pad_einsum
- split(i,[None,m_panel]) → split(·,[None,64]); j, k 동일 → reorder(loop_spec 순서 , ii, ji, ki)
- cache_write(C, SRAM) → set_scope → reverse_compute_at(enclosing tile loop); unpad·FP16 store 도 reverse_compute_at
- fused epilogue(add/SiLU): reverse_compute_at + set_scope/cache_write; 입력은 cache_read + compute_at
- compute_inline(unpad, write_back)
- operand 0/1: cache_read → compute_at(a_at / b_at). SRAM resident 면 생략
- decompose_reduction(block, k_group) → init block
- tensorize(k_tile loop, "npu_gemm_acc_sram"); tensorize(init, "npu_fill_zero_sram")
- 후처리 : LowerInitBlock → PlanAndUpdateBufferAllocationLocation → ConvertBlocksToOpaque → CompactBufferAllocation → lower_stores → forward_output_tiles → pointwise coalescing

compute_at 등은 IR 을 바꾸는 명령이라 결과 TIR 에 그 이름이 남지 않습니다 . 대신 loop 안으로 이동한 block 과 좁아진 buffer 를 볼 수 있습니다 .

근거 : docs/current/06_matmul_schedule.md, reference/tvm_api.md, npu_compiler/npu_intrin.py schedule_matmul_sram

compute_at 을 작은 예제로 보기 : 필요한 생산을 사용 위치로 옮긴다

B = A + 1, C = B × 2 (4×4 FP32 CPU 교육 예제 , NPU tile 이 2 라는 뜻은 아님). C 의 행 loop 를 2 행씩 나눈 뒤 B 를 그 안으로 옮깁니다 .

변환 전 (01_before)

```
B = T.alloc_buffer((4, 4))
for i, j in T.grid(4, 4):
  with T.block("B"):
    B[i, j] = A[i, j] + 1.0
for i, j in T.grid(4, 4):
  with T.block("C"):
    C[i, j] = B[i, j] * 2.0

# B 전체(4×4) 생성 후
# C 전체 생성 → B 수명 김
```

compute_at 뒤 (02)

```
for i_0 in range(2):        # 2행 묶음
  for ax0, ax1 in T.grid(2, 4):
    with T.block("B"):
      v_i = i_0*2 + ax0
      B[v_i, ax1] = A[v_i, ax1] + 1.0
  for i_1, j in T.grid(2, 4):
    with T.block("C"):
      v_i = i_0*2 + i_1
      C[v_i, j] = B[v_i, j] * 2.0
# 묶음마다 필요한 B 2행만 생성
```

reverse_compute_at 뒤 (03)

```
for i_0 in range(2):
  for i_1, j in T.grid(2, 4):
    with T.block("B"):       # 생산자 기준
      B[i_0*2+i_1, j] = A[..] + 1.0
  for ax0, ax1 in T.grid(2, 4):
    with T.block("C"):       # 소비자를 끌어옴
      C[i_0*2+ax0, ax1] = B[..] * 2.0

# compute_at: 생산자를 소비 위치로
# reverse_compute_at: 소비자를 생산 위치로
```

적용한 API trace (02_compute_at.trace.txt)

```
b0 = sch.get_block(name="C", func_name="main")
l1, l2 = sch.get_loops(block=b0)
l3, l4 = sch.split(loop=l1, factors=[None, 2], preserve_unit_iters=True, disable_predication=False)
b5 = sch.get_block(name="B", func_name="main")
sch.compute_at(block=b5, loop=l3, preserve_unit_loops=False, index=-1)
```

NPU 에서의 쓰임 : matmul 출력 tile 근처에 add/SiLU(epilogue) 를 reverse_compute_at 으로 배치하고 , 입력 cache stage 는 compute_at 으로 a_at/b_at loop 에 넣습니다 .

근거 : docs/examples/schedule_api/01_before.py, 02_compute_at.py, 02_compute_at.trace.txt, 03_reverse_compute_at.py

cache_read → compute_at → CompactBufferAllocation: 복사 buffer 가 실제 접근 범위로 줄어든다

cache_read 만 추가하면 입력 전체를 복사하는 buffer 가 생깁니다 . 복사 stage 를 행 묶음 안에 넣으면 필요한 영역이 좁아지고 , compaction 이 allocation 을 그 범위로 축소합니다 .

① cache_read(C, 0, "global.sram")

```
A_global_sram = T.alloc_buffer(
    (4, 4), scope="global.sram")
for ax0, ax1 in T.grid(4, 4):
  with T.block("A_global.sram"):
    A_global_sram[v0,v1] = A[v0,v1]
for i, j in T.grid(4, 4):
  with T.block("C"):
    C[i,j] = A_global_sram[i,j] + 1.0
```

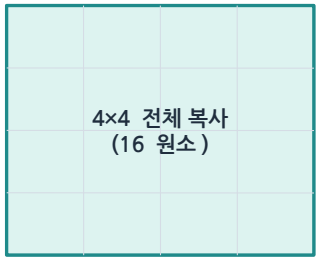
② compute_at(copy stage, i_0)

```
A_global_sram = T.alloc_buffer(
    (4, 4), scope="global.sram")   # 아직 4×4
for i_0 in range(2):
  for ax0, ax1 in T.grid(2, 4):
    with T.block("A_global.sram"):
      v0 = i_0*2 + ax0
      A_global_sram[v0,v1] = A[v0,v1]
  for i_1, j in T.grid(2, 4):
    with T.block("C"):
      C[i_0*2+i_1, j] = A_global_sram[..] + 1.0
```

③ CompactBufferAllocation

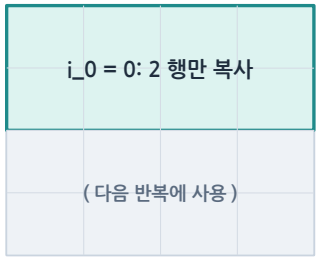
```
for i_0 in range(2):
  with T.block(""):
    T.reads(A[i_0*2:i_0*2+2, 0:4])
    A_global_sram = T.alloc_buffer(
        (2, 4), scope="global.sram")  # 2×4
    for ax0, ax1 in T.grid(2, 4):
      A_global_sram[i_0*2+ax0 - i_0*2, ax1]
          = A[i_0*2+ax0, ax1]
    for i_1, j in T.grid(2, 4):
      C[i_0*2+i_1, j] = A_global_sram[..] + 1.0
```

SRAM cache buffer 의 실제 모양 (진한 칸 = 한 반복 (i_0 = 0) 에서 채워지는 영역)



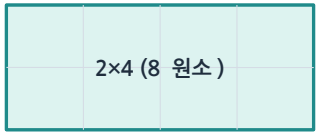
buffer (4,4): 입력 전체

→ compute_at



buffer 선언은 아직 (4,4)

→ compaction



buffer (2,4): 접근 범위로 축소 → USMP 가 이 크기로 offset 배치

Matmul 에서 : a_at = m_panel → X cache [m_panel, K] (K 전체), a_at = k_group → [m_panel, k_chunk]. 같은 원리로 panel 크기가 정해집니다 .

set_scope(기존 buffer scope 변경) 와 cache_write 예제도 같은 폴더 (docs/examples/schedule_api) 에 있습니다 .

근거 : docs/examples/schedule_api/04_cache_read.py, 05_cache_read_compute_at.py, 08_cache_read_compacted.py

Tensorize: 64×64 계산 조각을 등록한 TensorIntrin 으로 치환한다

수식 ·dtype·scope 가 일치하면 npu_gemm_acc_sram / npu_fill_zero_sram 으로 바뀝니다 . 이는 최종 opcode 가 아니라 codegen 이 descriptor·LOAD·M_MUL 로 펼칠 call_extern 입니다 .

TensorIntrin 등록 (description ↔ implementation)

```
# description: 계산의 수식 (schedule 이 이 모양을 찾음 )
A = T.match_buffer(a, (64,64), "float16", scope="global.sram", strides=[sa,1],
offset_factor=1)
B = ... (64,64) "float16" ; C = ... (64,64) "float32"
for i, j, k in T.grid(64, 64, 64):
    C[vi, vj] = C[vi, vj] + Cast(f32, A[vi, vk]) * Cast(f32, B[vk, vj])

# implementation: 장치 호출 (codegen 이 해석 )
T.call_extern("int32", "npu_gemm_acc", C.access_ptr("rw"), sc,
              A.access_ptr("r"), sa, B.access_ptr("r"), sb)
# fill: C[vi,vj] = 0  →  T.call_extern("npu_fill_zero", C.access_ptr("w"), sc)
TensorIntrin.register("npu_gemm_acc_sram", desc, impl)   # register_all()
```

실제 scheduled TIR 발췌 (작은 예제 fused 커널)

```
x_global_sram  = T.alloc_buffer((64,128), "float16", scope="global.sram")
lv_global_sram = T.alloc_buffer((128,64), "float16", scope="global.sram")
... x/lv 복사 block (K loop 바깥 : a_at = b_at = m_panel) ...
with T.block("matmul_init_o"):
    C = T.match_buffer(matmul_global_sram[0:64, 0:64], (64,64), "float16",
strides=("C_s0",1), ...)
    T.call_extern("int32", "npu_fill_zero", T.tvm_access_ptr(..., C.data, ...),
C.strides[0])
for k_0, k_1_0 in T.grid(2, 1):
    with T.block("matmul_update_o"):
        A = T.match_buffer(x_global_sram[0:64, k_0*64 : k_0*64+64], (64,64), ...)
        B = T.match_buffer(lv_global_sram[k_0*64 : k_0*64+64, 0:64], (64,64), ...)
        T.call_extern("int32", "npu_gemm_acc", C.ptr, C.strides[0], A.ptr, A.strides[0],
B.ptr, B.strides[0])
```

이름	어디에 나타나는가	뜻
npu_gemm_acc_sram / npu_fill_zero_sram	TensorIntrin 이름 (tensorize 인자)	등록된 desc/impl 쌍
npu_gemm_acc / npu_fill_zero	scheduled TIR 의 call_extern	codegen 이 descriptor 설정 + LOAD SRC1/SRC2 + M_MUL(MAC) 로 , fill 은 명령 없이 " 새 누산 " 상태
C 의 dtype 이 FP16?	match_buffer 가 FP16 SRAM view	lower_stores 가 FP32 논리 buffer 를 FP16 물리 slot 으로 retype (실제 FP32 상태는 accumulator)
strides=("C_s0",1)	symbolic row stride	64×64 view 가 더 넓은 panel/ 출력 안에 위치 가능 (직접 저장의 근거 , 8 장)

Tensorize 는 등록한 수식과 정확히 일치하는 조각만 치환합니다 . 틀린 수식을 이름만으로 치환하지 않습니다 .

근거 : npu_compiler/npu_intrin.py (_sram_gemm_desc/_impl, register_all), docs/examples/pipeline/07_scheduled_fused_matmul_add_silu.py

실제 선택 기록 : 작은 예제의 4 개 후보와 실제 폭 MLP 의 incumbent

작은 예제는 X/W 16 KiB 씩을 K loop 바깥에서 한 번 올리고 2 개의 M_MUL 이 같은 SRAM 배열의 다른 부분을 읽습니다 . 실제 폭 MLP 는 정적 compile 기록입니다 (executed=false). 후보 수는 개선 전 탐색 공간의 기록이며 현재 정규화 공간에서는 203 / 17 개입니다 .

작은 예제 fused_matmul_add_silu [64,64,128]

후보	mapping	추정 B	words	DMA B	SRAM nib	status
incumbent	("B_stream", 64)	32,768	1,574	41,088	68,632	legal
space #1	m64 n64 k64, a_at=b_at=k_group	32,768	1,575	41,088	68,632	legal
선택	m64 n64 k64, mn/mn, a_at=b_at=m_panel	32,768	1,572	41,088	85,016	legal ✓

- DMA 가 같으면 words 로 , 그 다음에야 SRAM peak 로 비교 → SRAM 을 더 써도 words 가 적은 후보가 이김
- a_at=b_at=m_panel: X [64,128]·W [128,64] 16 KiB 씩 K loop 바깥에서 1 회 복사 , k_0=0,1 의 두 M_MUL 이 같은 SRAM 배열의 다른 열 / 행을 읽음
- 최종 커널 : 4 DMA packets, 2 M_MUL, 1,572 words (link 뒤 전체 1,577 words)

실제 폭 Llama MLP (M=64, hidden 3072, MLP 8192) 정적 compile

커널	선택 mapping	후보 수 (개선 전)	words	비고
gate	("A", 64) incumbent	1,433	≈47,000	X 유지 , W panel 3072×64. legal 2 개 뒤 word 예산 도달
up	("A", 64)	1,433	≈47,000	gate 와 같은 구조 (recipe 공유)
down	("B", 256)	299	43,410	W 8192×256 4 MiB 유지 , H 는 SRAM resident

전체 (5 kernels: gate → SiLU → up → mul → down)	값
ISA words	140,594
DMA bytes	152,174,604 (≈145 MiB; weight 3×48 MiB + X/H 이동)
SRAM peak bytes	7,446,540 (8 MiB 중)

Gate 하나의 M_MUL 수 : 출력 열 tile 128 × K tile 48 = 6,144. 세 projection 합 18,432 = mlp64.json 의 opcode 0x42 count(18,432) 와 일치합니다 . 0x80(주소 descriptor) 82,093 words 와 0x90(LOAD) 37,504 words 가 프로그램의 대부분을 차지하며 , 이것이 " 정적 전개 명령 stream" 의 비용 구조입니다 .

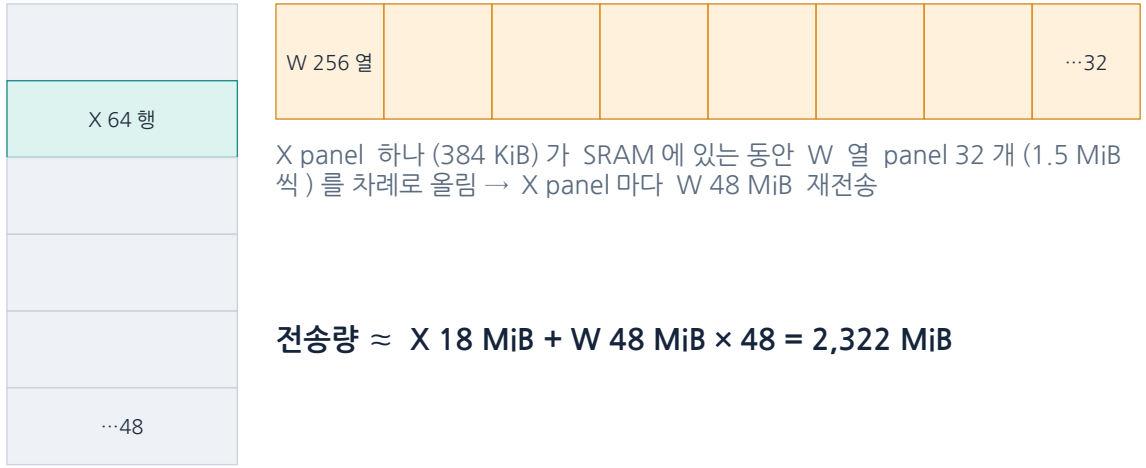
실제 폭에서는 후보가 수백 ~ 수천 개라 예산이 먼저 끝나고 incumbent 가 남는 경우가 있습니다 . 예산 안의 결과이지 전 공간의 최적이 아닙니다 .

근거 : docs/examples/pipeline/bundle/program.json schedules, evidence/current/refinement.json, mlp64.json

긴 prefill(S=3072) 에서 X 와 W 가 모두 큰 경우 : 재읽기 횟수가 비용을 정한다

설명용 계산이며 새 compile 측정이 아닙니다 . X 18 MiB, gate W 48 MiB. X 는 64 행 (384 KiB), W 는 256 열 (1.5 MiB) 씩 올린다고 가정합니다 .

X panel 을 바깥에 유지 (a_at 바깥 , b_at 안쪽)



X panel 하나 (384 KiB) 가 SRAM 에 있는 동안 W 열 panel 32 개 (1.5 MiB 씩) 를 차례로 올림 → X panel 마다 W 48 MiB 재전송

전송량 ≈ X 18 MiB + W 48 MiB × 48 = 2,322 MiB

W panel 을 바깥에 유지 (b_at 바깥 , a_at 안쪽)



W panel 하나 (1.5 MiB) 가 SRAM 에 있는 동안 X 행 panel 48 개 (384 KiB 씩) 를 차례로 올림 → W panel 마다 X 18 MiB 재전송

전송량 ≈ W 48 MiB + X 18 MiB × 32 = 624 MiB

가정	바깥에 유지	재읽기 때문에 생기는 전송	합계 (설명용)
X 64 행 panel, W 256 열 panel	X panel	X 18 MiB + W 48 MiB × 48 회	≈ 2.3 GiB
X 64 행 panel, W 256 열 panel	W panel	W 48 MiB + X 18 MiB × 32 회	≈ 0.6 GiB
더 큰 panel (예 : W 1024 열 6 MiB)	W panel	W 48 MiB + X 18 MiB × 8 회	≈ 0.19 GiB (SRAM 8 MiB 중 나머지가 출력 · 임시 · 보관에 필요)

SRAM 을 채우는 양보다 재사용 범위와 재읽기 횟수가 중요합니다 . 실제 탐색은 여기에 앞 커널의 SRAM 입력 (보관값) 까지 포함해 비교합니다 .

누산 체인 검사 : codegen 이 MAC chain 을 보장하고 , lower_stores 가 FP32 buffer 사용을 제한한다

"SRAM 에 공간이 충분하다 " 만으로 schedule 이 실행 가능한 것은 아닙니다 . 출력 tile 의 K 누적을 끊는 후보는 세 단계 중 하나에서 거절됩니다 .

```
# tir_codegen_v09.Walker (Python; native v09_walker.cc 동일 계약 )
def call("npu_fill_zero", C):  self.zeroed.add(C)    # 명령 없음
def emit_gemm(C, A, B):
    if C in zeroed:      flush(); pending = C; first = True   # MAC=0
    elif pending == C:   first = False                         # MAC=1
    else: raise LinkError("FP32 matrix chain interrupted:
                           resuming from an FP16 store would change the reduction")
    region(SRC1, A); LOAD(matrix, SRC1); region(SRC2, B); LOAD(matrix, SRC2)
    M_MUL(mode=VECTOR, activation=OFF, mac=not first)
def flush():   # 완료된 tile 을 FP16 으로 SAVE
    region(DST, pending); SAVE(matrix=1)
# flush 시점 : vector op / reduction / SRAM→SRAM 이동 / write-back DMA 전 , kernel 끝
# ( 입력 GLOAD staging 은 flush 하지 않음 )
```

chain 을 끊는 예	왜
K step 사이의 pad fill / dequantize (vector op)	vector 명령 전 flush → 누산 상태 손실
K step 사이의 SRAM→SRAM copy, bounce DMA	이동 전 flush
두 출력 tile 을 번갈아 누산	accumulator 1 개 : 다른 C 가 오면 LinkError

검사 층	규칙	위반 시
npu_accumulator.lower_stores	FP32 buffer store 는 copy / zero / acc + Cast(f32, fp16) 만 . load 는 FP16 cast 아래에서만	schedule_rejected ("unsupported FP32 accumulator use", "escapes without FP16 store")
Walker.emit_gemm	한 C tile 의 K chain 사이에 flush 를 일으키는 명령 금지	codegen_rejected (LinkError, native: invalid_schedule → fallback 없음)
npu_sram.plan_sram	gemm/fill 목적지 buffer 는 pinned: 모든 buffer 와 충돌로 취급	SAVE 지연 중 덮어쓰기 방지
forward_output_tiles	출력이 SRAM 에 보관되면 fill+gemm view 를 출력에 직접 → identity copy 제거	조건 불충족 시 임시 tile 유지

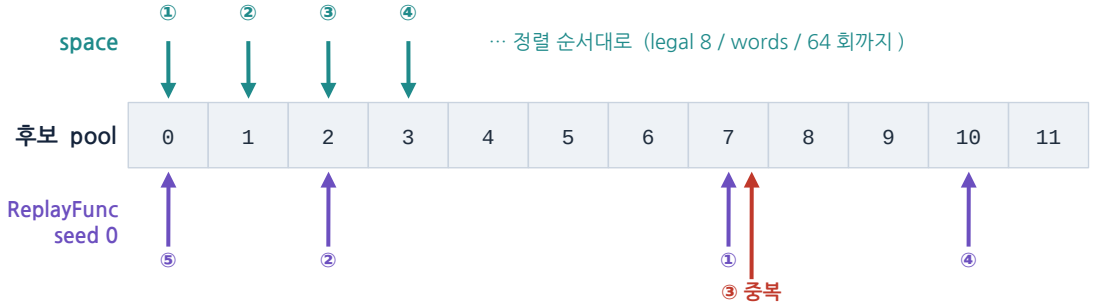
기록 : 전체 Llama compile 에서 native_rejections["FP32 matrix chain interrupted"] = 5. 이런 후보는 조용한 fallback 이 아니라 invalid_schedule 로 거절되어 다음 후보로 넘어갑니다 . 내부 오류 · 계약 hash 불일치는 별도로 propagate 됩니다 .

근거 : npu_compiler/tir_codegen_v09.py (emit_gemm, flush), npu_accumulator.py (lower_stores, forward_output_tiles), npu_sram.py (pinned)

MetaSchedule 연결 (선택형): ReplayFunc 추첨 순서와 JSONDatabase trace 재생

기본은 space(추정 DMA 순). schedule_search="meta_schedule" 는 같은 후보 pool 에서 sample_categorical 로 번호를 뽑고 , compilation(schedule_database=…) 는 실행 조건이 같은 커널의 trace 를 저장 · 재검증합니다 . 학습형 cost model·latency 측정은 없습니다 .

같은 pool (정규화 · 용량 필터 후 , 추정 DMA 순 index) — 검사 순서만 다르고 평가는 공통



sample_categorical(range(N), 균등) → 번호 → 그 mapping 으로 schedule_matmul_sram. 중복 번호는 schedule 을 다시 만들지 않고 시도만 소비 . 시도 한도는 유한해야 함 .

schedule_matmul_sram → _lower_scheduled → plan_sram · 용량 → probe codegen → (dma, words, sram)

TuneContext(mod, target=npu, ScheduleFn(make_schedule, sch_rules=[], postprocs=[]), ReplayFunc(), rand_state=seed) · pre_tuning(max_trials=schedule_max_attempts) · 후보마다 notify_runner_results(RunnerResult(None, None)): latency 를 넣지 않고 NPU 평가기가 승자를 고름

JSONDatabase 기록의 실제 구성 (run_secs=None, 실행 조건이 같은 커널만 적중)

```
# <schedule_db>/<fingerprint: source sha · TVM · contract · native>/
Workload.mod = PrimFunc − {global_symbol, op_pattern}
  .attr["npu.selected_schedule_contract"] = {
     profile − {search, seed, trials, attempts, words,
                joint_memory, beam, graph_search_seconds},
     scratch_base, slot_count, native_probe }
TuningRecord(trace, workload, run_secs=None, target=npu)
trace = split · reorder · cache_read(global.sram) ·
        compute_at · tensorize(npu_gemm_acc_sram) …
```

lookup
structural hash + equal (shape·dtype·scope·SRAM·버전)

→ **trace 재적용**
apply_to_schedule (새 tir.Schedule)

→ **공통 재검증**
lowering → SRAM 배치 → 용량 → probe codegen

→ **reuse / tune**
채택 (탐색 0 회) / incumbent → 개선 시만 commit

작은 예제 (FP16, 각 1 회 측정)	기존 build	첫 저장 build	DB 재사용 build	새 후보 TIR 생성
64×64 matmul 3 회 , 커널 사이 SRAM 보관	0.419 s	0.499 s	0.307 s	8 → 0
M=64 K=128 N=64, SiLU+multiply fusion	0.343 s	0.383 s	0.264 s	3 → 0
M=7 K=128 N=64, padding	0.369 s	0.422 s	0.164 s	7 → 0

시간에는 context/DB 준비가 포함되며 일반적인 배율이 아닙니다 . 재사용 시 " 탐색 " 이 0 회이지 trace 재생 ·lowering· 검증이 없어지는 것은 아닙니다 .

세 경로의 명령이 같고 C-model 출력은 bit 단위로 같았습니다 . Llama 4 개 커널에서는 space 와 ReplayFunc 의 최종 binary 가 같았고 (E8), 512-token down 은 개선 후 두 방식 모두 4 회 검사 · 합법 4 개였습니다 (E9). DB 는 최외곽 compilation context 에서만 설정하며 E10 은 DB 없이 실행합니다 .

ReplayFunc 와 DB 모두 NPU 공통 평가기 (실제 SRAM 배치 · 명령 생성) 를 거칩니다 . TVM 의 latency 순위로 고르지 않고 , DMA 를 run_secs 로 넣지도 않습니다 .

근거 : npu_compiler/npu_meta_schedule.py (replay_candidates, SelectedScheduleDatabase), npu_schedule.py _fit_matmul, evidence/current/meta_schedule_reuse.json, docs/current/06_matmul_schedule.md

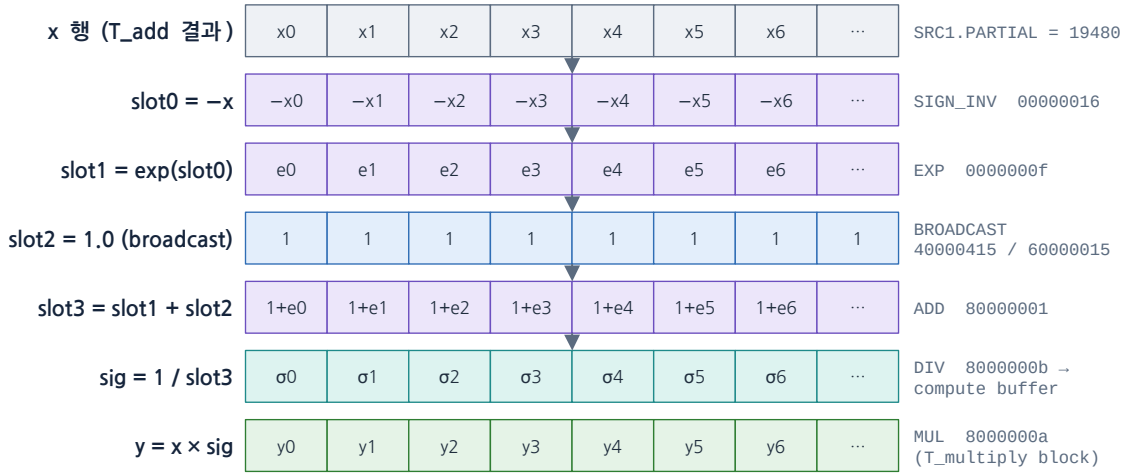
원소별 연산의 codegen: SiLU 하나가 ISA 하나가 아니다

VLEN=128 인 한 행이 SRAM scratch slot 을 거치며 SIGN_INV → EXP → BROADCAST(1.0) → ADD → DIV 로 sigmoid 가 되고, 마지막 곱은 별도 block 입니다. 각 단계마다 descriptor·LOAD·SAVE 가 붙습니다.

실제 packet (fused 커널, 한 행 VLEN=128, peephole 뒤)

```
00008082  VLEN 128
104c1880  SRC1.P = x 행     00000090 LOAD   00000016 SIGN_INV   90001880 DST.P=slot0   00000098 SAVE
10001880  SRC1 = slot0     00000090 LOAD   0000000f EXP        90021880 DST=slot1     00000098 SAVE
40000415  BROADCAST lo (1.0 @ pool 4)   60000015 BROADCAST hi                90041880 DST=slot2
SAVE
10021880 50041880  LOAD×2 (slot1, slot2)   80000001 ADD   90061880 DST=slot3   00000098 SAVE
10041880 50061880  LOAD×2 (slot2, slot3)   8000000b DIV   90cc1880 DST=sig     00000098 SAVE
... (별도 T_multiply block) LOAD x, LOAD sig   8000000a MUL   SAVE y
```

SRAM scratch slot 을 지나는 한 행 (128 원소, FP16 저장) — 오른쪽은 실제 opcode word



E1 fused 프로그램 opcode	count	역할
0x80 ADDR	606	descriptor 주소 (peephole 뒤에도 대부분)
0x90 LOAD / 0x98 SAVE	388 / 258	SRAM ↔ vector/matrix unit
0x01 ADD	96	bias add 64 행 + sigmoid 의 +1 32 행
0x16 SIGN_INV / 0x0f EXP	32 / 32	sigmoid 전개 (행 32 = 64×64 를 128 폭으로)
0x0b DIV / 0x0a MUL	32 / 32	$1/(1+e^{-x})$, x·sig
0x15 BROADCAST	64	상수 1.0 (2 words 씩)
0x42 M_MUL / 0xa0 GLOAD / 0xa8 GSTORE	2 / 4 / 1	matmul 2 K-tile, 입력·상수 DMA, 출력
합계	1,577 words	1,562 instructions

중간값은 FP16 scratch slot(6 개 × 최대 행 폭) 에 저장·재사용됩니다. 한 단계 = VLEN·ADDR·LOAD(·LOAD)·OP·ADDR·SAVE 이므로 원소별 연산 하나가 명령 여러 개가 되고, fusion 해도 vector 명령이 matmul activation bit 로 바뀌지 않습니다. 행이 32 개 (64×64 를 128 폭으로 결합) 라 각 opcode 가 32 회씩 나타납니다.

명령 수 (words) 는 짧은 vector 행과 descriptor 반복에 좌우됩니다. 이것이 정적 전개 stream 의 비용이며 cycle 과는 다른 지표입니다.

근거 : npu_compiler/tir_codegen_v09.py (_emit_pointwise, _materialize, _sigmoid), docs/examples/pipeline/09_packets.json words 440-468

물리 256 lanes 와 논리 VLEN, 그리고 generic vector scheduling

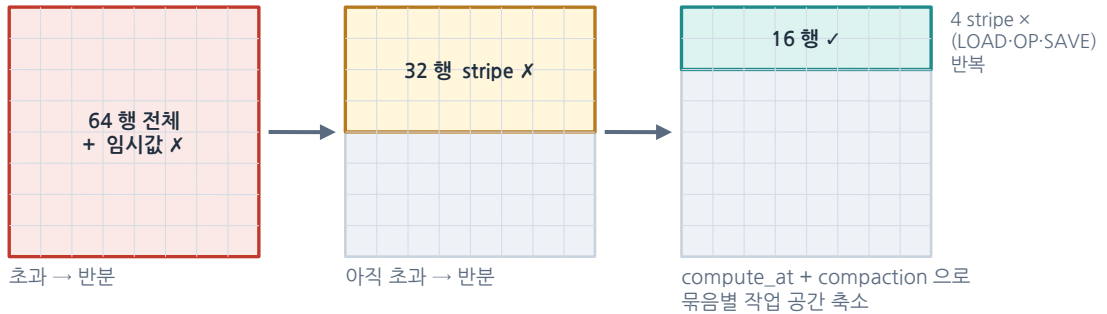
v09 규약은 물리 폭 256 과 논리 길이 ≤ 65535 를 구분합니다 . Generic vector 커널은 tensor 전체를 SRAM 에 놓아 보고 안 들어가면 행 묶음을 반으로 줄이며 , 연속 원소별 축은 결합해 긴 행으로 만듭니다 .

VLEN 1000 = 256 + 256 + 256 + 232 (장치 내부 반복 전제)

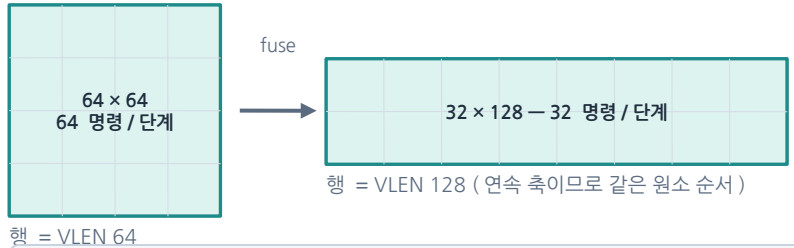
256 lanes	256 lanes	256 lanes	232 (tail)

- Compiler 가 배열을 1024 로 padding 해야 한다는 뜻은 아님 . C-model 은 VLEN 전체를 C++ 배열로 보관
- 실제 장치가 256 만 지원한다면 공통 vector lowering 에서 분할하고 긴 reduction 의 누산 · 반올림 계약도 재검토 (미확인 항목)
- 길이 1024 명령 하나 ≠ 길이 256 명령 하나의 비용 . 현재 (DMA, words, SRAM) 모델에는 lane 활용률이 없음 . REDUCE 길이 ≤ 65535, softmax 한도 65504

Generic vector 커널 : SRAM 에 안 들어가면 행 묶음 (stripe) 을 반으로



축 결합 : 64 행 × VLEN 64 → 32 행 × VLEN 128 (명령 수 절반)



축 결합 검사	결과
broadcast, 비연속 stride, 겹치는 입출력 , 임시 공간 한도	검사 통과 시에만 결합
행별 reduction (softmax sum, RMSNorm)	축을 합치지 않음 (의미가 달라짐)
SiLU 처럼 입력을 두 번 읽는 경우	같은 묶음의 cache 공유 (DRAM 재적재 없음)

```
# 작은 예제 scheduled TIR: npu.pointwise_vector_width = 128
for ax0_ax1_fused_0, ax0_ax1_fused_1 in T.grid(32, 128):
    compute_global_sram[(f0*128+f1)//64, (f0*128+f1)%64] =
        T.sigmoid(T_add_intermediate_global_sram[...])
```

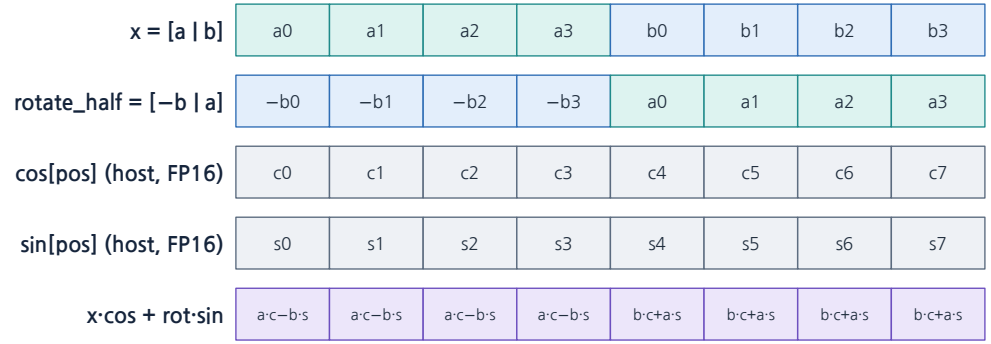
Vector schedule 은 " 공간에 들어가는 큰 행 묶음 " 을 찾는 성격이 강합니다 . 짧은 명령과 SRAM 임시값 비용은 남은 검토 항목입니다 .

근거 : docs/current/07_operators.md, reference/execution_contract.md, npu_schedule.py (generic stripe halving, _coalesce_pointwise)

RoPE 와 GQA: host 가 cos/sin 을 만들고 NPU 는 곱셈 · 부호 반전 · 재배치만 한다

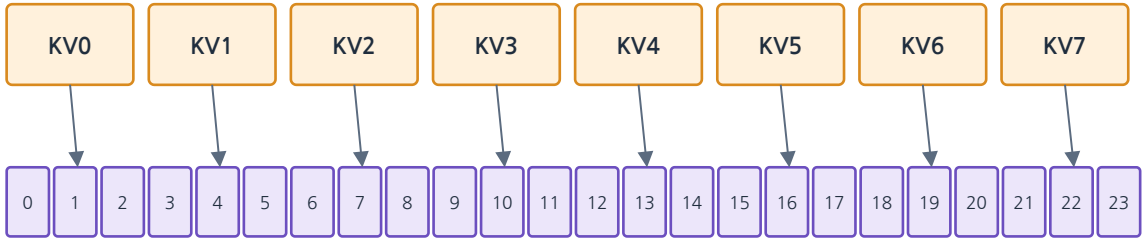
Head dimension 128 을 앞 / 뒤 절반 (a | b) 로 나누고 rotate_half = [−b | a] 를 만든 뒤 x·cos + rotate_half(x)·sin 을 계산합니다 . Q/K 에만 적용하고 V 에는 적용하지 않습니다 .

RoPE: 한 token · 한 head 의 128 원소 (8 칸으로 축약)



IR: split(64) → negative → concat → multiply(cos), multiply(sin) → add. new_a = a·cos − b·sin, new_b = b·cos + a·sin (짝 (a_i,b_i) 의 2D 회전). V 에는 적용하지 않음 .

GQA: 24 Q heads ← 8 KV heads



prefill: op.repeat(k, 3, axis=1) → [S,24,128]. persistent cache 는 KV head 8 개로 저장 (Q head 수만큼 복제하지 않음)

host 준비 (nn_models/llama.py)	내용
rope_freqs(cfg)	θ^(−2i/128), θ=500000; linear: ÷factor
llama3_scale_freqs	wavelength > 8192/1 → f/32; < 8192/4 → f; 사이는 보간
rope_inputs(cfg, positions)	float64 각도 → FP16 cos/sin [S,1,128] (heads 로 broadcast)
validate_rope_scaling	default / linear / llama3 만 허용 , 알 수 없는 키 ·factor<1 거절

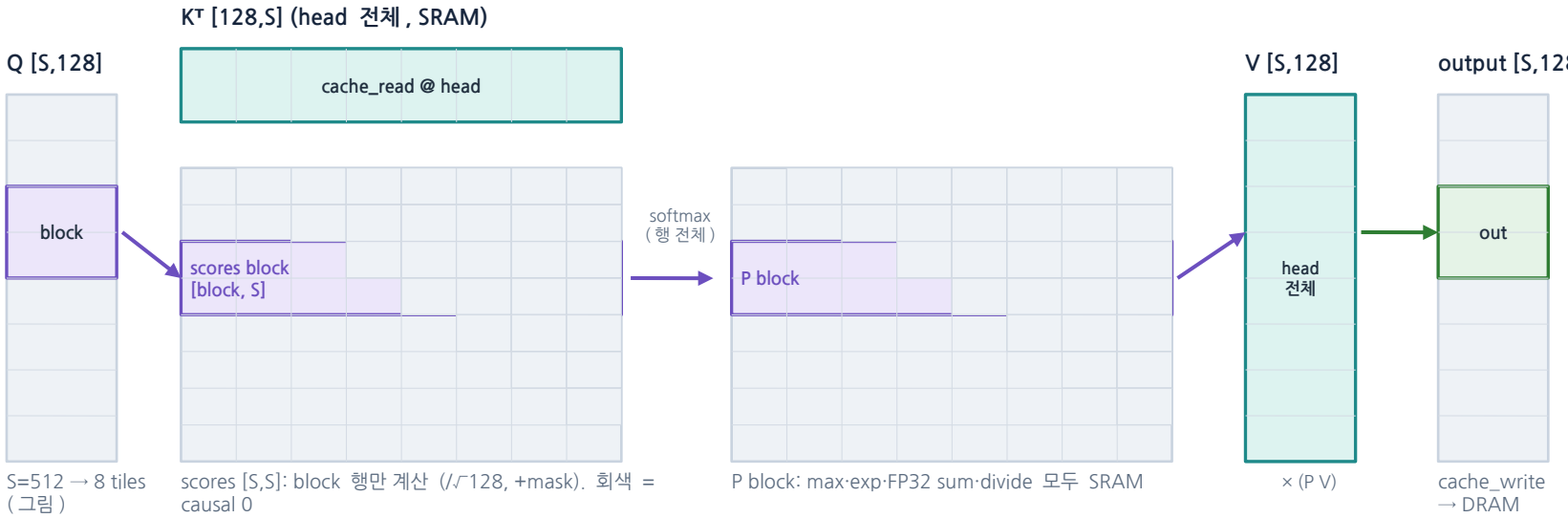
GQA 관련 후보 / 계약	내용
FoldRepeatedGemv (decode, m=1)	matmul(A[24,1,K], repeat(B[8,K,N],3)) → matmul(A.reshape(8,3,1,K), B.expand_dims(1)) broadcast. 후보이며 항상 선택되지는 않음
prefill 의 repeat (axis 1)	compound attention schedule 에 맡김
reshape / transpose / concat	reshape 는 view 가능 , transpose·concat·repeat 는 모두 view 는 아님 → lowering 이 결정

RoPE 설정은 검사 후 지원하거나 거절합니다 . 알 수 없는 설정을 조용히 근사하지 않습니다 .

근거 : npu_compiler/nn_models/llama.py (_rotate_half, rope_freqs, llama3_scale_freqs, validate_rope_scaling), npu_graph_opt.FoldRepeatedGemv

Attention 을 연결해서 읽기 , 그리고 full-fusion 의 query 분할

Prefill head 하나 : Q[S,128]×Kᵀ[128,S] → scores[S,S] → /√128 + mask → 행별 softmax → P×V[S,128]. 전체를 묶으면 query 행 block 만 계산하고 그 block 의 score 행 전체를 SRAM 에서 소비합니다 .



S=3072 크기	값
24-head score 전체	432 MiB
head 1 개 64 행 score	384 KiB
K, V (head 1 개)	768 KiB 씩
Q block 64 행	16 KiB

다음 row block (아래로 이동) → 같은 head 의 K/V 를 다시 사용 . mask [1,S,S] 은 head 축이 1 이라 row_o loop 에서 한 번만 GLOAD (head 들이 공유).

schedule_attention_blocked 단계	내용
split(row, [None, block]) → reorder(row_o, head, row_i, …)	row block 이 head loop 바깥 : block 마다 모든 head 처리
모든 중간 block compute_at(head), set_scope(SRAM)	scores·max·exp·sum· 정규화 ·PV accumulator 가 SRAM 에만 존재
cache_read Q/K/V → compute_at(head); mask → row_o	K/V 는 block 마다 다시 읽음 (block 이 클수록 재읽기 감소)
두 matmul: pad_einsum? → split 64 → decompose_reduction → tensorize ×2	일반 matmul 과 같은 64×64 MAC chain
_fit_attention: block = rows 부터 , 실패 시 max(floor, block//2)	floor = min(rows, 64). floor 에서도 실패하면 LinkError

없는 것 : key 까지 나누어 진행 중 max·sum·output 을 보정하는 online softmax(FlashAttention) 는 없습니다 . 한 block 이 query 행의 key score 전체를 갖고 있어 softmax 는 정확합니다 .
조건 : full 패턴은 정적 rank-3 FP16, m·d·t·dv 64 배수 . decode(m=1) 는 해당 없음 . mask 는 계산을 동적으로 생략하지 않습니다 .

384 KiB score 만 있으면 커널이 들어간다는 뜻이 아닙니다 . K/V, mask, 확률 , 출력 , 임시값이 함께 SRAM 에 있어야 하므로 block 크기를 줄여가며 맞춥니다 .

근거 : npu_compiler/npu_intrin.py schedule_attention_blocked, npu_schedule._fit_attention, docs/current/07_operators.md

세 가지 메모리 계획이 서로 다른 범위를 본다

① DRAM 은 Relax tensor 수명으로 pool 을 나누고 flat 주소를 주며 , ② 커널 안에서는 USMP 가 buffer offset 을 겹치고 , ③ 커널 사이에서는 보관값을 SRAM 위쪽에 예약합니다 . 커널을 합치지 않아도 SRAM 만으로 연결할 수 있습니다 .

DRAM image (flat, unit 2 B) — ① StaticPlanBlockMemory 의 pool + backend 의 flat 주소

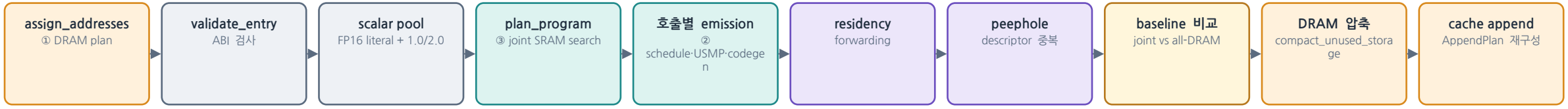
parameter leaves (변환된 순서)	graph 상수	storage pool A	storage pool B	출력 tree	literal pool · FP32 1.0
p.0.1 … weight, bias	const_data	G → U 재사용 (수명 안 겹침)	S → H	logits, K_new, V_new	linker 가 마지막에

SRAM 8 MiB (nibble) — ② USMP: 커널 buffer offset 공유 ③ 커널 사이 보관값 : top-down 예약

literal·scratch	② buffer b_0 (pinned)	② b_1 / b_3 (offset 공유)	② b_2 / b_4 / input panel	(cap)	③ 보관값 H (다음 커널이 직접 LOAD)
6 slots	gemm 목적지	충돌 없음	compaction 된 크기	workspace 천장	reserve(): 위에서 아래로 first-fit

계획	분석하는 범위	도구와 책임	결과
① 기본 DRAM storage	Relax 함수의 tensor 수명과 재사용 (호출 순서 기준)	TVM StaticPlanBlockMemory → storage pool; backend 가 pool 을 flat image 주소로 (npu_memplan.assign_addresses)	plan.address[name] (unit 2 B), pool_base/bytes, outputs
② 커널 내부 SRAM	Scheduled TIR 하나의 임시 buffer 수명 · 충돌	TVM USMP extract_buffer_info + greedy 배치 vs bump; matrix 목적지 pinned (npu_sram.plan_sram)	SramLayout: offset(nibble), peak, 알고리즘
③ 커널 사이 보관	생산 호출부터 마지막 소비 호출까지 (값의 버전)	NPU whole-tensor 보관 탐색 (beam 4) + 보관 공간을 뺀 용량으로 호출별 재 schedule (npu_memory_search.plan_program)	retained_values, 호출별 recipe/words, 보관 arena

compile_program 안에서의 실제 순서 (entry 하나)



같은 DRAM pool 이 나중에 다른 값으로 재사용되므로 " 주소가 같다 " 와 " 같은 데이터 " 는 다릅니다 . 보관 탐색은 값의 버전 (생산 호출 , 인자) 을 키로 삼습니다 .

근거 : docs/current/08_memory.md, npu_compiler/npu_link.py _compile_program (단계 순서)

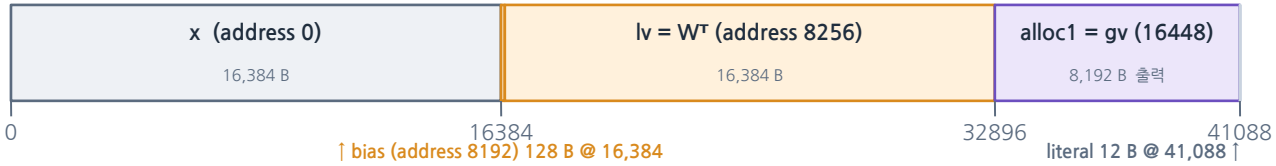
기본 DRAM 배치는 표준 TVM 결과에서 시작한다

ToNonDataflow → RemovePurityChecking → CallTIRRewrite → StaticPlanBlockMemory. CallTIRRewrite 가 allocation 과 TIR 호출을 명시하고 , StaticPlanBlockMemory 가 동시에 살아 있지 않은 tensor 의 pool 공유를 정합니다 .

```
# 06_initial_dram_plan.py ( 작은 예제 , plan_pipeline 뒤 )
@R.function
def prefill(x, bias, lv) -> R.Tensor((64, 64), "float16"):
    R.func_attr({"num_input": 1, "relax.force_pure": True})
    alloc1 = R.builtin.alloc_tensor(R.shape([64, 64]), R.dtype("float16"),
                                    R.prim_value(0), R.str("global"))
    cls.fused_matmul_add_silu(x, lv, bias, alloc1)   # 출력 buffer 를 가진 호출
    gv = alloc1
    return gv
# 중간값이 여러 개면 memory.alloc_storage(pool) + memory.alloc_tensor(offset)
```

assign_addresses 순서	규칙
(a) parameters	인자 순서 . tuple 은 leaf p.0.1 로 평탄화 , plan.tuples 기록
(b) graph constants	(dtype, shape, bytes) 내용 키로 dedupe → const_data
(c) bindings (프로그램 순서)	alloc_storage → pool_base; alloc_tensor → pool_base + offset//unit; builtin.alloc_tensor(결과) → 자체 slot; reshape/view/ensure_zero_offset → alias + plan.views
(d) literal pools (linker)	FP16 scalar pool (1.0, 2.0 포함 , 짝수 개) → constant_base; FP32 1.0 → one_fp32_base

assign_addresses 결과 (작은 예제 bundle, bytes = plan.address × unit 2 B)



주소는 4-byte cell 로 반올림 (bias 128 B 는 cell 정렬됨). image 크기 = plan.top × 2 = 41,100 B. 이 배치는 SRAM 선택 · 최종 압축 이전의 기본 storage 계획이며 , 8 장 뒤의 compact_unused_storage 가 미사용 pool 을 제거하면 주소가 당겨집니다 .

StaticPlan 객체의 주요 field

배치 : address, nbytes, pool_base, pool_bytes, tuples, const_data, graph_constants, top, views

ABI·linker: func, outputs(tree), constant_values, constant_base, one_fp32_base, codegen_contract, numeric_policy

codegen 결과 : kernel_records(index, name, backend, sram_peak, arguments[address/size/dtype], span), sram_layouts, schedules, call_schedules

메모리 결과 : joint_memory, residency, dram_compaction, cache_append/append_plan, compilation

수명은 생성부터 마지막 사용까지입니다 . G 의 공간에 U 를 쓸 수 있는 이유는 같은 배열로 변환해서가 아니라 G 의 옛 값이 더 이상 관찰되지 않기 때문입니다 .

근거 : npu_compiler/npu_memplan.py (plan_pipeline, assign_addresses), docs/examples/pipeline/06_initial_dram_plan.py, bundle/program.json memory

수명 분석 : 실제 폭 MLP 의 G/S/U/H 는 언제까지 살아 있는가

gate → SiLU → up → multiply → down (최신 실제 폭 MLP 측정의 순서). M=64 에서 G/S/U/H 는 각각 [64,8192] FP16 = 1 MiB 입니다 .



계산 중	동시에 필요한 주요 값	완료 후 보관
gate	X, G 출력 + W panel 작업 공간	G
SiLU	G, S + 임시값	S
up	X, S, U + W panel 작업 공간	S, U
multiply	S, U, H	H

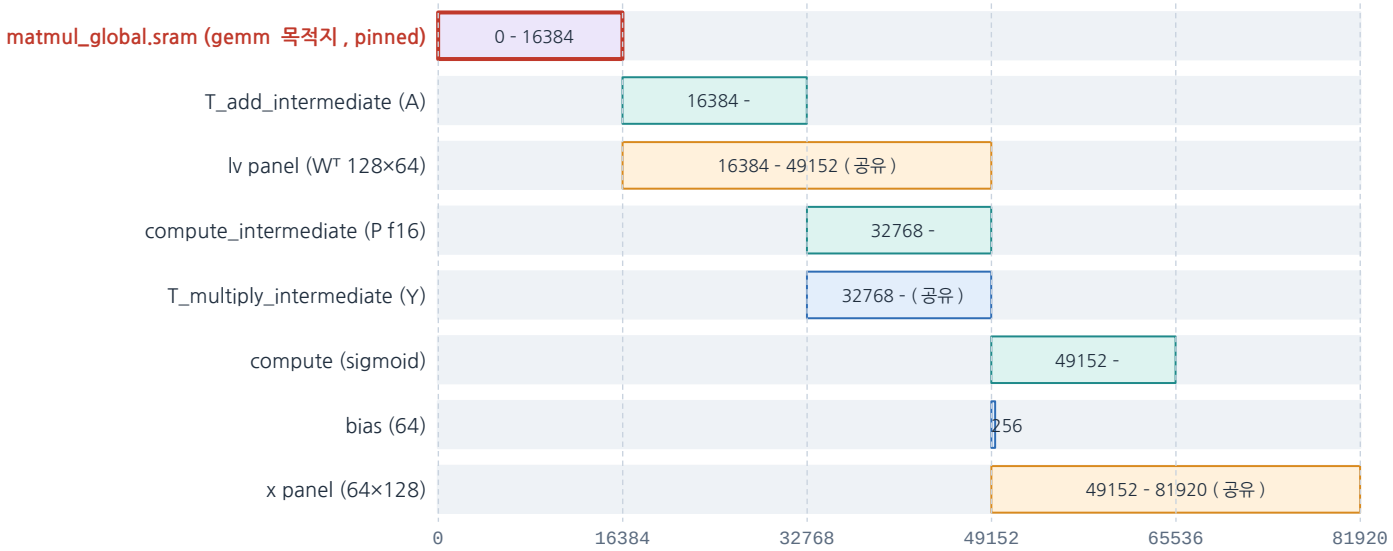
실제 peak ≠ 살아 있는 값의 합 : weight panel(gate/up 384 KiB, down 4 MiB), scratch, 정렬 , 빈 공간이 더해집니다 . M=64 MLP 기록의 SRAM peak 는 7,446,540 B (8 MiB 중).

보관 탐색은 이 수명을 값의 버전 단위로 분석하고 , 보관할 값의 공간을 예약한 뒤 남은 용량으로 각 커널을 다시 schedule 합니다 .

USMP: 커널 내부 buffer 의 충돌 분석과 greedy 배치

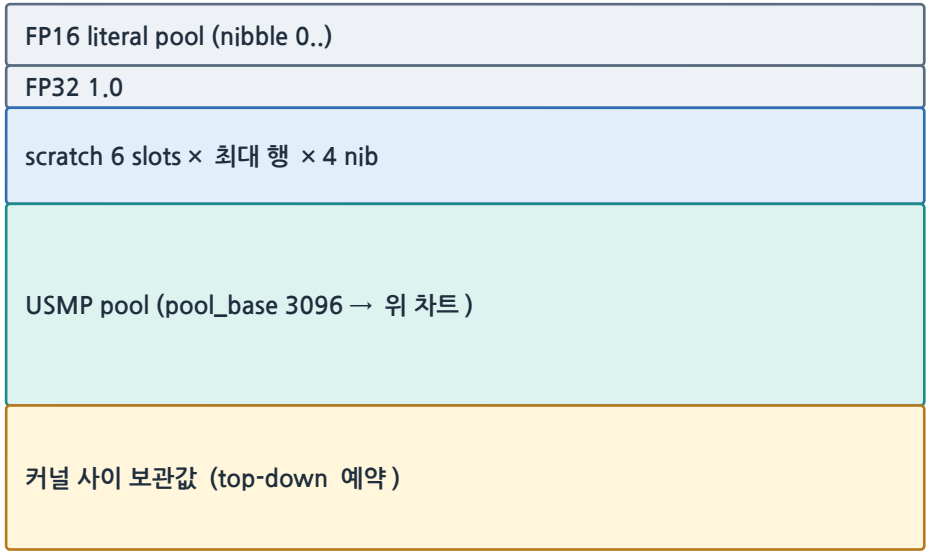
작은 예제 fused 커널의 실제 결과 . 8 개 buffer(총 163,840 nibbles) 가 충돌하지 않는 것끼리 offset 을 공유해 pool 81,920 nibbles 에 들어갑니다 (baseline bump 150,808 → peak 85,016 nibbles, 32,896 B 절약).

USMP greedy_by_conflicts 결과 — 가로축 = pool 안의 offset (nibble, pool base 3096 기준)



같은 세로 구간을 공유하는 buffer 는 conflicts 목록에 서로 없는 쌍 (수명이 겹치지 않음). pinned buffer 는 모든 buffer 와 충돌로 취급되어 혼자 offset 0 을 씁니다 (SAVE 가 지연되는 동안 덮어쓰기 방지).

SRAM 지도 (주소 0 부터 위로)



작은 예제 값	
pool_base	3096 nibbles = 24 + 6 × 128 × 4
peak / baseline	85,016 / 150,808 nibbles
절약	32,896 B (usmp_pressure 40,960 B)
알고리즘	greedy_by_conflicts ("validated lifetime reuse")

plan_sram 순서

크기 (nibble, 8 정렬) → 분석 복사본 (LowerMatchBuffer·LowerOpaqueBlock) → pinned 표시 → WorkspacePoolInfo + extract_buffer_info(conflicts) → greedy_by_size / greedy_by_conflicts vs bump → validate(정렬 · 겹침 ·peak) → place(cursor, capacity)

충돌하지 않는 buffer 가 offset 을 공유합니다 . 지연된 accumulator 상태나 opaque 호출 때문에 사용이 끝났다고 단정할 수 없는 buffer 는 pin 해 조기 재사용을 막습니다 .

근거 : npu_compiler/npu_sram.py plan_sram, docs/examples/pipeline/bundle/program.json sram_layouts

커널 사이 SRAM 보관 탐색 : 값의 버전을 찾고 , 보관 공간을 예약한 뒤 , 호출별로 다시 schedule 한다

npu_memory_search.plan_program. 보관값은 SRAM 위에서 아래로 예약되고 , 커널 workspace 는 그 아래 천장 (cap) 까지만 쓰도록 재 schedule 됩니다 . beam width 4, all-DRAM 진행도 항상 비교 . tensor 전체 보관만 있습니다 .

```
calls, values, excluded = discover(module, plan)     # 값 = ( 생산 호출 , 인자 ) 버전
states = [(live={}, prefix=[], dma=0, words=0, peak=0)]
for i, call in enumerate(calls):
  outs = [v for v in values if v.producer == i]; ext = []
  for (live, prefix, dma, words, peak) in states:
    R = [live]                                  # 보관 / 비보관 조합 (≤ 2·beam)
    for v in outs: R += [r + {v: reserve(r, v.size)} for r in R if fits]
    for r in R:
      bind = SRAM 주소 ( 읽는 입력 : live[v], 쓰는 출력 : r[v])
      cap  = min(reservation addrs in r) or SRAM   # workspace 천장
      prim = parameter_scopes(call.prim, bind)      # bound param → global.sram
      recipe = _fit_kernel(prim, profile(sram_bytes=cap//2))   # 재 schedule
      stats  = probe(...)                          # emit ( 또는 cost cache)
      live2 = {v in r : v.last > i}                # 마지막 소비 지나면 해제
      ext.append((live2, prefix+[choice], dma+stats.dma, words+stats.words, max(peak, top)))
  states = best per live-layout, sorted by (dma, words, peak)[:beam] (+ all-DRAM 상태 )
winner = min(states, key=(dma, words, peak))
# 재 emit: cost cache 재사용분을 실제 주소로 다시 생성 · 검증
# 보관 arena 를 최대 workspace peak 바로 위로 내림 → 재 emit
# 프로그램 수준 : all-DRAM baseline 과 cost 비교 , 엄격히 작을 때만 채택
```

discover 의 보관 후보 조건	
dtype	FP16
같은 호출에서 읽지 않음	RMW 제외
compact	stride 없음 , elem_offset 0
보호 범위와 겹치지 않음	출력 tree, parameter leaf, graph 상수
이후 호출 검사	부분 겹침 /dtype 재해석 ✗ , in-place ✗, 부분 덮어쓰기 ✗ , 전체 덮어쓰기 = 버전 종료

SRAM 예약 방향 (두 matmul 예제 , 44 KiB)



↑ 위 : 보관값이 top-down first-fit 로 예약 (8-nibble 정렬). 이 예제는 arena 를 workspace peak 바로 위 (26,124) 로 내림

cap = min(예약 주소) → profile.sram_bytes = cap/2 로 커널 재 schedule

↓ 아래 : 커널 workspace (scratch + USMP pool). 보관 parameter 가 workspace 와 겹치면 LinkError

beam 4 + all-DRAM 상태 유지 · residency=True, snapshot 없음 , w8a8 tir 일 때만

보관하지 못하면 DRAM 경로를 쓰므로 큰 activation 이 곧 컴파일 불가는 아닙니다 . 보관은 이득이 실제 비용으로 확인될 때만 채택됩니다 .

근거 : npu_compiler/npu_memory_search.py (discover, reserve, parameter_scopes, plan_program)

작은 예제 ②: 두 matmul 이 SRAM 44 KiB 에서 H 를 공유하는 실제 주소

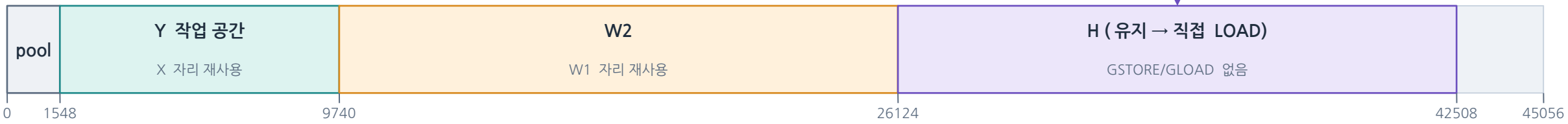
X[64,64] @ W1[64,128] → H[64,128]; H @ W2[128,64] → Y[64,64]. 단위 bytes. H 는 DRAM 에 저장되지 않고 두 번째 matmul 이 이 위치를 직접 읽습니다.

커널 1: H 생성 (matmul)



H 같은 allocation 유지

커널 2: H 사용 (matmul1)



값	SRAM 시작	크기	이후 재사용
X	1,548	8,192	두 번째 커널의 Y 작업 공간
W1	9,740	16,384	두 번째 커널의 W2
H	26,124	16,384	두 번째 커널까지 유지
peak	42,508		용량 45,056 (44 KiB)

joint_memory 기록	값
policy	bounded call-site schedule and retained-SRAM search
beam / candidates / trials	4 / 1 / 4
H 예약	nibble 57344 → arena 2,548 B 하향 → 52248 (byte 26,124)
USMP	두 커널 모두 bump (greedy 가 더 줄이지 못함)
제거된 DRAM pool	storage 16,384 B @ byte 40,960 (H 의 DRAM 예약)

H 의 GSTORE/GLOAD(32,768 B) 가 사라져 DMA 81,932 → 49,164 B. 32 KiB 에서는 첫 matmul 이 더 이상 들어가지 않아 보관이 적용되지 않습니다 (E2).

근거 : docs/examples/two_matmuls_recorded/program.json, scheduled_0000.py, scheduled_0001.py, docs/current/08_memory.md

임시 출력 tile 을 없애고 H 에 바로 쓴다 : view 와 row stride

H 는 행마다 128 FP16 = row stride 256 B. 두 번째 열 tile 의 시작은 H + 128 B 이지 H + 8192 B 가 아닙니다 . 두 tile 은 각 행의 왼쪽 / 오른쪽 절반이므로 stride 를 유지해야 합니다 .

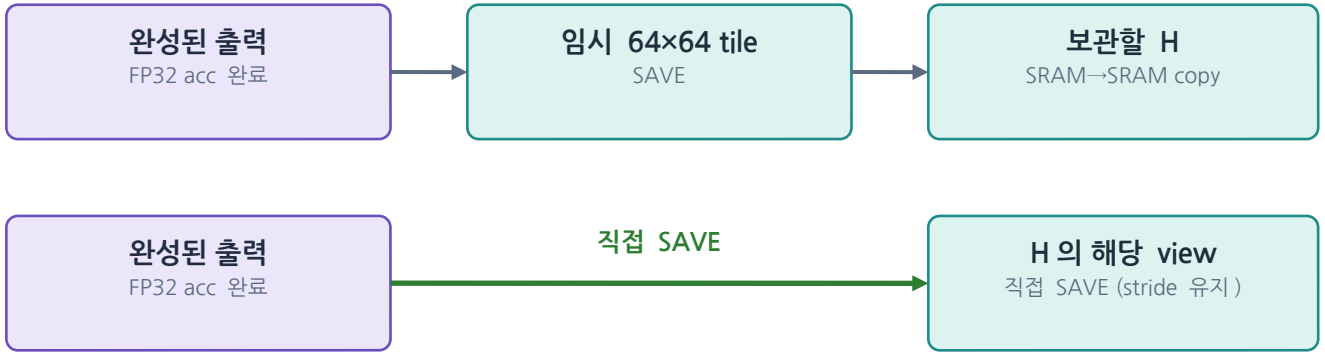
H [64,128] FP16 in SRAM (row-major, stride 256 B)

tile 0: 열 0..63	tile 1: 열 64..127
H + 256	H + 256 + 128
H + 63·256	H + 63·256 + 128

tile 1 의 시작 = H + 128 B (첫 행의 오른쪽 절반). MAIN descriptor 는 전체 stride(128 elements), PARTIAL descriptor 는 이번 tile 의 시작과 64×64 크기 .

E3 (44 KiB 두 matmul)	전	후
ISA words	85	77
DMA bytes	49,164	49,164
SRAM peak bytes	42,508	42,508
DRAM image bytes	65,548	49,164
출력	bit-exact	

이전 (임시 tile 경유) → 현재 (직접 SAVE)




```
# scheduled_0000.py: 출력 compute가 SRAM scope, gemm view가 compute를 직접 겨냥
compute = T.match_buffer(var_compute, (64, 128), "float16", scope="global.sram")
C = T.match_buffer(compute[0:64, i1_1_0*64 : i1_1_0*64+64], (64,64), "float16",
                   strides=("C_s0", 1), scope="global.sram", offset_factor=1)
T.func_attr({"npu.accumulator_store_lowered": 1, "npu.forwarded_output_tiles": 1})
```

조건 : 64×64 FP16 tile 을 정확히 하나의 npu_fill_zero 와 하나의 npu_gemm_acc view 가 쓰고 identity copy 하나만 있을 때 . 절약한 8 KiB 를 더 큰 W panel 에 쓸 수 있어 peak 가 그대로인 경우도 있습니다 .

FP32 누산이 끝난 최종 저장 위치만 바꾸는 것이며 , K 중간 partial sum 을 H 에 내리는 것이 아닙니다 (누산 계약 유지).

근거 : npu_compiler/npu_accumulator.forward_output_tiles, docs/examples/two_matmuls_recorded/scheduled_0000.py, evidence/current/refinement.json (E3)

보관 후 남은 DRAM 예약 제거와 주소 재배치 , 그리고 후단 residency forwarding

SRAM-only 전달이 확정되면 아무 곳에서도 쓰지 않는 pool 전체를 제거하고 나머지 주소를 당깁니다 . 두 matmul 예제에서 H 의 DRAM pool 16,384 B 가 사라지고 출력 gv 가 57,344 → 40,960 으로 이동합니다 .

compact_unused_storage

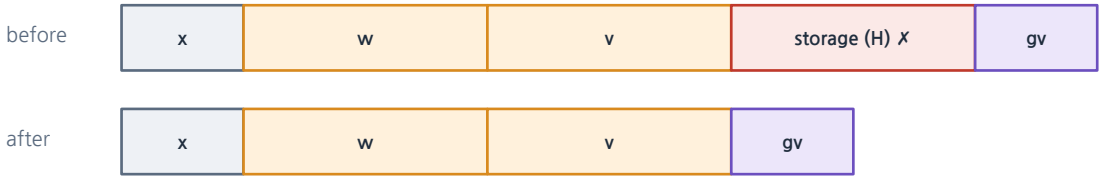
단계	내용
반드시 남김	parameter leaf, 출력 , const_data, literal pool, FP32 1.0, 모든 DMA 의 strided span(빈틈 포함)
제거 대상	cell 반올림 범위가 위와 겹치지 않는 pool 전체 (내부 부분 제거 없음)
relocate	제거된 pool 크기만큼 아래 주소를 당김
갱신 metadata	DMA g_addr word, plan.address, pool_base/bytes, const_data, constant_base, one_fp32_base, top, outputs offset, views, kernel_records storage(sram/dram/unused), joint_memory, residency
후속	saved_bytes > 0 이고 KV 계약이 있으면 AppendPlan 재구성 (주소 기준 변경)

예	before → after	제거
두 matmul (44 KiB)	65,548 → 49,164 B	storage 16,384 B (H 의 DRAM 예약)
Llama prefill_cache	6,429,599,864 → 6,429,587,576 B	storage61/62 각 6,144 B (작은 중간값 pool)
실제 폭 MLP M=64	154,927,116 → 151,781,388 B	3 MiB (중간값이 같은 pool 을 재사용해 4 MiB 아님)

npu_memory_search vs npu_residency (둘 다 켜져 있음)

	memory search (보관 탐색)	residency forwarding
실행 시점	codegen 전	codegen 뒤 , peephole 전
SRAM 공간	명시적으로 예약	예약 없음 , 기회주의적
커널 재 schedule	함	안 함
생산자 GSTORE	제거	유지 (write-through)
소비자 GLOAD	애초에 없음 (SRAM 입력)	커널 진입 시 완전 포함되면 제거 또는 SRAM→SRAM V_COPY
효과 분석	값 버전 /byte 범위	WriteEffects: 각 명령의 쓰기 범위 ; 미지 opcode → cache 전체 무효

두 matmul 예제의 DRAM image (bytes)



gv 57,344 → 40,960; GSTORE packet 의 g_addr word 와 plan.outputs.offset_bytes 를 함께 갱신 . residency.events = [] (보관 탐색이 이미 H 전송을 없앰)

DMA 49,164 B 와 최종 image 49,164 B 가 같아진 것은 이 예제의 우연이며 두 지표는 원래 다른 양입니다 .

근거 : npu_compiler/npu_memplan.compact_unused_storage, npu_residency.forward, docs/examples/two_matmuls_recorded/program.json (memory, dram_compaction)

Residual 과 KV cache 는 왜 다르게 취급되는가 , 그리고 현재 메모리 계획의 경계

Residual 도 중간값이면 마지막 add 까지 보관 후보입니다 . KV cache 는 다음 프로그램 호출에도 필요하고 ABI 입력 / 출력으로 관찰되므로 DRAM 상태를 유지합니다 .

값	보관 후보 ?	이유
Residual X (attention 전 hidden)	후보	중간값 . 마지막 add(R = X + attn) 까지 SRAM 보관 가능 . 384 KiB(M=64) 이지만 사이 작업 공간과 비용에 따라 결정
MLP 중간 G/S/U/H	후보	FP16, 같은 호출에서 RMW 아님 , compact → 실제 폭 MLP 에서 보관 적용 기록
KV cache (k_cache, v_cache)	보호	ABI 입력 parameter 이자 append 대상 . 다음 호출에서 관찰 → DRAM 유지
최종 출력 (logits, K_new, V_new)	보호	출력 tree 의 leaf: runtime 이 그 주소에서 읽음
graph 상수 , literal pool	보호	const_data / constant_base
alias 가 모호한 view, in-place 갱신	제외	값 버전을 확신할 수 없음 → DRAM 계약 유지

현재 메모리 계획의 경계

Whole-tensor 보관만 : 일부 tile 만 남기기 , 중간 퇴거 , 호출 순서 변경은 없음
고정 호출 순서 : Relax binding 순서대로 . 재배치로 수명을 줄이는 최적화 없음
조각화 : 빈 구간 재사용과 arena 하향은 있지만 전역 allocator 는 아님
DRAM 압축 : 완전히 미사용인 pool 만 제거 . 사용 중 pool 내부 낭비는 미해결
Snapshot: 관찰해야 하는 중간값이 있으면 SRAM-only 전달을 제한하거나 끔
실패 시 : 보관하지 못하면 DRAM 경로 → 큰 activation 이 곧 컴파일 불가는 아님 . 개별 연산의 shape/reduction 한도는 별개

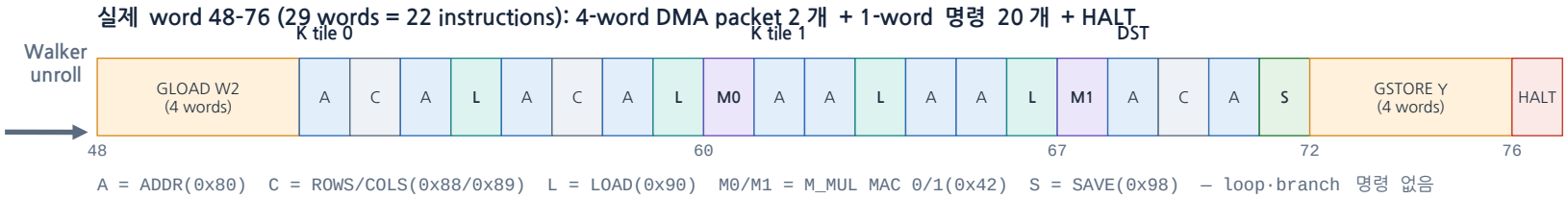
"kv" 라는 이름이 아니라 외부 가시성 계약으로 보호합니다 . 남은 낭비가 실제 프로그램 비용에 큰지 확인한 뒤에 다음 개선을 검토합니다 .

근거 : docs/current/08_memory.md, npu_memory_search.discover (보호 범위)

Codegen: 주소가 연결된 scheduled TIR 를 정적으로 전개된 명령 stream 으로

Walker 는 TIR 을 직접 해석합니다 . 모든 For 는 완전히 unroll 되고 loop/branch 명령은 없습니다 . 아래는 두 matmul 예제의 두 번째 커널 (K loop 2 회) 이 29 words 의 직선 stream 이 되는 실제 결과입니다 .

```
# scheduled_0001.py ( 핵심만 )
v_sram = cache_read(W2) → GLOAD
npu_fill_zero(C)        # MAC=0 상태
for k_0 in range(2):    # unroll
  npu_gemm_acc(C, lv[:,k_0*64:], v_sram[k_0*64:,:])
compute[i,j] = C[i,j]   # SAVE → GSTORE
```



TIR 조각	인식	생성하는 명령
call_extern npu_fill_zero / npu_gemm_acc (tensorize 결과)	_bind_match 로 64×64 view 해석	fill: 명령 없음 (zeroed 기록). gemm: region(SRC1) LOAD, region(SRC2) LOAD, M_MUL(mac). flush: region(DST) SAVE
원소별 BufferStore (innermost 축 = 행)	_emit_pointwise → _materialize (6 scratch slot)	VLEN, ADDR SRC1.P, LOAD, ADDR SRC2.P, LOAD, OP(VECTOR), ADDR DST.P, SAVE. 상수 /stride-0 → BROADCAST
reduction (LowerInitBlock 형태 : init + 누산 store)	_emit_reduction (Add/npu_sum_fp32_update → RSUM, Max → RMAX)	출력 위치마다 VLEN, SRC1.P(행 또는 x·x 같은 materialized 피연산자), LOAD, REDUCE, DST.P, SAVE. 길이 ≤ 65535
복사 block (scope 다름 : global ↔ global.sram)	_emit_dma: [global byte, sram nibble, nbytes] 구간 병합	2D DMA(정규 행 ·cell 정렬) / 1D dma_in·out / bounce(scratch gather + cell RMW)
복사 block (같은 scope)	_emit_movement	연속 : LOAD → COPY(0x17) → SAVE; stride-0: BROADCAST; strided: strided LOAD(matrix, ncols=1)
select / if_then_else 구간 (concat, padding)	_pieces / _split_row	행을 구간 경계에서 나눠 각각 emission

주소 단위 : _flat 은 SRAM 주소를 nibble(fp16 4, fp32 8, int8 2, int4 1) 로 , global 주소를 byte 로 돌려줍니다 . Python walker(tir_codegen_v09.py) 와 native C++ walker(v09_walker.cc) 는 같은 contract JSON hash 를 공유하며 커널 단위로 섞일 수 있습니다 .

TIR 에 loop 가 있어도 최종 ISA 에는 loop 명령이 없습니다 . words 는 " 명령 저장 공간 " 이며 실행 cycle 이 아닙니다 (Llama prefill 8.58M words).

근거 : npu_compiler/tir_codegen_v09.py (Walker.visit, _match_nest, _emit_pointwise, _emit_reduction, _emit_movement, emit_gemm), docs/examples/two_matmuls_recorded/program.json

네 가지 이동 명령 , 세 가지 주소 단위 , descriptor 상태 모델

Descriptor 는 다음 LOAD/ 연산 /SAVE 가 쓸 주소 ·shape·dtype " 상태 " 입니다 . 명령마다 배열 정보를 반복하는 대신 register 를 바꾸고 뒤 명령이 참조합니다 . 아래는 77-word 예제 두 번째 matmul 직전의 실제 register 값입니다 .

명령	출발 → 도착	DRAM 을 지나는가
GLOAD 0xA0	DRAM → SRAM	예 (4-word packet)
LOAD 0x90	SRAM → 연산장치 입력	아니오
SAVE 0x98	연산장치 결과 → SRAM	아니오
GSTORE 0xA8	SRAM → DRAM	예 (4-word packet)

위치	단위	byte 변환
plan.address (DRAM)	unit_bytes = 2	× 2
DMA global 주소 / stride / cols	32-bit cell	× 4
SRAM descriptor 주소	nibble	÷ 2 (fp16 = 4 nibbles)
shape / row stride / VLEN	element	× dtype 크기

Register 상태 예 (word 52-71, H[64,128] @ W2[128,64] → Y)

register	SRC1 (A = H)	SRC2 (B = W2)	DST (Y)
MAIN addr	0xCC18 → nib 52,248 (byte 26,124)	0x4C18 → 19,480 (9,740)	0x0C18 → 3,096 (1,548)
MAIN cols = stride	128 el (256 B)	64 el	64 el
PARTIAL addr (K tile 1)	+0x100 = +64 열	+0x4000 = +64 행	—
rows × cols / dtype	64 × 64 FP16	64 × 64 FP16	64 × 64 FP16

Descriptor 상태 모델 (설정 → 참조)

opcode	field	쓰는 상태
0x80 ADDR	operand[31:30], high[29], PARTIAL[28], value[23:8]	{SRC1,SRC2,DST} × {MAIN,PARTIAL} 주소의 16-bit 절반 (nibble)
0x88 / 0x89 ROWS / COLS	operand[31:30], PARTIAL[29], dtype[26:25], count[23:8]	rows/cols (element) + 그 operand 의 dtype. MAIN cols = 전체 row stride
0x82 VLEN	n[23:8]	vector 논리 길이 1..65535
0x90 LOAD	matrix[31], operand[30], strided[29], ncols[23:16], start[15:8]	SRAM→unit. matrix 는 PARTIAL rows/cols + MAIN stride, vector 는 VLEN
0x98 SAVE	matrix[31], strided[29], ncols, start	결과 → DST
0x42 M_MUL	mode[31:30], activation[29:28], MAC[27], imm[23:8]	SRC1×SRC2 → accumulator (MAC=0 새로 , 1 누적)
0x8A / 0x8B	high[29], value[23:8]	activation / weight scale 의 SRAM 주소 (양자화)

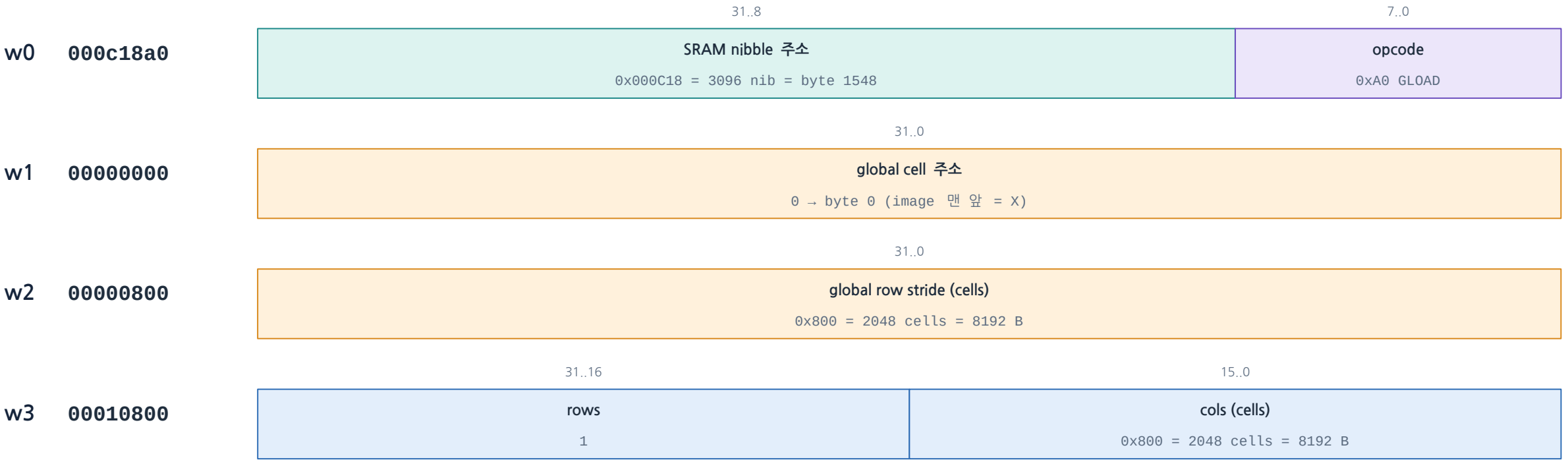
region(operand, view) = 8 words: ADDR MAIN lo/hi, ROWS MAIN, COLS MAIN(=stride), ADDR PARTIAL lo/hi, ROWS/COLS PARTIAL. 주소가 이전과 같아 보여도 중간 명령이 설정을 바꾸었는지 확인해야 합니다 .
Enum: operand SRC1=0, SRC2=1, DST=2 · mode IMM=0, SCALAR=1, VECTOR=2 · dtype FP16=0, FP32=1, INT8=2, INT4=3 · activation OFF=0, tanh-GELU=1, SiLU=2, legacy GELU=3. 모든 field 는 range-check 됩니다 ([7,128256] tensor 가 16-bit field 를 조용히 넘친 사례 이후).

기본 FP16 행렬의 stride 128 은 128 bytes 가 아니라 256 bytes 입니다 . MAIN stride 는 유지하고 PARTIAL 주소 · 크기만 바꿔야 올바른 위치에 SAVE 합니다 .

근거 : docs/current/09_isa.md, reference/execution_contract.md, npu_compiler/isa_compute.py, isa_v09.py, two_matmuls_recorded/program.json

실제 명령 해독 ① : X 를 SRAM 으로 가져오는 4-word GLOAD packet

77-word 예제의 word offset 8: 000c18a0 00000000 00000800 00010800. DRAM 의 X[64,64] FP16 8,192 B 가 SRAM byte 1548 에 도착합니다 .



해독 결과	X[64,64] FP16 = 64 × 64 × 2 = 8,192 B = 2,048 cells 를 DRAM byte 0 에서 SRAM byte 1548 로 한 행 (1D) 으로 이동 . rows 1 이므로 stride 는 의미 없음
같은 프로그램의 GSTORE (offset 72)	000c18a8 00002800 …: opcode 0xA8, SRAM nibble 3096 → global cell 0x2800 = byte 40,960 (Y 출력 , plan.outputs.offset_bytes = 40960)

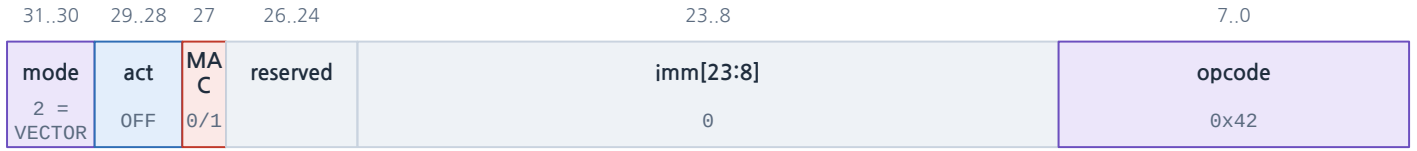
77 words = 62 instructions 인 이유 : DMA 는 명령 하나가 4 words 이고 나머지는 1 word 입니다 (GLOAD 4 개 , GSTORE 1 개 → 15 words 추가).

근거 : docs/examples/two_matmuls_recorded/program_memory.bin, program.json, npu_compiler/isa_v09.py

실제 명령 해독 ② : M_MUL 의 MAC bit 와 77-word 프로그램의 흐름

80000042 와 88000042 의 차이는 bit 27(MAC) 뿐입니다 . 둘 다 mode VECTOR, activation OFF. 아래 strip 은 77 words 전체를 opcode 종류별로 칠한 것입니다 .

M_MUL word (opcode 0x42)



word	값	뜻
30, 44	80000042	첫 matmul 의 두 H 출력 tile, 각각 MAC=0 (새 accumulator)
60	80000042	두 번째 matmul, 첫 K tile, MAC=0
67	88000042	같은 출력의 다음 K tile, MAC=1 (bit 27)
404c1880 / 504c1880	ADDR	SRC2 MAIN lo = 0x4C18 (W1 nibble 19480 = byte 9740) / PARTIAL
40008089	COLS	SRC2 MAIN cols = 128 (= row stride, elements)
c0000090 / 80000098	LOAD / SAVE	matrix LOAD SRC2 / matrix SAVE → DST

77 words 전체 (색 = opcode 종류 ; 주황 4 칸 = DMA packet 1 개)



주황 DMA (0xA0/0xA8) · 파랑 ADDR (0x80) · 회색 ROWS/COLS (0x88/0x89) · 청록 LOAD (0x90) · 초록 SAVE (0x98) · 보라 M_MUL (0x42) · 빨강 HALT. 62 instructions = 77 words = 308 bytes. DMA load 40,972 B + store 8,192 B (C-model counters 와 일치).

77-word 프로그램 timeline

0–3	상수 pool GLOAD (3 cells → SRAM nibble 0)
4–7	W1 GLOAD → SRAM 9740
8–11	X GLOAD → SRAM 1548
12–29	descriptor 설정 (SRC1/SRC2 region)
30	M_MUL MAC=0 → H tile 0
31–43	SAVE tile 0, SRC2 재설정 (+64 열)
44	M_MUL MAC=0 → H tile 1
45–47	SAVE tile 1 (H 직접)
48–51	W2 GLOAD → SRAM 9740 (W1 재사용)
52–59	descriptor (0x88 없음 : rows 64 유지)
60	M_MUL MAC=0 (K tile 0)
61–66	K tile 1 operand 설정
67	M_MUL MAC=1 (K tile 1)
68–71	Y SAVE (FP16)
72–75	Y GSTORE → DRAM byte 40960
76	HALT 0xFF

Peephole 의 효과 : 두 번째 tile 에서 SRC1 descriptor 생략 , 커널 1 에서 0x88 생략 . 값이 알려져 있고 바뀌지 않았을 때만 삭제합니다 .

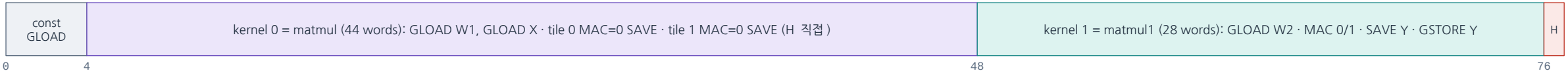
근거 : docs/examples/two_matmuls_recorded/program.json, program_memory.bin, docs/current/09_isa.md, npu_compiler/isa_compute.py

Linker: 커널 stream 을 호출 순서로 연결하고 , forwarding·peephole· 주소 압축으로 마무리한다

npu_link._compile_program. 커널은 독립적으로 emit 되고 (각자 필요한 descriptor 를 다시 냄) 경계의 중복 설정은 뒤의 peephole 이 제거합니다 . 아래 strip 은 77-word 예제의 실제 구성입니다 .



연결된 stream (77 words): kernel_records.span = [4, 48), [48, 76)



각 커널은 독립적으로 emit(필요한 descriptor 를 다시 냄) → 경계의 중복 설정은 peephole 이 제거 (커널 1 에 0x88 없음). residency 는 GSTORE 된 SRAM 사본을 커널 진입에서 재사용 .

Peephole (peephole.py): 정확한 register 상태 추적

- 추적 상태 : (addr, operand, partial, high), (vlen), (0x88/0x89, operand, partial), (dtype, operand), (scale, op, high)
- 값이 알려져 있고 같을 때만 write 삭제 . shape word 는 count 또는 dtype 이 바뀌면 유지
- DMA packet 은 4-word 단위로 건너뜀 . LOAD·SAVE· 연산 ·DMA·BROADCAST(0x15) 는 건드리지 않음 . branch 가 없어 정확함

Residency forwarding (npu_residency.forward): 효과 분석 기반

- GSTORE 마다 (global byte 구간 ↔ SRAM nibble 구간 , version, producer, canonical FP16) 사본 등록 . GSTORE 자체는 유지 (write-through)
- 겹치는 GSTORE 는 global 쪽 , GLOAD 목적지와 정밀 compute write 는 SRAM 쪽에서 무효화 . WriteEffects.step: vector SAVE = 길이 × dtype nibble, matrix SAVE = ((rows−1)·stride + cols)·4; 미지 opcode → cache 전체 무효
- 커널 진입 시 완전 포함된 연속 GLOAD 만 : 같은 주소면 삭제 , 아니면 SRAM→SRAM V_COPY + descriptor 복원

커널 emission 경로	언제	내용
joint search 가 미리 emit 한 words	plan_program 이 채택된 호출 (이 예제의 두 커널)	보관 주소로 이미 검증된 stream 재사용 (재 link 없음)
W8A8 reference emitter	w8a8 PrimFunc	전용 경로 (부록 A2)
scheduled recipe	나머지 호출	GlobalVar 별 recipe cache (28 layers: 약 32 개 커널이 1,206 회 호출) → _place_sram → native/Python walker

효과 분석은 " 이 명령이 어떤 주소를 읽고 쓰는가 " 를 아는 것입니다 . 인코딩이 같다는 이유로 명령을 삭제하지 않습니다 .

근거 : npu_compiler/npu_link.py, peephole.py, npu_residency.py, v09_effects.py, two_matmuls_recorded/program.json kernel_records

Python codegen 과 native C++ codegen 의 일치 , 그리고 통계의 정의

하나의 contract JSON 이 양쪽 op table 을 만들고 hash 로 묶입니다 . native 의 unsupported 는 Python fallback, invalid_schedule 은 fallback 없는 LinkError, 내부 오류는 propagate 합니다 .



status	처리
ok	native words 사용
unsupported	Python fallback (native_fallbacks 기록)
invalid_schedule	LinkError, fallback 없음
그 외 오류	propagate

모드와 검증

use (기본): native 사용 . NPU_NATIVE=0 으로 끔 , NPU_CODEGEN_SO 로 경로 지정
compare: 둘 다 실행해 첫 번째 다른 word 를 보고 (_compare_native)
build_identity: npu_codegen/*.cc,*.h 정렬 hash → npu.codegen_source_hash 로 export, provenance 에 기록
gate: EncodeSelfTest 인코더 자체 검사 , family 별 compare, 전체 프로그램 end-to-end 비교

v09_stats.analyze / cost

지표	정의
words / instructions	32-bit word 수 / packet 수 (DMA = 4 words = 1 instruction)
opcodes	각 packet 첫 word 의 하위 byte histogram
dma_load/store_bytes	Σ rows × cols × 4 (이동 bytes, strided span 아님)
sram_peak_bytes	max(커널 top) ÷ 2 (pool + scratch + buffers)
global_image_bytes	plan.top × unit_bytes
cost	(dma_bytes, words, sram_peak_bytes) 사전식

Native/Python 일치는 두 구현이 같은 계약을 따른다는 근거입니다 . 둘이 공유한 모델 수식의 오류는 HF/NumPy 또는 LLVM 기준으로 따로 확인합니다 .

근거 : npu_compiler/v09_contract.json, native.py, npu_codegen/build_identity.py, v09_stats.py, tests/test_native_codegen.py

결과물 : NpuExecutable 과 export_bundle

Binary 만으로는 inference 결과를 얻을 수 없습니다 . 같은 plan 에 맞는 입력과 weight image, 그리고 출력 tree 의 shape/dtype/offset 이 필요합니다 .

파일	담긴 내용	담기지 않은 것
program_memory.bin	little-endian uint32 명령 stream (HALT 로 끝)	weight 값 전체
program.json	statistics, memory(addresses/sizes), schedules, sram_layouts, joint_memory, kernel_records, dram_compaction, fusion, residency, outputs, cache_append, provenance, compilation	실장치 측정 cycle (hardware_measurement = null)
lowered.py	선택된 semantic IRModule (TVM Script)	최종 주소를 포함한 runtime 상태
scheduled_NNNN.py	joint-memory 에서 실제 선택된 호출별 TIR	원래 모델 설명 전체
global_memory.bin	values 를 지정한 경우 초기 DRAM image	지속적인 SRAM 상태

Runtime 이 필요로 하는 것 : program words + plan.address(입력 ·parameter 위치) + plan.outputs(leaf 별 shape/dtype/offset) + cache_append(있으면). C-model 은 program_memory.bin 과 global_memory.bin 을 읽고 saved_global_memory.bin(HALT 시 1 개 , SNAPSHOT 마다 추가) 과 perf_counters.txt 를 씁니다 .

C-model counters: words_executed, gload/gstore, dma_cells_loaded/stored, loads/saves, vector_ops, matrix_ops, vquant/vdequant → v09_stats.check_counters 가 정적 해독과 대조

```
class NpuExecutable:
    words, plan, func, kernel_count, target, module
    param_names   # 최종 signature 순서
    image_bytes = plan.top * plan.unit_bytes
    build_image(values) → npu_link.build_image
    run(values) → (output, counters)   # C-model
    save(path)  # little-endian <u4
    export_bundle(dir, values=None)

# provenance: python/numpy/torch/transformers,
#   tvm 0.19.0 commit, llvm 18.1.8, compiler_head,
#   compiler_source_sha256, native .so sha256,
#   cmodel_source_sha256, contract_sha256
```

작은 예제 bundle/program.json 발췌

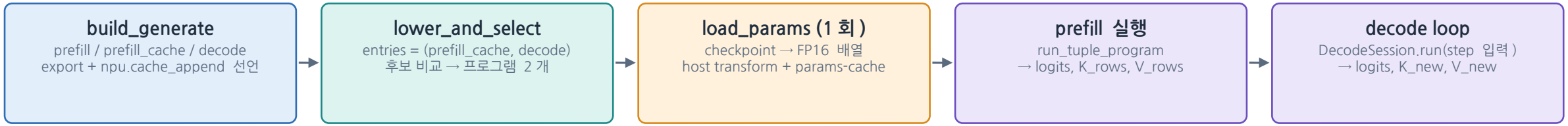
```
"statistics": {"words": 1577, "instructions": 1562,
   "dma_packets": 5, "dma_bytes": 41100,
   "sram_peak_bytes": 42508, "global_image_bytes": 41100}
"outputs": {"name": "gv", "shape": [64, 64],
   "dtype": "float16", "offset_bytes": 32896}
"kernel_records": [{"name": "fused_matmul_add_silu",
   "backend": "native", "sram_peak_nibbles": 85016,
   "arguments": [{"address_bytes": 0, "size_bytes": 16384}, …]}]
```

작은 pipeline bundle 은 입력 image 까지 보존합니다 . 전체 모델은 수 GB image 대신 측정값과 hash 를 evidence 로 남겼습니다 .

근거 : npu_compiler/npu_target.py NpuExecutable, docs/current/09_isa.md, docs/examples/pipeline/bundle/

Runtime 전체 흐름 : generate() 는 프로그램 2 개를 한 번 링크하고 입력만 바꿔 반복한다

배포된 모델 = prompt 길이 S 의 prefill_cache 1 개 + 고정 capacity C 의 decode 1 개 . 장치에 동적 shape 가 없으므로 decode 는 항상 C+1 슬롯을 attention 하고 mask 가 현재 길이를 전달합니다 .




```
def generate(family, config, assets, token_ids, steps, capacity=None, ...):
    capacity = seq + steps - 1 if capacity is None else capacity
    for cap in used:                                    # bucket 별 ( 기본 1 개 )
        mod, params, cfg = family.build_generate(config, seq, cap, last_logits=True)
        weights = load once; ABI signature must match across buckets
        modules[cap] = _lower(mod, fusion, compiled=programs[cap],
                              entries=("prefill_cache", "decode"))
    inputs = family.runtime_inputs(assets, cfg, tokens)      # embeds, mask
    cos, sin = family.rope_inputs(cfg, np.arange(seq))
    (logits, k_rows, v_rows) = execute(cap, "prefill_cache", {...})
    generated = [argmax(logits[-1].astype(float32))]        # host greedy
    k_cache[:, :, :, :seq] = k_rows;  v_cache[:, :, :seq, :] = v_rows
    for step in range(1, steps):
        position = seq + step - 1
        step_inputs = {input_embeds: embedding([last]), cos, sin: rope_inputs([position]),
                       mask: length_mask(cap, position)}    # + cache ( 첫 step 만 )
        (logits, k_new, v_new), words, decode = execute(cap, "decode", step_inputs, decode)
        generated.append(argmax(logits[-1]))
```

runner	무엇을 실행하나
run_stateful_tuple_program (기본)	DecodeSession: persistent image + DMA append tail
run_tuple_program	호출마다 image 전체 재구성 (cache 를 host 가 전달)
llvm_runner	같은 프로그램을 tile_order_reference 로 CPU 실행 (step 별 cross-check)

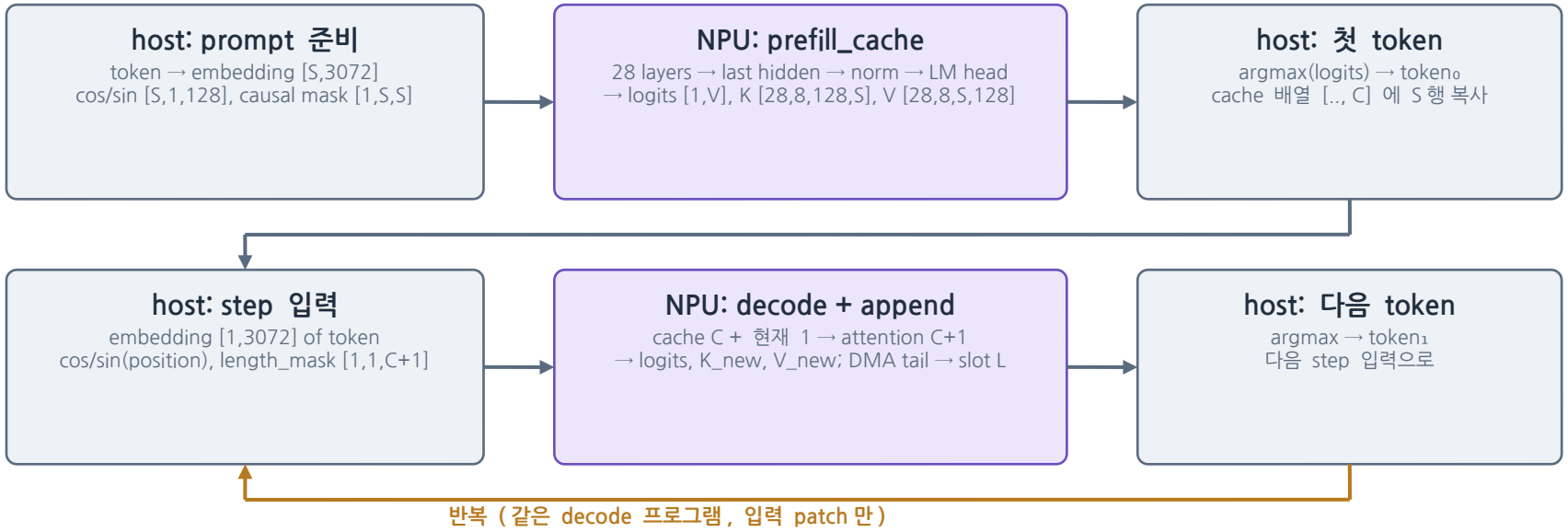
프로그램 수 ≠ layer 수 ≠ 후보 수 : 기본은 S 에 대한 prefill 1 개와 C 에 대한 decode 1 개 . 28 층은 하나의 entry stream 안에 모두 전개되어 있습니다 .

Greedy argmax 는 host 에서 합니다 . 마지막으로 예측한 token 은 아직 NPU 입력으로 처리되지 않았으므로 생성 token 수와 decode 실행 횟수가 한 번 차이 납니다 .

근거 : npu_compiler/npu_generate.py generate(), run_tuple_program, DecodeSession; d_compiler/run_nn_generate.py

Prefill 프로그램과 decode 프로그램 : host 와 NPU 가 나누어 맡는 일

Prefill 은 prompt S 개 token 을 한 번에 , decode 는 새 token 하나와 과거 cache 를 입력으로 다음 logits 를 만듭니다 .



mask	값
prefill causal [1,S,S]	대각선 위 – 1e4
decode length [1,1,C+1]	유효 슬롯 0, 나머지 – 65504, 마지막 (현재) 0
이유	FP16 floor; softmax 가 exp 를 0 으로

	Prefill (S=7)	Decode (C=9)
device 입력	input_embeds [7,3072], cos/sin [7,1,128], mask [1,7,7], packed_params	input_embeds [1,3072], cos/sin [1,1,128], k_cache [28,8,128,9], v_cache [28,8,9,128], mask [1,1,10], packed_params
출력	logits [1,128256] (last_logits), K [28,8,128,7], V [28,8,7,128]	logits [1,128256], K_new [28,8,128,1], V_new [28,8,1,128]
프로그램 통계 (기록)	8,582,352 words, DMA 6.46 GB, SRAM peak 8.37 MB	7,310,801 words (+ append tail 13,896 words), DMA 6.44 GB, SRAM peak 7.34 MB

Prefill 의 logits 는 마지막 hidden 행만 (last_logits). 전체 위치 logits 를 반환하는 prefill API 와 구분합니다 .

근거 : docs/current/10_runtime_llama.md, npu_compiler/nn_models/llama.py (runtime_inputs, causal_mask), npu_generate.length_mask

고정 shape 와 유효 길이는 다르다 : capacity C=9, 유효 길이 L=7

Decode 는 기존 C 개 슬롯과 현재 K/V 하나를 논리적으로 연결해 C+1 길이로 attention 합니다 . 현재 token 의 임시 위치는 C, persistent cache 에 기록할 위치는 첫 빈 슬롯 L 입니다 .

이번 attention 이 보는 논리적 입력 (C+1 = 10 슬롯)

0	1	2	3	4	5	6	7	8	9
과거 (유효)	과거 (유효)	과거 (유효)	과거 (유효)	과거 (유효)	과거 (유효)	과거 (유효)	빈 슬롯	빈 슬롯	현재 token

mask [1,1,10]: 0 0 0 0 0 0 0 −65504 −65504 0

cache 갱신 : 현재 K/V 를 persistent 슬롯 L = 7 에 기록 → 다음 step 의 유효 길이 8

```
def length_mask(capacity, length, dtype=np.float16):
    mask = np.full((1, 1, capacity + 1), -65504.0, dtype=np.float32)
    mask[0, 0, :length] = 0.0        # 유효 prefix
    mask[0, 0, capacity] = 0.0       # 현재 token ( 임시 슬롯 C)
    return mask.astype(dtype)
# AppendPlan.position(mask): 유효 prefix 길이 L 복원 , L >= C 이면 "cache is full"
```

구분	값 (C=9, L=7)
persistent cache shape	K [28,8,128,9], V [28,8,9,128] (고정)
attention 길이	C + 1 = 10 (항상)
현재 token 의 임시 위치	C = 9 (concat 결과의 마지막)
기록 위치	L = 7 (첫 빈 슬롯), 다음 step L = 8
빈 슬롯의 계산	shape· 명령에 남아 있음 . mask 가 기여만 막음

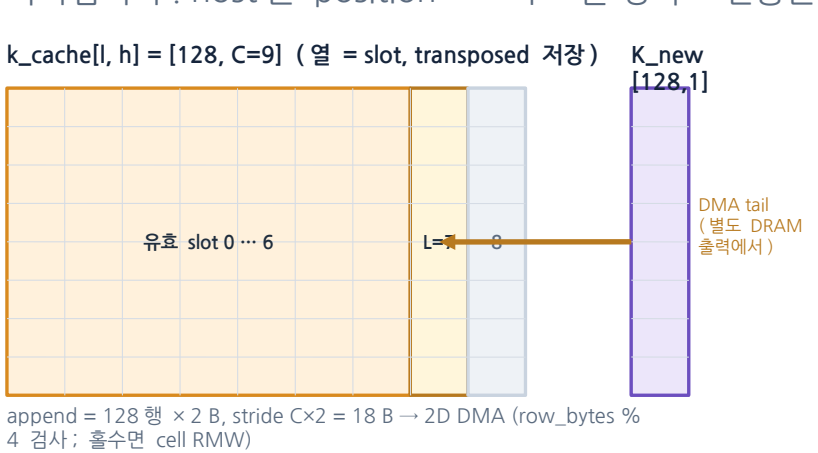
Prefill → decode 초기화 : host 가 prefill K/V 를 capacity 크기 배열의 앞부분에 복사합니다 . 이후 step 에서는 새 K/V 가 별도 DRAM 출력으로 나오고 program 끝의 DMA tail 이 cache 의 올바른 슬롯으로 옮깁니다 . " 매 step NPU 가 마지막 슬롯에 저장하면 CPU 가 다시 옮긴다 " 는 설명과 다릅니다 .

큰 음수는 FP16 −65504 입니다 . 의미상 제외할 슬롯도 계산은 수행되며 , capacity 전체 계산이 현재 decode 길이 비용의 한계입니다 (bucket 으로 완화).

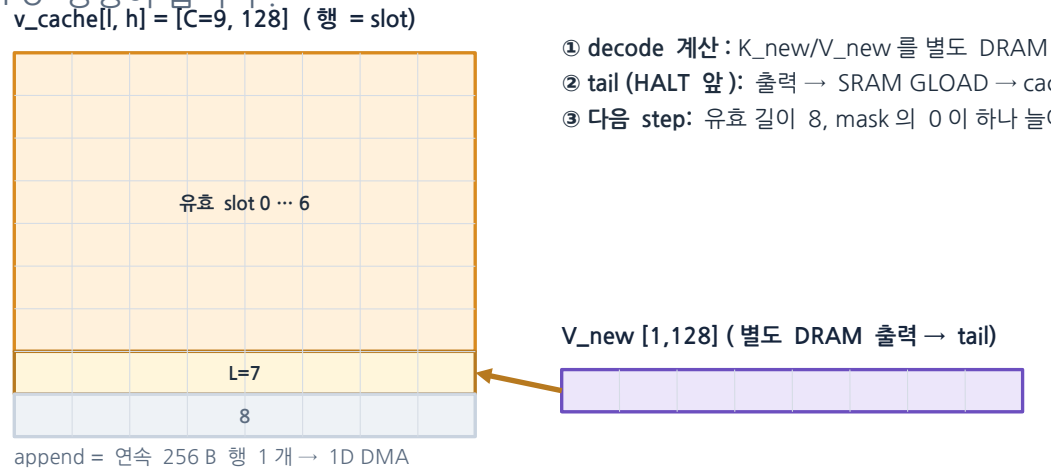
근거 : docs/current/10_runtime_llama.md, npu_generate.length_mask, npu_cache.AppendPlan.position

KV append 는 누가 실행하는가 : npu.cache_append 계약 → AppendPlan → DMA tail

K cache 는 [128, C] 로 transposed 저장이라 새 K(128×1) 는 열 하나 = 행마다 2 B 를 stride C·2 B 로 쓰고 , V cache 는 [C, 128] 이라 새 V 는 연속 256 B 행 하나입니다 . host 는 position 으로 주소를 정하고 전송은 NPU 명령이 합니다 .



append = 128 행 × 2 B, stride C×2 = 18 B → 2D DMA (row_bytes % 4 검사 ; 홀수면 cell RMW)



append = 연속 256 B 행 1 개 → 1D DMA

① **decode 계산 :** K_new/V_new 를 별도 DRAM 출력에 SAVE→GSTORE
② **tail (HALT 앞):** 출력 → SRAM GLOAD → cache[…, L] GSTORE. 주소는 position fixup
③ **다음 step:** 유효 길이 8, mask 의 0 이 하나 늘어남

```
mod["decode"] = npu_cache.declare(mod["decode"],
    [("k_cache", 1, 3), ("v_cache", 2, 2)])   # (param, output idx, time axis)
templates = [_template(parity) for parity in (0, 1)]   #  짝 / 홀  position
def words(position):
    words, fixups = templates[position % 2]
    for index, coeff in fixups: words[index] += (position // 2) * coeff   # g_addr 만 재배치
```

AppendPlan 검사	조건
출력 tuple	FP16, output[i].shape == cache.shape with axis → 1
alias	source(출력) 와 destination(cache) 이 겹치지 않음
capacity / mask	모든 slice 동일 ; mask FP16 [1,1,C+1], position 은 mask 에서 복원

기록 (C=9, 28 layers)	bytes / words	뜻
host 입력 patch (step 당)	6,676 B	embedding 6,144 + cos 256 + sin 256 + mask 20
새 K/V 논리 크기	114,688 B	2 × 28 × 8 × 128 × 2 B
append DMA 전송	458,752 B	layout· 정렬 · 이동 명령 때문에 논리 크기보다 큼
decode base / 실행 stream	7,310,801 / 7,324,697 words	tail 13,896 words; 두 번째 decode 의 program suffix patch 55,568 B

cache 이름을 추측하거나 layer 수를 codegen 에 하드코딩하지 않습니다 . metadata 의 parameter·output index·time axis 로만 append 를 만듭니다 .

근거 : npu_compiler/npu_cache.py (declare, AppendPlan, _template, words), docs/current/10_runtime_llama.md

DecodeSession 과 PersistentTransport: 무엇이 재사용되고 무엇이 바뀌는가

이전 실행의 최종 global image 파일을 다음 입력으로 이어 쓰고, 바뀐 host 입력과 program suffix 만 patch 합니다. 아래 두 strip 은 step 마다 실제로 바뀌는 바이트입니다.

대상	step 사이의 동작
기본 decode 계산 명령	재사용 (prefix 동일)
KV append 명령의 주소	position 에 맞춰 변경 (program suffix patch)
embedding, cos/sin, mask	host 가 새 값으로 patch (6,676 B)
DRAM weight	초기 image 이후 host 가 다시 쓰지 않음 (rewrite 0 B)
SRAM 의 weight 조각	실행 중 NPU 가 DRAM 에서 다시 가져옴
Persistent DRAM cache	saved image → 다음 입력 image (rename)
C-model 내부 SRAM	호출 사이 지속성을 전제하지 않음

Persistent transport 의 실제 뜻 : host weight rewrite 를 없애는 transport 재사용입니다. 지속적인 simulator process 나 실제 장치의 zero-copy runtime 이 아니며, C-model subprocess 는 호출마다 전체 image 를 읽고 씁니다. NPU_V09_TMPDIR 로 큰 image 의 임시 위치를 지정합니다.

검사 : position 순차성, cache 입력 재전달 거부, 입력 shape/dtype, HALT 로 끝나는 프로그램, patch 범위가 image 안인지, 최종 image 가 정확히 1 개인지.

global_memory.bin (≈6.43 GB): step 마다 바뀌는 곳



program_memory.bin (7,324,697 words): 바뀐 첫 word 부터만 다시 씀



suffix patch 55,568 B (두 번째 decode). C-model subprocess 는 호출마다 두 파일을 읽고 saved_global_memory.bin(전체 image) 을 씀 → rename 으로 다음 입력.

```
changed = flatnonzero(program[:common] != self.program[:common])
file.seek(first*4); program[first:].tofile(file)      # program suffix
for offset, data in patches: file.seek(offset); data.tofile(file)
subprocess.run([mysim_v09]); saved.replace(global_path)
```

측정 층	지표	섞지 않을 것
compiler 정적 통계	DMA bytes, words, SRAM peak	실장치 latency
host 전송	input patch, program patch bytes	NPU bandwidth
simulator 파일 I/O	image read/write (전체 image)	zero-copy runtime

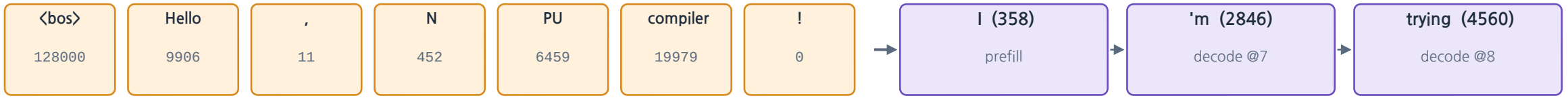
Compiler 의 DMA 통계, host patch bytes, simulator 파일 I/O 를 한 표에서 더해 "NPU bandwidth" 로 보고하지 않습니다. 측정 층이 다릅니다.

근거 : npu_compiler/npu_generate.DecodeSession, v09_runtime.PersistentTransport, docs/current/10_runtime_llama.md

실제 검증된 3-token 생성 : "Hello, NPU compiler!" → " I'm trying"

FP16, 28 layers, prompt 7 tokens, capacity 9, greedy 3 tokens. 보존된 기록이며 이번 문서 작업에서 전체 모델을 다시 실행하지 않았습니다 .

prompt 7 tokens



단계	NPU 에 넣는 것	계산 후 cache 유효 길이	예측 token	HF argmax 일치	logits cosine	최대 절대차
prefill_cache	prompt 7 tokens	7	358 (" I")	예	0.9999977148	0.03515625
decode 1	token 358, position 7	8	2846 ("'m")	예	0.9999958473	0.02343750
decode 2	token 2846, position 8	9	4560 (" trying")	예	0.9999929546	0.03906250

```
python d_compiler/run_nn_generate.py --model llama \
    --prompt "Hello, NPU compiler!" --tokens 3 --capacity 9
# --llvm: 같은 프로그램을 CPU 에서 , --cache-buckets, --fusion auto|off|on, --params-cache
DIR
```

판정 기준 / 기록	값
판정	유한 logits, token 일치 , cosine ≥ 0.999 (절대차는 기록만)
실행 시간	948.99 s (첫 entry 재사용 검증 ; cold compile 아님)
graph 탐색	120 s 예산 소진 → unfused 기준 module
compiler source	sha256 44694fc6…, TVM 0.19.0 c4dc0c29, LLVM 18.1.8

범위 : 한 prompt 의 짧은 greedy 생성입니다 . 긴 문맥 품질 , 다양한 prompt, 양자화 generation 까지 검증됐다고 확대하지 않습니다 .

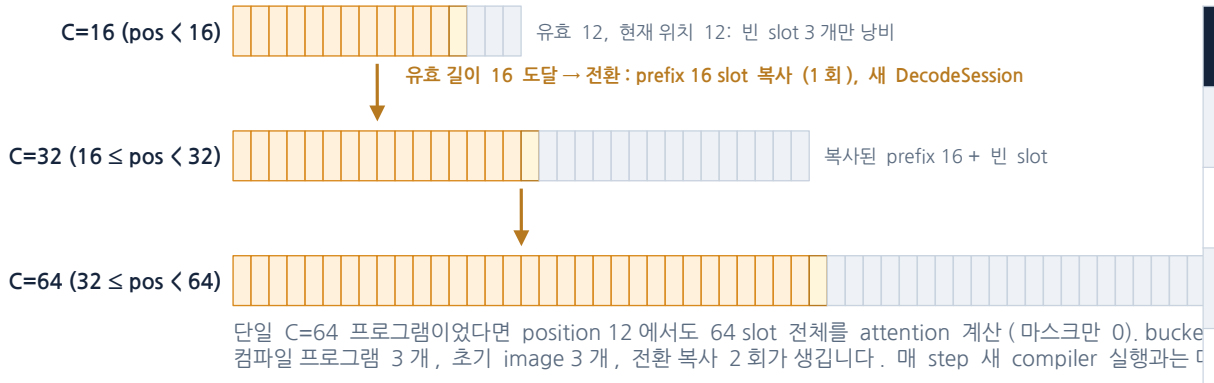
최종 문자열 " I'm trying". 마지막 4560 은 아직 NPU 입력으로 처리되지 않았으므로 이 시점 cache 에는 그 token 의 K/V 가 없습니다 .

근거 : docs/evidence/current/llama_validation.json, completion.json (compiler source sha256 44694fc6…)

Bucket, layer 반복 , 그리고 C-model 실행 규약

여러 capacity 를 미리 컴파일하고 유효 길이가 커질 때 큰 bucket 으로 바꿀 수 있습니다 . 전환 시 유효 prefix 를 한 번 복사합니다 . 28 층은 최종 stream 에 모두 전개되며 schedule 판단만 재사용합니다 .

cache_buckets = [16, 32, 64] (capacity 64): slot 사용과 전환



C-model (mysim_v09.cpp) 파일 규약

파일	방향	내용
program_memory.bin	입력	little-endian uint32 명령 words
global_memory.bin	입력	초기 DRAM image (32-bit cells)
saved_global_memory.bin	출력	SNAPSHOT(0xF0) 마다 + HALT(0xFF) 에서 전체 image 추가
perf_counters.txt	출력	words_executed, gload, gstore, dma_cells_loaded/stored, loads, saves, vector_ops, matrix_ops, vquant, vdequant

layer 반복	내용
재사용하는 것	같은 구조 PrimFunc 의 schedule recipe, 영역 /entry 비용 cache
호출마다 반영하는 것	layer 별 weight 주소 , 입력 · 출력 값 , residual 관계 → 최종 stream 에 28 층 모두 전개
main backend 와의 차이	main 은 host 가 같은 layer 프로그램을 weight 만 바꿔 반복 실행 (부록 A1)

```
def run(program_words, global_cells):        # v09_runtime.py
    build_v09_model()   # g++ -std=c++17 -O2,  소스가 바뀌면 재빌드
    with TemporaryDirectory(dir=NPU_V09_TMPDIR):
        write program_memory.bin, global_memory.bin; subprocess.run([mysim_v09])
        images = split(saved_global_memory.bin, image_bytes)   # snapshot list
    return images, counters   #  크기 불일치 → 디스크 부족 진단 메시지
```

기능 검증용 실행기 : cycle 을 세지 않고 counters 만 냅니다 . words_executed 와 DMA cells 는 정적 해독과 대조하지만 latency 로 바꾸지 않습니다 .

C-model 의 성공은 기능 검증입니다 . RTL 구조 (256 lane 버퍼 , VLEN tail, accumulator 상태) 의 검증을 대신하지 않습니다 .

Compile 시간은 어떻게 제어되는가 : build 안의 cache, 예산 , phase 계측

compilation() context 는 build 하나 동안 살아 있으며 nested 호출은 바깥 context 를 재사용합니다 . 모든 cache 는 structural_hash 로 찾고 structural_equal 로 확인합니다 .

cache	키	재사용 대상	Llama 기록 (hit / miss)
recipe	PrimFunc 구조 hash (symbol/op_pattern 제외) + profile(sram_bytes· 탐색 설정 포함 , joint/beam/graph 시간 제외) + scratch/slot	커널의 schedule + SRAM layout	272 / 383
trial	(tile, reuse_sram, scratch, slot, stripe, mapping, tile_pad, capture)	후보 측정 결과 (실패 포함), 용량은 키 밖	184 / 248
probe (cost)	SRAM binding, scratch, 상수 , native mode, DRAM 주소 mod 4	비용만 ; 승자는 실제 주소로 재 emit· 검증	294 / 270
region	fusion 영역의 entry identity (param 이름 제외)	FuseOps 영역 unfused/fused 비용	0 / 40
EntryCache	entry identity: private GlobalVar 이름 정규화 , npu.repeated_gemv/op_pattern 제거한 module hash	graph 후보 사이 linked entry 전체 (LRU 256 MiB / 32)	0 / 3

예산 / 제어	값
graph_search_seconds	기본 120 / E10 None
schedule_max_attempts	기본 64 / E10 128
schedule_space_trials	기본 8 / E10 16 legal 목표
schedule_probe_word_budget	max(설정 , 기준 words × legal 목표) / E10 10 억
schedule_search / seed	space (기본) · meta_schedule = ReplayFunc 추첨 , seed 0
schedule_database	선택형 trace DB (reuse / tune). E10 미사용
early abort	entry 하나가 incumbent 보다 나쁘면 후보 중단
joint memory	baseline 보다 싸야 채택

progress 이벤트와 phase

이벤트 : phase_begin/end(seconds), optimization_budget_started, candidate_begin/end(status: duplicate/legal/noncompetitive/unsupported/budget_exhausted), entry_compiled/entry_reused

phase: schedule_and_sram, schedule_tir, schedule_sram, schedule_probe_*, entry_identity/compile/statistics, memory_planning, joint_schedule_memory, native_codegen, residency, peephole, dram_compaction, fusion_regions, program_cost, meta_schedule_replay / generate / extract_tasks (DB·ReplayFunc 사용 시)

describe(): hit/miss, inclusive·self seconds, 호출 수 , native fallbacks/rejections, meta_schedule(enabled/mode/hits/misses/writes/rejections) → plan.compilation (bundle 에 기록)

Shape 만 같으면 같은 결과로 취급하는 cache 가 아닙니다 .

구조 ·profile· 입출력 배치가 키에 들어가고 , 반복 layer 에서도 실제 weight 주소가 들어간 명령은 호출별로 확인합니다 .

근거 : npu_compiler/npu_compilation.py (recipe, trial, probe, region_cost, EntryCache, optimization_budget, phase, describe)

실험 근거의 세 수준과 서로 다른 지표

모든 숫자를 하나의 성능 점수로 합치지 않습니다. 컴파일 시간, 장치가 요청한 전송량, 명령 크기, 저장 공간, 수치 결과를 따로 측정합니다.

표시	의미	이 발표의 예	source hash
새 실행 검증	이번 문서 작업에서 현재 소스로 작은 입력을 실제 실행 (C-model + LLVM)	E1 pipeline (fusion off/auto), E2 SRAM 용량 5 구성	44694fc6…
보존된 실행 검증	기존 실행 로그·JSON·source hash 를 그대로 보존	E3 두 matmul 전후, E6 전체 Llama 3 tokens, E7 회귀 gate 17 suites	44694fc6… (E3 는 42843f7e…)
정적 분석 / 단일 측정	명령·배치 분석 또는 한 번의 compile wall time (executed=false)	E4 실제 폭 MLP, E5 compile 시간 비교	7d13cfd9… 등 (개선 단계별로 다름)

지표	뜻	무엇이 아닌가
DMA bytes	정적 명령 해독의 GLOAD/GSTORE 이동량 (C-model counters 와 대조)	host patch, simulator 파일 I/O
ISA words	32-bit 명령 저장 공간	실행 cycle, 명령 수 (instructions)
SRAM peak	커널 top 의 최댓값 (pool + scratch + buffers)	살아 있는 값 크기의 합
DRAM image	plan.top × unit_bytes (weight + 입력 + 중간 pool + 출력)	DMA 전송량 (우연히 같을 수 있음)
compile seconds	단일 wall-clock 관측 (다른 작업과 동시 실행 가능)	반복 평균, cold compile 보장

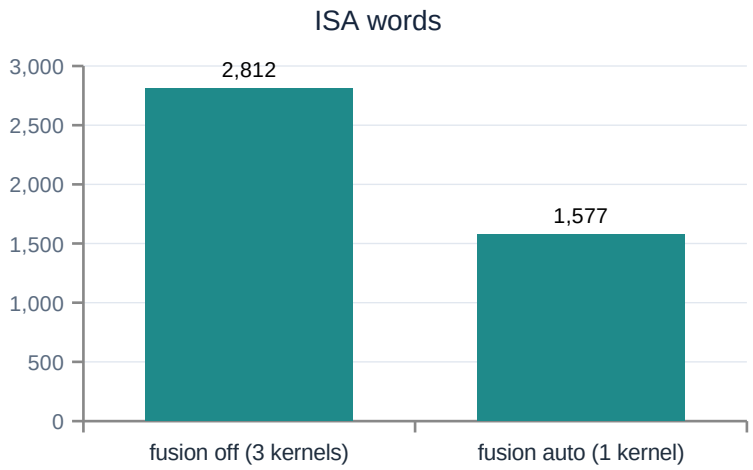
provenance 기록 항목	
compiler_source_sha256	모든 .py/.cc/.h
tvm_version / commit	0.19.0 / c4dc0c29
llvm_version	18.1.8
native_sha256	libnpu_codegen.so
cmodel_source_sha256	_poc/mysim_v09.cpp
contract_sha256	v09_contract.json
python / numpy / torch	3.11.15 / 2.4.6 / 2.12.0+cpu

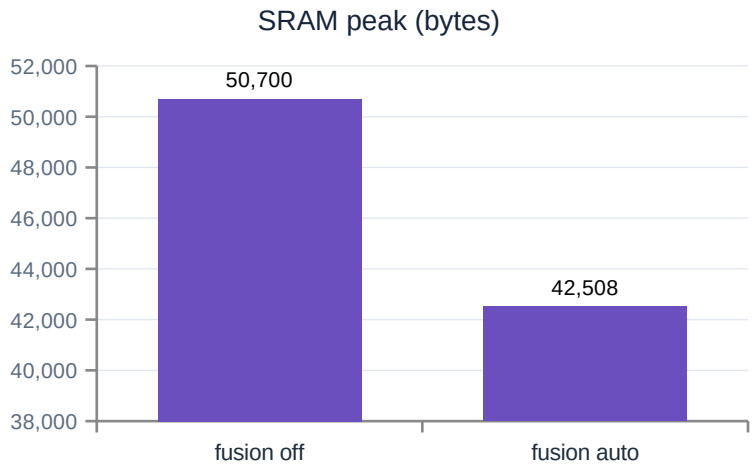
ISA words 나 simulator wall time 으로 하드웨어 latency 를 주장하지 않습니다. 실장치 timing 이 없을 때는 words/DMA 감소까지만 말합니다.

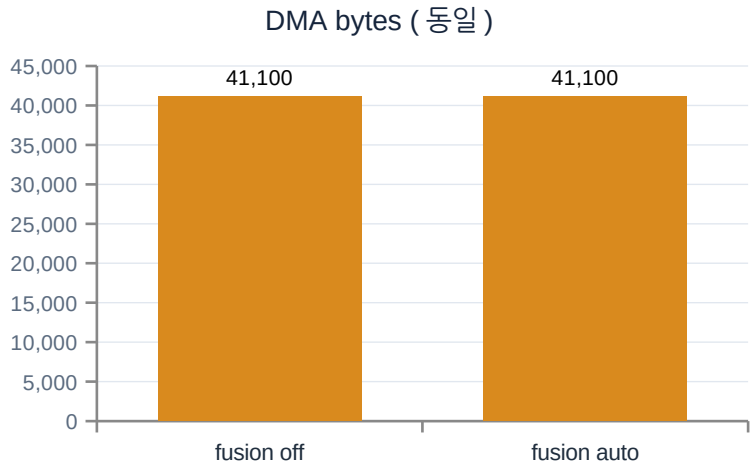
근거 : docs/current/11_experiments.md, docs/evidence/README.md, examples/reproduction.json

E1. Fusion 은 DMA 가 같아도 명령과 작업 공간을 줄일 수 있다

SiLU(X[64,128] @ W[64,128].T + bias[64]), seed 914, FP16. 현재 public build 로 fusion off/auto 를 각각 컴파일해 같은 값으로 C-model 을 실행했습니다 . 두 구성 모두 SRAM 전달 기능을 사용합니다 .







off 의 커널별 words	matmul 43	add 391	silu 2,373	합 2,812 (+ 상수 GLOAD 4 + HALT 1)
auto	fused_matmul_add_silu 1,572			합 1,577
C-model counters	gload 4, gstore 1	loads 644 → 388	saves 449 → 258	vector_ops 512 → 288, matrix_ops 2

검증	결과
fused vs unfused FP16 출력	bit-exact
LLVM reference (K=64 grouping)	최대 절대차 0
DMA counters vs 정적 해독	일치

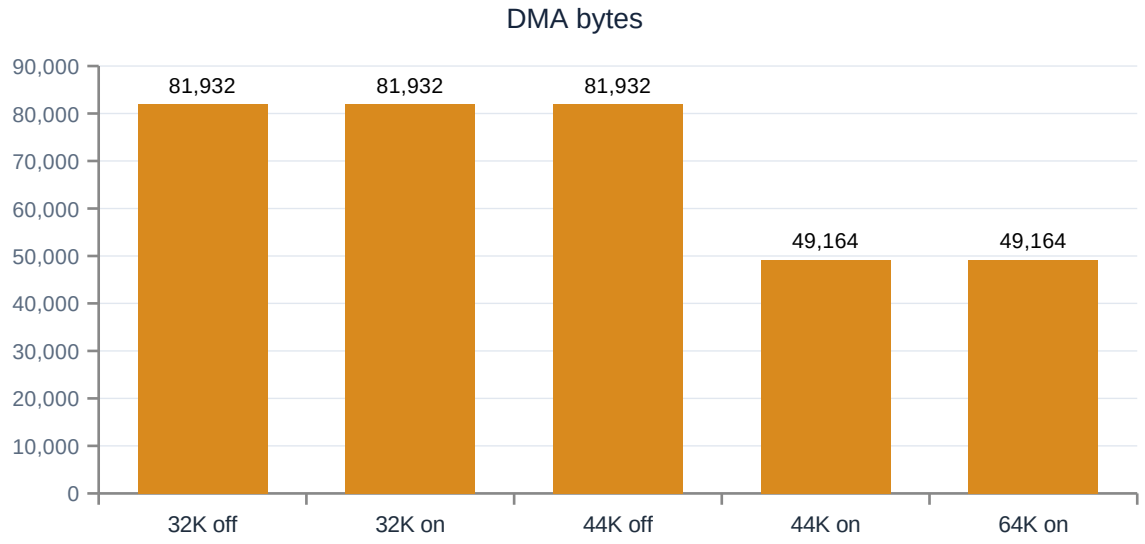
왜 words 가 줄었나 : add·SiLU 가 같은 SRAM tile 에서 이어져 descriptor 설정 (0x80 1,171→606) 과 LOAD/SAVE(644/449→388/258) 가 줄고 , 축 결합으로 vector 행이 64→128 폭이 되어 명령 수가 절반 (0x16/0x0f 64→32) 이 되었습니다 . DMA 는 입력 · 출력이 그대로라 같습니다 .

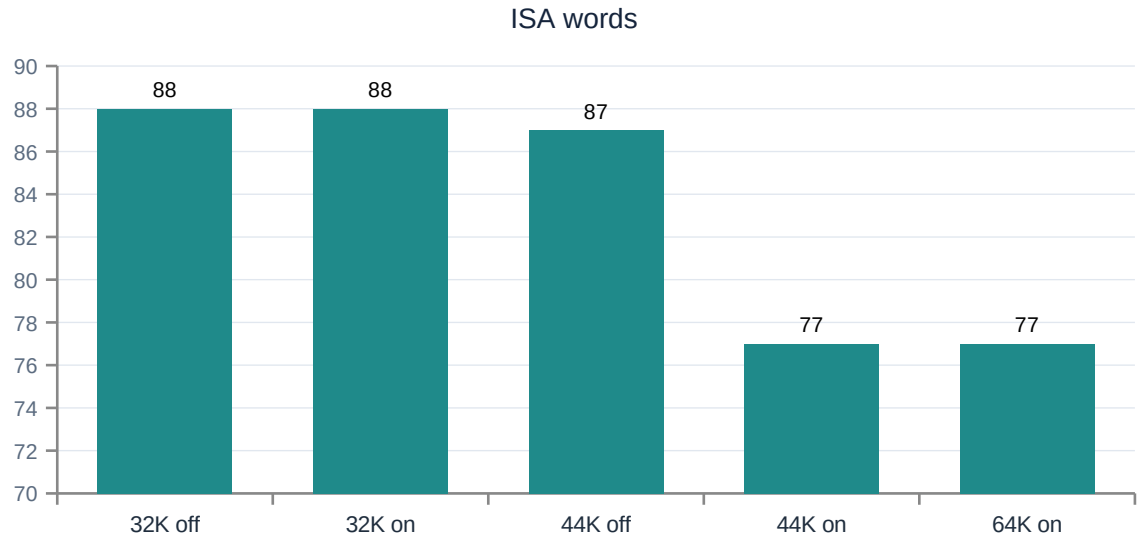
전송량은 같아도 명령과 작업 공간이 줄어드는 사례입니다 . 이득의 형태는 예제마다 다르므로 실제 비용으로 확인합니다 .

근거 : docs/examples/pipeline/stage_summary.json (새 실행 검증)

E2. SRAM 을 늘리면 언제 중간 DRAM 왕복이 사라지는가

X[64,64] @ W1[64,128] @ W2[128,64], seed 928, fusion off. 보관 탐색 옵션과 SRAM 용량만 바꾸어 5 개 구성을 실제 실행했습니다 . 모든 구성이 같은 FP16 출력이며 DMA counters 가 일치했습니다 .





SRAM 용량	whole-tensor 보관 탐색	DMA bytes	words	실제 SRAM peak	DRAM image	해석
32 KiB	off	81,932	88	26,124	65,548	H 를 DRAM 에 저장 · 재읽기
32 KiB	on	81,932	88	26,124	65,548	보관 불가 : 첫 matmul 이 들어가지 않음 → 동일 선택
44 KiB	off	81,932	87	34,316	65,548	후단 residency/peephole 만
44 KiB	on	49,164	77	42,508	49,164	H 32,768 B 왕복 제거 , DRAM pool 제거
64 KiB	on	49,164	77	42,508	49,164	추가 이득 없음

"SRAM 을 최대한 채우기 " 보다 실제 재사용과 작업 공간의 관계가 중요합니다 . 용량이 문턱을 넘는 순간에만 왕복이 사라집니다 .

근거 : docs/examples/memory/measurements.json (새 실행 검증)

E3 · E4. 직접 SRAM 출력과 미사용 DRAM 제거 : 작은 예제와 실제 폭 MLP

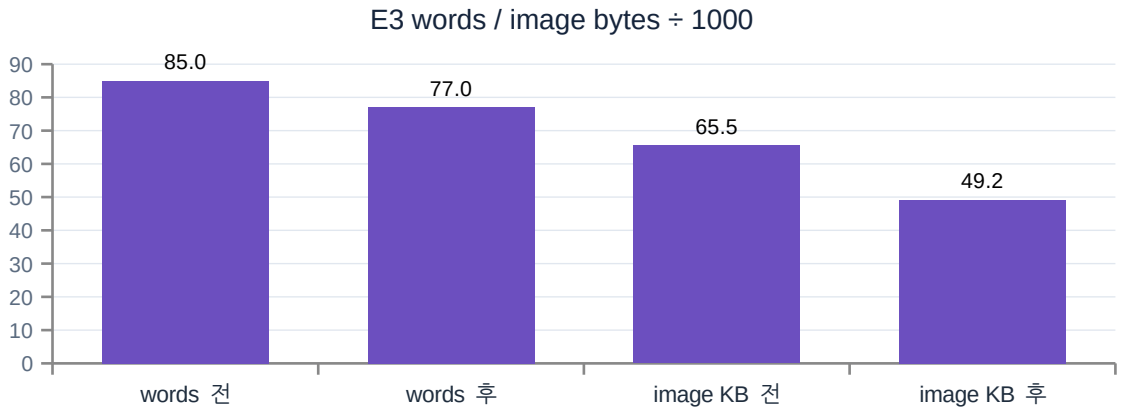
E3 는 44 KiB 두 matmul 에서 이미 SRAM 보관을 사용하는 상태로 pointwise 결합 · 출력 직접 저장 · 미사용 DRAM 제거를 비활성 / 활성한 보존 비교 . E4 는 M=64 실제 폭 MLP 의 정적 compile 기록 (executed=false).

E3. 44 KiB 두 matmul (보존 , 실행 bit-exact)

항목	개선 전	개선 후
ISA words	85	77
DMA bytes	49,164	49,164
SRAM peak bytes	42,508	42,508
DRAM image bytes	65,548	49,164
출력	bit-exact	

E4. 실제 폭 MLP M=64 (정적 compile, executed=false)

항목	개선 전	개선 후
선택 커널 수	5	5
ISA words	143,404	140,594
DMA bytes	152,174,604	152,174,604
SRAM peak bytes	7,446,540	7,446,540
DRAM image bytes	154,927,116	151,781,388



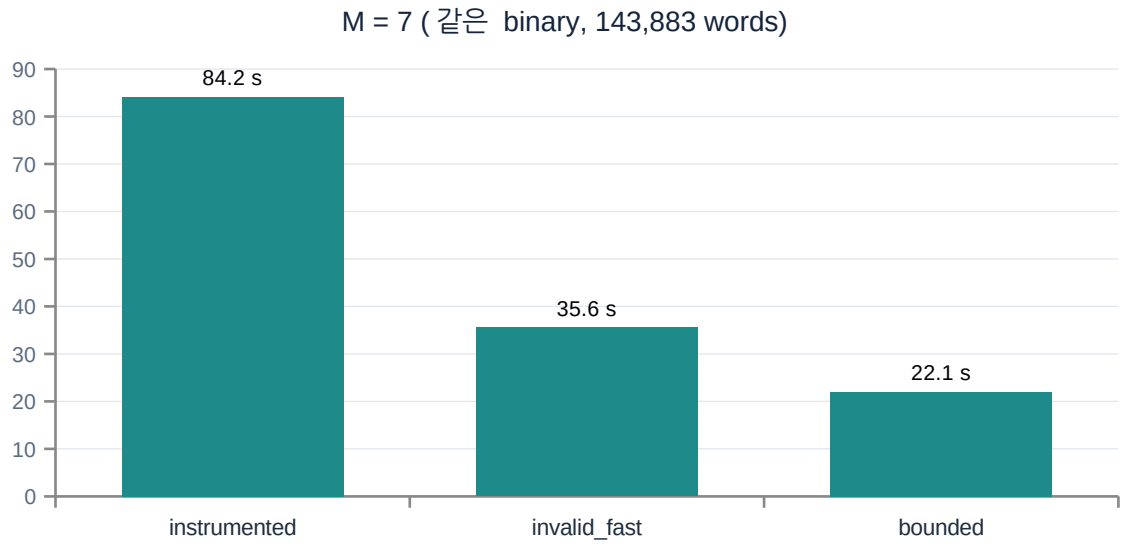
무엇이 바뀌었나 : pointwise 결합 , 출력 tile 직접 저장 (H view + stride), 미사용 DRAM pool 제거
E4 의 3 MiB: G/S/U/H 가 같은 기본 pool 을 재사용하므로 4 MiB 가 아니라 3 MiB 제거
E4 의 선택 : gate → SiLU → up → multiply → down 5 커널 유지 , 중간값은 SRAM 전달
혼동 주의 : E2 의 87→77(보관 on/off) 과 E3 의 85→77(직접 저장 · 정리) 은 다른 비교
fusion: fused epilogue words 를 줄였어도 최종 선택은 여전히 분리 커널 (커널 수가 아니라 비용으로 판단)

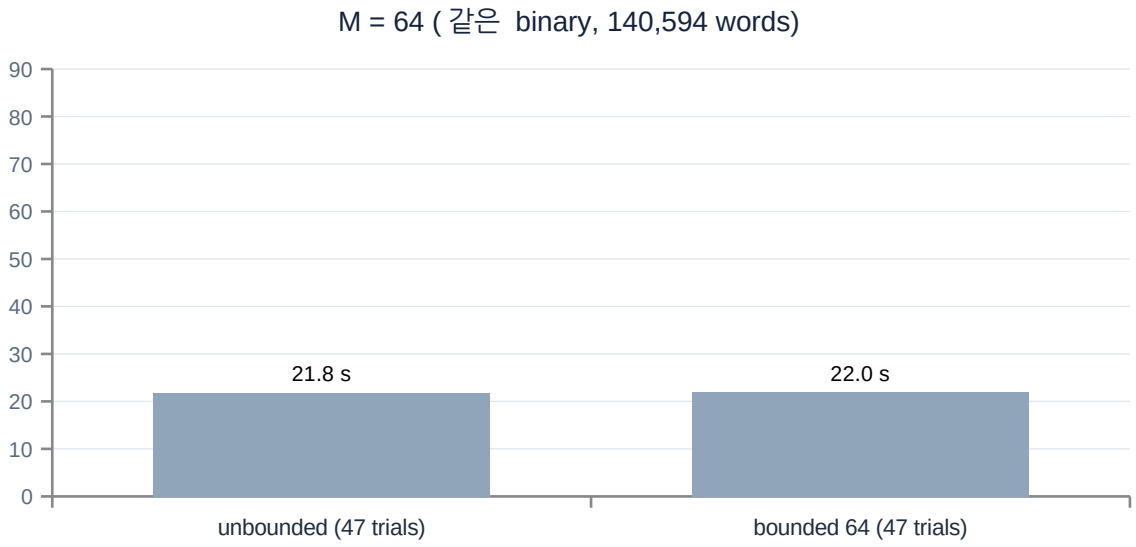
DMA 와 SRAM peak 가 그대로여도 명령과 image 가 줄어드는 개선이 있습니다 . 지표를 따로 보아야 어떤 최적화가 무엇을 바꿨는지 알 수 있습니다 .

근거 : docs/evidence/current/refinement.json, mlp64.json, examples/two_matmuls_recorded/program.json

E5. Compile 속도 개선은 코드 품질을 바꾸지 않았다 (binary hash 동일)

기존에 기록한 단일 wall-clock 관측값. 일부 측정 중 다른 검증도 실행됐습니다. 반복 평균이나 하드웨어 inference latency 가 아닙니다.





label	shape (M, K, N)	seconds	words	trial misses	word sha256	비고
instrumented_mlp7	7 × 3072 × 8192	84.163	143,883	347	c63b82a2…	개선 전
invalid_fast_mlp7	7 × 3072 × 8192	35.570	143,883	347	c63b82a2…	invalid accumulator schedule 중복 검사 제거
bounded_mlp7	7 × 3072 × 8192	22.073	143,883	112	c63b82a2…	실패 포함 시도 제한
unbounded / bounded_mlp64	64 × 3072 × 8192	21.791 / 21.958	140,594	47 / 47	ec55566f…	제한이 작동하지 않은 경우 (시간 동일)

전체 build 의 기본 120 초는 최초 합법 module 이후의 추가 탐색 예산이며, 전체 검증의 948.99 s 는 완료된 첫 entry 를 재사용한 값이라 cold compile 시간이 아닙니다.

근거 : docs/evidence/current/compile_comparison.json

E6. 전체 Llama 3.2 3B 출력 검증과 프로그램 규모

FP16, 28 layers, prompt 7 tokens, capacity 9, greedy 3 tokens. HF 와 argmax 가 모두 일치하고 logits cosine ≥ 0.99999 입니다 .

실행	HF 와 같은 argmax	logits cosine	최대 절대차
prefill	358	0.9999977148	0.03515625
decode position 7	2846	0.9999958473	0.02343750
decode position 8	4560	0.9999929546	0.03906250

기본 entry	words	DMA bytes	SRAM peak	DRAM image
prefill_cache	8,582,352	6,457,527,556	8,374,292	6,429,587,576
decode (base)	7,310,801	6,438,980,676	7,342,100	6,427,349,032

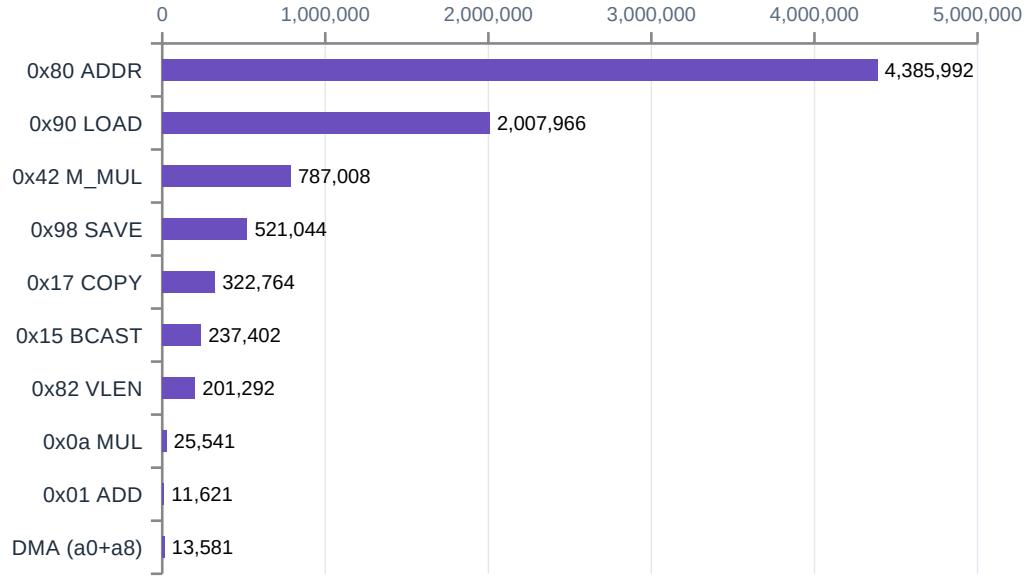
DMA 6.46 GB 의 구성 : weight 192 MiB × 28 layers + LM head 751.5 MiB ≈ 6.43 GB 를 panel 로 읽는 것이 대부분 . 저장은 17.4 MB

words 8.58M: 주소 descriptor(0x80) 51%, LOAD 23%, M_MUL 787,008 (28 layers 의 모든 matmul + LM head) 등 정적 전개

graph 탐색 : 120 s 예산 소진 → unfused 기준 module. " 모든 fusion 이 더 나쁘다 " 의 증거 아님

decode 실행 : base 통계 + append tail. 두 통계를 섞지 않음

prefill_cache opcode 구성 (words 기준 상위)



판정 : 유한 logits, token 일치 , cosine ≥ 0.999. 최대 절대차는 기록만 하고 threshold 로 쓰지 않았습니다 . 한 prompt 의 짧은 greedy 생성이므로 긴 문맥 · 다양한 prompt· 양자화 generation 은 별도 검증 범위입니다 .

수치 정확성 (HF 대비) 과 프로그램 비용 (words/DMA/SRAM) 은 별개의 표입니다 . 두 표가 같은 source hash 44694fc6… 를 가리킵니다 .

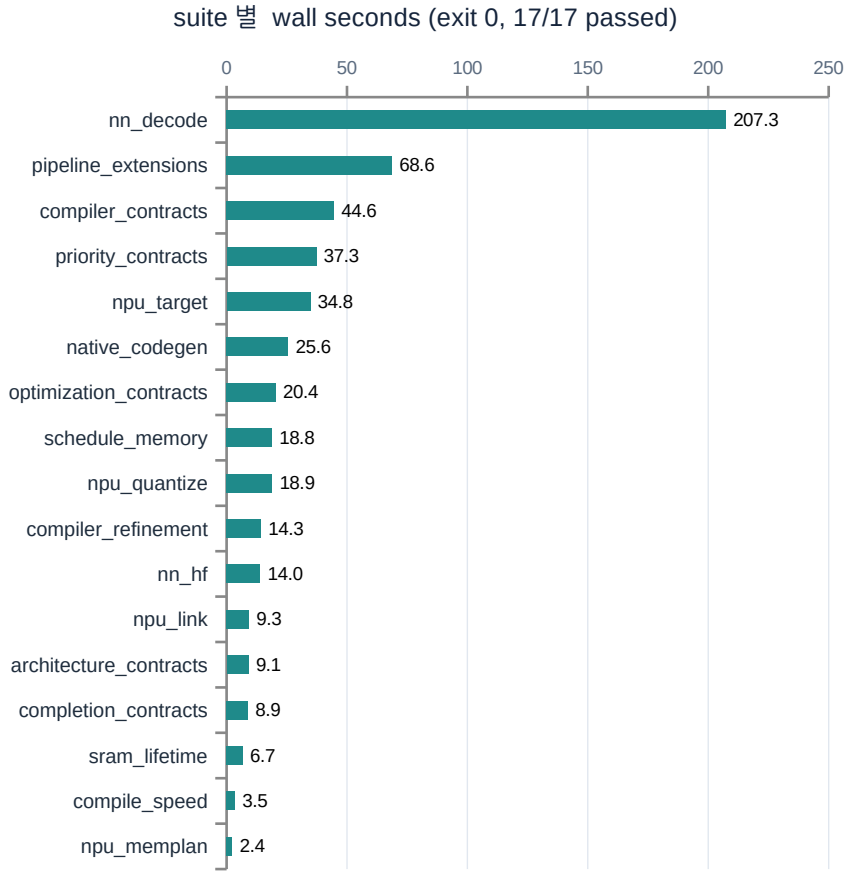
근거 : docs/evidence/current/llama_validation.json, llama_selection.json, completion.json

E7. 기능 검증 gate: 17 개 suite 가 확인하는 축

기존 최종 gate 는 17 개 검사 묶음이 실패 /skip 없이 완료되었습니다 . 묶음 안에 여러 assertion 이 있으므로 "17 개 shape 만 테스트했다 " 는 뜻이 아닙니다 . 이번 문서 작업에서는 재실행하지 않았습니다 .

검증 축	대표 검사 묶음	확인할 의미
표준 pipeline 과 ABI	architecture_contracts, compiler_contracts, npu_target	entry, output tree, 미지원 입력 거절
Schedule · 메모리	schedule_memory, sram_lifetime, npu_memplan	보관 , 주소 , 수명 충돌 , fallback
Fusion · 수치 · 확장	pipeline_extensions, optimization_contracts, priority_contracts	후보 지원 조건과 결과
명령 생성	native_codegen, npu_link	Python/native 계약과 연결
회귀 개선	compiler_refinement, compile_speed, completion_contracts	출력 직접 저장 · 정리 ·cache· 예산
Frontend · 생성	nn_hf, nn_decode	모델 의미와 cached decode
부록 기능	npu_quantize	작은 양자화 kernel/model

```
bash d_compiler/run_gate.sh --tests architecture_contracts schedule_memory native_codegen \
     --require-no-skips --output /tmp/npu-gate
# validation.json: 실제 subprocess exit, timeout, skip 기록 . "[PASS]" 문자열 검색으로 판정하지 않음
python d_compiler/docs/examples/reproduce.py --out /tmp/npu-doc-examples   # E1/E2 재현 ( 작은 compile 7 개 + C-model)
```



Native/Python 일치와 gate 통과는 계약 준수의 근거입니다 . 모델 수식 자체의 오류는 HF/NumPy·LLVM 기준 (E6, run_nn_npu.py) 으로 따로 확인합니다 .

근거 : docs/evidence/current/regression_validation.json, gate_logs/, d_compiler/run_gate.sh

E9. 탐색을 보완해도 512-token down 의 최종 명령은 같았다

실제 Llama layer-0 weight + 합성 activation. 단일 projection 의 비교이며 전체 모델 generation 검증이 아닙니다 .

측정 / 512-token down	Space	TVM ReplayFunc
추가 합법 후보	4 개	4 개
커널 탐색 시간	10.586 s	33.264 s
DMA bytes	112,197,644	112,197,644
ISA words	372,637	372,637
SRAM peak	5,349,388 B	5,349,388 B
binary / C-model	동일 binary	같은 프로그램 출력 검증

낭비를 줄이는 변경 : 같은 순회 · 복사 주기의 후보를 정규화하고 , 증명할 수 있는 용량 초과만 제거합니다 .

충분한 검사인지 확인 : 시간만 보지 않고 추가 합법 후보 수와 종료 사유를 기록합니다 .

관측한 한계 : 이 커널에서는 ReplayFunc 의 비용이 더 컸고 최종 명령은 같았습니다 . 학습형 비용 모델은 사용하지 않았습니다 .

다음 검증 : Fusion 효과는 전체 28-layer·256-token prefill 을 별도로 컴파일해 확인합니다 .

출력 검증 통과 : C-model 최대 절대차 0.000030517578125 · DMA counters 일치 · 관련 회귀 34 개 통과

불필요한 시도 감소와 trace 재사용을 확인했습니다 . 추가 탐색까지 포함한 총 시간이 더 짧아졌다는 비교는 아닙니다 .

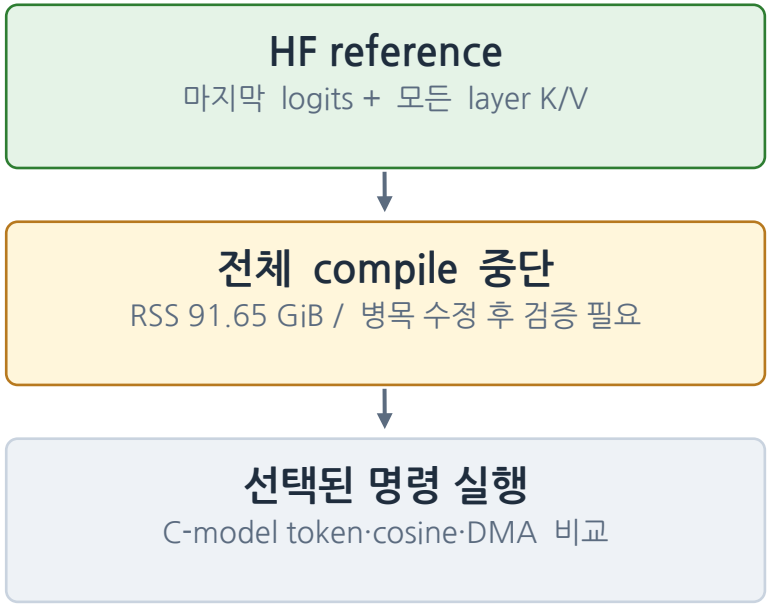
근거 : docs/current/11_experiments.md E9; evidence/current/llama_schedule_refinement.json

E10 중단 : 큰 cache fusion 의 명령 생성 병목을 수정했다

28 layers 전체 · prefill_cache 하나 · graph 시간 제한 None · matmul 합법 목표 16 / 최대 시도 128 · SRAM 8 MiB · 전체 재검증 미실행

Attention / Generic	Off	Vectors	Matmul
off	기준 구성	영역 10 종	영역 11 종
score	1 개 구성	영역 9 종	영역 10 종
full	1 개 구성	영역 9 종	영역 10 종

9 개 = graph 구성 수 . 영역 수는 종류이며 최종 채택 수가 아닙니다 .
동일 layer 의 판단은 재사용하고 실제 주소와 전체 수명은 다시 연결합니다 .



비교 기준도 SRAM 보관 ON. Fusion 이 전체 prefill 의 DMA·ISA words·SRAM 을 더 줄이는지 확인합니다 .
큰 K/V 출력은 각각 14 MiB. Decode 를 실행하지 않아도 이 prefill 함수가 만드는 출력입니다 .

병목 독립 검사는 통과했습니다 . 전체 256-token 재컴파일 · 출력 검증은 아직 수행하지 않았습니다 .

근거 : docs/current/11_experiments.md E10; evidence/current/llama_prefill256_full_search_progress.json

현재 지원 경계 : 무엇을 받아들이고 무엇을 거절하는가

ABI verifier 는 정의 · 호출 · 배치의 일관성을 검사하지만 임의 TIR 의 모든 index 접근이 안전함을 증명하는 정리 증명기는 아닙니다 . 지원 subset 밖의 계산을 조용히 통과시키지 않습니다 .

항목	지원 또는 제한
Shape	정적 tensor shape. FP16 matmul rank ≥ 2, batch broadcast, unit/ragged 축 (pad_einsum). rank-1 은 명시적 reshape 필요
Control flow	지원된 정적 호출을 직선 stream 으로 전개 . 임의 dynamic control flow / packed call 은 거절
I/O	tensor 와 nested tuple, dtype 별 byte 크기 , typed output tree (leaf 별 shape/dtype/offset)
View	지원되는 연속 reshape / axis-0 split / offset view. 임의 strided slice 나 모든 INT8 byte offset 이 zero-copy 는 아님
FP16 수치	matmul/sum FP32 누산과 정의된 FP16 저장 경계 . 범용 FP32 vector tensor 실행은 별도 범위
Softmax / RMSNorm	마지막 축 , 정적 길이 1..65504, 유한 logits 전제 / 마지막 축 , FP16 입력 ·weight, 정적 양의 width, $2^{-48} \le \varepsilon \le 65504^2$
Attention	지원되는 정적 FP16 layout / bias / scale / causal / window. Full-fusion 은 rank-3 64 배수 조건으로 더 좁음 , online softmax 없음
RoPE / GQA	default / linear / llama3 의 지원 필드만 . 다른 설정은 확인 없이 허용하지 않음 . GQA 는 repeat 또는 broadcast matmul 후보
Cache	FP16 정적 capacity, additive mask [1,1,C+1], versioned append metadata, 선택적 bucket
Runtime	C-model adapter 와 image/file patch. RTL / 실장치 runtime 검증 아님
Snapshot	관찰해야 하는 중간값 때문에 SRAM-only 전달을 제한하거나 끔
양자화	W8A16 / W8A8 pass·kernel 구현 , 작은 kernel bit equality. 전체 generation 에는 미연결 (부록 A2)

보장하지 않는 것 : 실장치 latency, 모든 동적 shape·TVM op·HF 모델 , NaN/Inf edge semantics, 자동 mixed precision, quantized KV, key online-softmax, 임의 inter-kernel tile streaming.

근거 : docs/current/12_operations_limits.md, reference/execution_contract.md

옵션과 API: 무엇을 켜고 끄며, 오해하기 쉬운 점은 무엇인가

npu_target.build(module, func_name, fusion, residency, profile) 와 NpuProfile 필드. 각 옵션은 한 단계만 바꾸며 다른 최적화를 함께 끄지 않습니다.

옵션	기본 동작	오해하기 쉬운 점
fusion = auto	합법 기준 + graph 후보 비교 (5 장)	전역 최적 보장 아님
fusion = off	attention / generic fusion 을 끔	메모리 최적화까지 전부 끄는 것은 아님
fusion = on	pattern / generic fusion 요청	지원되지 않는 복합 kernel 은 실패 가능
joint_memory (True)	중간 FP16 tensor 전체 보관 + schedule 동시 선택	부분 tile 보관이나 runtime eviction 아님
reuse_sram (True)	커널 내부 수명에 따른 USMP 공간 재사용	커널 사이 보관과 다른 옵션
residency (True)	후단 versioned SRAM forwarding + joint search 활성	compile_program 기본은 False, public build 는 True
graph_search_seconds	기본 120 초 / None 해제	E10 은 None, 전체 timeout 아님
schedule_max_attempts = 64	성공 / 실패 / 용량 초과 포함 시도 제한	64 개의 성공 후보 보장 아님. None = 무제한
schedule_search = "space"	추정 DMA 순 탐색 (기본)	"meta_schedule" 은 ReplayFunc 추첨, "legacy" 는 기준 규칙만. 학습형 탐색 아님
schedule_probe_word_budget = 100,000	max(설정, 기준 words × legal 목표)	설정값이 항상 상한은 아님. 후보 완료 후 검사
schedule_database (compilation)	trace DB 저장 · 재사용, mode reuse / tune	기본 꺼짐. 최외곽 context 에서만, 버전별 디렉터리
cache_buckets	미리 만든 capacity 별 decode 선택	매 step 임의 shape 의 동적 실행 아님
NPU_NATIVE / NPU_CODEGEN_SO	native codegen 사용 여부 / 경로	결과 words 는 compare 모드로 동일성 검사

```
from npu_compiler import npu_target
from npu_compiler.npu_compilation import compilation

with compilation(progress=lambda e: print(e)):
    exe = npu_target.build(module,
            func_name="prefill", fusion="auto")

# values: 최종 signature 이름 → 배열
output, counters = exe.run(values)
exe.export_bundle("/tmp/npu-bundle", values)

# 선택형 trace DB (기본 꺼짐, mode: reuse | tune)
with compilation(schedule_database="build/schedule_db",
                 schedule_database_mode="reuse"):
    exe = npu_target.build(module, func_name="prefill")

# generation (checkpoint 준비 후)
python d_compiler/run_nn_generate.py --model llama \
  --prompt "Hello, NPU compiler!" --tokens 3 --capacity 9
```

build_image / run 은 raw checkpoint 의 변환을 자동 추측하지 않습니다. host transform 과 _bind_inputs 의 역할 (3 장, 9 장) 을 따라 최종 signature 에 맞는 values 를 넘겨야 합니다.

옵션 하나는 한 단계만 바꿉니다. "fusion off" 가 메모리 최적화를 끄지 않고, "joint_memory" 가 runtime eviction 을 뜻하지 않습니다.

근거 : docs/current/12_operations_limits.md, npu_compiler/npu_target.py, device.py

오류가 나면 어느 단계부터 보는가

관찰한 증상을 단계의 결정에 대응시키면 확인할 파일과 예제가 정해집니다 . Native/Python 일치는 계약 준수의 근거이지 모델 수식 오류까지 배제하지 않습니다 .

관찰한 증상	먼저 확인할 것	관련 단계 / 파일
LLVM 부터 HF 와 크게 다름	모델 수식 , weight 순서 , mask/RoPE, 정밀도 정책	2~4 장 : nn_models, npu_legalize, npu_numeric
작은 K 는 맞고 큰 K 가 틀림	FP32 accumulator 연결과 부분합 순서	6 장 : npu_accumulator, emit_gemm, tile_order_reference
첫 tile 만 맞고 뒤 tile 이 틀림	MAIN stride, PARTIAL 주소 , dtype/ 주소 단위 (nibble·cell·unit)	9 장 : tir_codegen_v09, isa_compute, isa_v09
SRAM 보관을 켤 때만 틀림	값 버전 , 마지막 소비 , alias, 덮어쓰기 판단	8 장 : npu_memory_search.discover, parameter_scopes
주소 압축 후 출력 /cache 가 틀림	DMA 와 output/constant/append 주소의 일관성	8 장 : compact_unused_storage, npu_cache 재구성
Prefill 은 맞고 다음 decode 가 틀림	현재 token, position, mask, cache append 슬롯	9 장 : length_mask, AppendPlan.position/words, DecodeSession
Words 가 너무 많음	짧은 vector 행 , 주소 설정 반복 , 정적 loop 전개	7 장 /9 장 : pointwise coalescing, peephole
Compile 이 오래 걸림	trial/entry/region cache hit·miss, 거절 이유 , phase 시간 , 종료 한도	10 장 : compilation.describe, budget, native_rejections
native 와 Python 이 다름	compare 모드의 첫 다른 word, contract/source hash	9 장 : native.py, v09_contract.json, build_identity

증상 → 단계 → 예제의 IR/metadata 에서 실제 선택 확인 . 처음부터 파일 순서로 읽기보다 관심 있는 현상이 어느 단계의 결정인지 먼저 정합니다 .

근거 : docs/current/12_operations_limits.md

남은 이슈 : 구현 착수 목록이 아니라 검토 목록

효과가 작거나 custom 복잡도가 큰 변경은 원하지 않습니다 . 기존 구현을 유지한 상태에서 필요성과 규모를 먼저 평가합니다 . 이번 문서 작업에서 최적화를 추가 구현하지 않았습니다 .

항목	현재 한계	재개할 때 먼저 확인
Vector 256 / 긴 VLEN	과거 상한 설명과 후속 결정 충돌 , C-model 은 물리 버퍼 모형 아님	실제 장치의 LOAD/ 연산 /SAVE· 유효 길이 · 버퍼 계약
Vector schedule / 비용	용량에 맞춘 행 축소 위주 , cycle·lane 활용률 미반영	대표 kernel 의 짧은 명령과 SRAM 임시값 비용
작은 fusion 영역	큰 TVM 영역 채택 / 해제 중심	일부만 합치면 최종 프로그램이 좋아지는지 제한 비교
ISA activation	자동 matmul activation OFF	마지막 K, bias 순서 , FP16 반올림과 품질 차이
Attention	Query 분할 , key online softmax 없음	장치 누적 상태와 긴 context 에서의 예상 이득
Decode 길이	Capacity 전체 계산과 append 전송	bucket 효과 , 빈 슬롯 낭비 , KV 최종 위치 직접 저장 계약

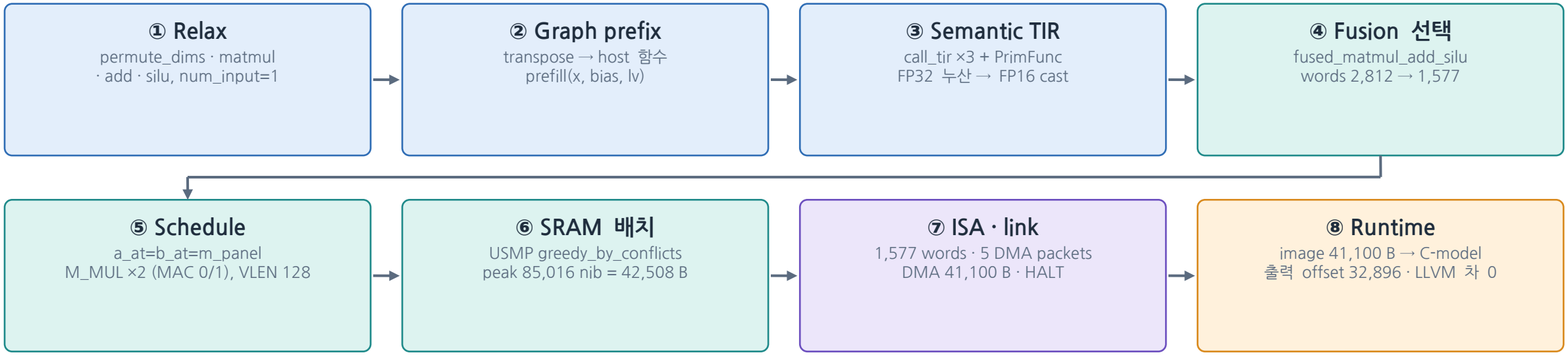
항목	현재 한계	재개할 때 먼저 확인
메모리	Whole-tensor 보관 , 고정 호출 순서 , 일부 조각화	보관 포기 사례의 실제 손실 . 실행 순서 / 퇴거 / 부분 tile 은 큰 변경으로 분리
DRAM 압축	완전 미사용 pool 제거	사용 중 pool 내부 낭비가 의미 있는지 확인 후 수명 재분석
병렬 matmul 결합	QKV / gate-up 표준 pass 효과 미확인	CombineParallelMatmul 과 split/layout 비용 포함 비교
DLight / MetaSchedule	DLight 미사용 ; trace DB·ReplayFunc 선택형	여러 seed 의 개선과 탐색 비용 ; 학습형 모델은 별도
정밀도 선택	고정 계약 , 자동 오차 예산 탐색 없음	허용 오차 · 모델 품질 기준을 먼저 정함
성능 모델	DMA → words → SRAM 비교	RTL/ 장치 timing 확보 뒤 상관 확인
양자화 generation	공개 generate 에 선택 미연결	후반부 범위부터 검증 , KV 양자화까지 완료로 확대하지 않음

새 기능을 늘리기 전에 예상 이득 , 공통 구현 가능성 , 검증 비용을 함께 확인합니다 . 지표 (DMA/words/SRAM) 로 확인되지 않는 개선은 채택하지 않습니다 .

근거 : docs/current/12_operations_limits.md

정리 : 작은 예제 하나가 지나온 길 — 계산에서 실행까지

SiLU(X[64,128]·Wᵀ+bias) 가 각 단계에서 무엇으로 바뀌었는지 . 같은 흐름이 실제 Llama 28 layers 에서는 8.58M words 의 prefill 과 7.31M words 의 decode 가 됩니다 .



세 질문	답을 정하는 단계	실제 Llama 에서의 결과 (보존 기록)
무엇을 계산하는가 (계산의 의미)	Relax export → graph prefix → LegalizeOps custom map (FP16 저장 /FP32 누산 , softmax·RMSNorm 정의)	HF 와 같은 greedy token 3 개 , logits cosine ≥ 0.99999
어떻게 나누는가 (배치)	fusion 후보 비교 → matmul panel 탐색 → tensorize → USMP	5 커널 MLP 140,594 words, gate W panel 3072×64 128 회 , down W 4 MiB 유지
어디에 보관하는가 (수명 · 주소)	StaticPlanBlockMemory → 커널 사이 보관 탐색 → 직접 저장 → DRAM 압축	H SRAM 전달로 DMA 81,932 → 49,164 B (44 KiB 예제), image 6.43 GB (prefill)

" 모델을 compile 한다 " = graph 선택 + schedule + 주소 배치 + 명령 생성 . 각 결정은 실제 프로그램의 (DMA, words, SRAM peak) 로 확인하고 , 수치는 HF/LLVM 기준으로 따로 검증합니다 .

근거 : docs/examples/pipeline/*, two_matmuls_recorded/*, evidence/current/*

기존 main(ver.08 / source-0818) 과 최신 v09 는 실행 경계가 다르다

main commit 1143abbc… 로 고정해 설명합니다 . 둘의 words 나 실행 시간을 숫자만 비교해 성능 순위를 만들 수 없습니다 : 컴파일 단위 , 메모리 경계 , 실행기 , codegen 세대가 다릅니다 .

항목	main ver.08 / source-0818	최신 v09 (이 발표)
주된 lowering	Relax graph 를 수작업 연산별 emitter 로 연결	LegalizeOps / TIR / Schedule 을 거쳐 공통 codegen
메모리	Flat G-buffer 와 host 의 실행 경계 (vendor 8192-entry, 확장 source target)	DRAM ↔ SRAM ↔ 연산장치 , 명시적 GLOAD/GSTORE 와 SRAM 보관 탐색
Layer 반복	같은 layer 프로그램을 host 가 weight 를 바꾸어 실행	전체 entry stream, 컴파일 판단 (recipe) 만 재사용
Decode 길이	Context 별 정적 프로그램 cache	고정 capacity + mask, 선택적 bucket
KV 상태	Host 가 반환된 K/V 로 배열을 확장	Persistent DRAM cache 에 device DMA append
LM head	큰 stride 를 K×64 packed panel 로 처리 (16-bit stride field 한계)	일반 compiler 의 지원되는 배치 · 명령 경로
종료 / 출력	0xF0 snapshot 과 ver.08 실행 규약	HALT 0xFF 와 v09 image 규약
Vendor quirk	reduce-max 0 seed 등을 compiler 에서 우회	v09 ISA/C-model 에서 의미를 명시적으로 고침 (첫 원소 seed)

main 의 보존 결과	Llama 3.2 3B: token [358, 2846, 4560] HF 일치 , 최종 decode cosine ≈ 0.9999881, context8 decode 453,349 words (layer 프로그램 통계 , v09 entry 의 수백만 words 와 컴파일 범위가 다름)
다른 family (main)	Gemma 4 E2B [108, 236777, 236789] cosine ≈ 0.9999975 · Qwen3-4B [358, 1184, 311] cosine ≈ 0.9999915 — v09 generation CLI 재검증은 아님

현재 작업 트리에 구형 backend 를 복원하거나 main branch 를 수정하지 않습니다 . 원본은 git show 1143abb:d_compiler/npu_compiler/backend_0818.py 등으로 읽습니다 .

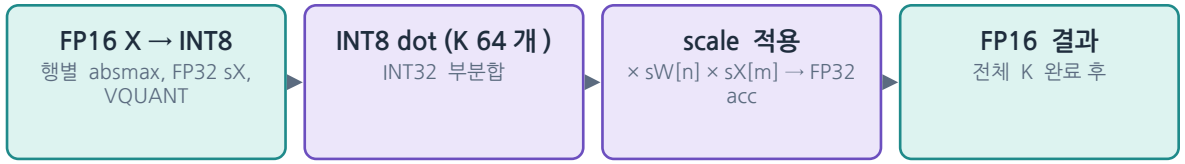
양자화 W8A16 / W8A8: 저장 dtype 과 연산 dtype 을 구분한다

Pass 와 커널은 구현되어 있지만 최근 전체 28-layer 연속 생성 검증은 FP16 입니다. 양자화 generation 까지 완성 · 검증됐다고 이해하지 않도록 본문과 분리합니다.

W8A16



W8A8



계산 / 데이터	현재 pass 의 범위
Q/K/V/O, gate/up/down, LM head weight matmul	조건 만족 시 양자화 대상
Attention QK^T, PV	runtime operand → 대상 아님
RMSNorm, softmax, RoPE, residual	FP16 경로
Projection 입출력	FP16 (W8A8 은 내부에서 동적 양자화)
KV cache	FP16

항목	내용
scale	sW[j] = max(|W[:,j]|, tiny)/127, per-output-channel; W8A8 activation per-row FP32
반올림	half-to-even; 예 : scale 1/127 에서 0.5 → 64 → 복원 0.50394
pass 위치	LiftTransformParams 전에 rewrite → scale/quantize 가 host 로
W8A16 경로	FP16 복원 weight + 기존 GEMM chain (ISA mixed matrix mode 는 미선택)
W8A8 수치	64 개 부분합마다 scale 적용 순서까지 기준. 전체 K INT32 합 뒤 1 회 scale 과 항상 같지 않음
schedule	versioned W8A8 TIR: K/N 64 배수, 정적 rank-2, K ≤ 65535; FP16 X 와 INT8 X 공존 구간
검증	W8A16 matmul, W8A8 [64,64]@[64,64], [7,128]@[128,192] bit equality; 작은 모델 cosine ≈ 0.999998
미연결	run_nn_npu.py --quantize 경로는 있으나 generate 에는 미연결

INT4 형식이 ISA 에 있어도 INT4 모델 compiler 완성이 아니고, 작은 kernel 의 bit equality 가 양자화 전 모델 품질 보존을 뜻하지 않습니다.

근거 : docs/current/13_quantization.md, npu_compiler/npu_quantize.py, npu_w8a8_schedule.py, evidence/current/quantization.log

TVM API 를 동작 기준으로 읽기 : 무엇을 받아 무엇으로 바꾸며 , 아직 무엇을 정하지 않는가

설치된 TVM 0.19.0 기준 . 이름을 외우기보다 각 API 가 결정하는 것과 결정하지 않는 것을 구분합니다 .

Graph pass	하는 일	현재 사용의 경계
LiftTransformParams	parameter-only 계산을 별도 함수로	실행은 host 준비 단계
LegalizeOps(map)	Relax op → TE/TIR 호출	Relax 함수는 남음
FoldConstant	상수만의 계산 미리 평가	NPU 는 K=64 grouping wrapper 사용
AnnotateTIROpPattern	TIR 성격 분류	SRAM 적합성 · 속도 판정 아님
FuseOps / ByPattern	의존 그래프 ·pattern 으로 영역	TIR 결합 · 채택 판단은 별도
FuseTIR	영역의 TIR 을 한 PrimFunc 로	SRAM 배치는 후속
RewriteDataflowReshape	reshape 를 view 표현으로	transpose 는 복사 없이 처리 안 됨
DeadCodeElimination	미사용 함수 · 값 정리	읽는 원소 수 감소와 다름

MetaSchedule API (선택형)	제공 기능	현재 사용 범위
Workload / TuningRecord / JSONDatabase	작업 ·trace· 기록 보존	선택된 trace 만 , run_secs=None, 버전별 디렉터리
tir.Schedule.trace	적용한 Schedule 명령 기록	새 TIR 에 재적용 후 lowering·SRAM·codegen 재검증
TuneContext / ScheduleFn / ReplayFunc	탐색 문맥 · 후보 생성 · 재실행 전략	같은 NPU 후보 pool 의 검사 순서를 바꾸는 비교 경로
sample_categorical	범주 선택을 확률적으로 기록	정리된 후보 번호를 같은 확률로 추첨
relax_integration.extract_tasks	같은 TIR 작업 합치기 · 호출 수 집계	작업 목록 진단만 . TaskScheduler 미사용

Schedule / TIR / 메모리 API	역할	확인할 것
split / fuse / reorder	loop 분할 · 평탄화 · 순서	predicate, 의존성 , 단일 accumulator
cache_read / cache_write	별도 scope 복사 stage	어느 소비자가 공유하는지
compute_at / reverse_compute_at	생산 · 소비 block 이동	관련 소비가 허용 범위인지
set_scope / pad_einsum / decompose_reduction	scope 변경 / tile 배수 확장 / init 분리	zero padding 의미 , 누산 시작 loop
TensorIntrin.register / tensorize	intrinsic 등록 · 치환	shape/dtype/scope/ 수식 계약
rfactor	reduction 을 부분합으로	CPU reference 의 K=64 grouping
CompactBufferAllocation	접근 범위로 allocation 축소	cache stripe 크기
StaticPlanBlockMemory / USMP	DRAM pool / SRAM 충돌 분석	flat 주소는 backend 가 부여
structural_hash / equal	IR 동일성	cache 키 + equality 확인

쓰지 않았다고 문제인 것은 아닌 API: DLight 는 schedule 규칙 체계 , MetaSchedule 은 탐색 · 측정 framework 입니다 . 09-29 에 MetaSchedule 의 trace DB 와 ReplayFunc 를 선택형으로 연결했고 DLight 는 미사용입니다 . 이 API 가 장치 계약 ·SRAM 보관 · 비용 모델까지 자동 해결하지는 않으며 EvolutionarySearch· 학습형 CostModel 은 도입하지 않았습니다 . CombineParallelMatmul 같은 추가 pass 도 concat/split 비용을 측정한 뒤 판단합니다 . CanonicalizeAttention, FoldRepeatedGemv, 양자화 rewrite, OutputViews 는 우리 모듈의 변환입니다 .

근거 : docs/reference/tvm_api.md

재현 환경과 명령

작은 예제부터 시작하고 소스 · 환경 · 설정을 기록합니다 . 기존 측정값이 현재도 같을 것이라고 가정해 재사용하지 않습니다 .

환경	값
TVM	0.19.0, commit c4dc0c29ff81…
LLVM	18.1.8
Python	3.11.15 (conda env npu-tvm)
numpy / torch / transformers	2.4.6 / 2.12.0+cpu / 5.12.1
C-model	_poc/mysim_v09.cpp (g++ -std=c++17 -O2)
native codegen	npu_codegen/libnpu_codegen.so (build.sh)
lock	d_compiler/toolchain.lock.json (conda solver lock 아님)

```
conda activate npu-tvm
bash d_compiler/npu_codegen/build.sh                # native library ( 필요 시 )
python d_compiler/docs/examples/reproduce.py --out /tmp/npu-doc-examples
    # E1 pipeline + E2 memory:  작은  compile 7 개  + C-model  실행  (--case pipeline|memory)
python d_compiler/docs/examples/export_llama_ir.py --out /tmp/npu-doc-llama-ir
    #  실제  3B  폭  1-layer IR export 만  (weight  읽기 ·codegen  없음 )
python d_compiler/docs/examples/schedule_api/reproduce.py   # compute_at  등  교육  dump
python d_compiler/docs/tools/check_docs.py             #  문서  링크 · 수치  검사
python d_compiler/run_nn_generate.py --model llama --prompt "Hello, NPU compiler!" \
    --tokens 3 --capacity 9                          #  전체  모델  (checkpoint  필요 ,  장시간 )
```

새 실험을 추가할 때 함께 남길 기록	
identity	compiler source hash, TVM/native/C-model identity, contract hash
입력	shape/dtype, profile, fusion/residency 설정 , seed 또는 checkpoint/prompt
실행	실제 실행 여부 (executed), 비교 기준 (HF/LLVM/NumPy) 과 tolerance
해석	같은 입력으로 한 변수만 바꿈 . 큰 context 는 예상 compile·memory 비용부터 산정 . timing 없으면 words/DMA 감소까지만 주장

작은 예제의 실행 검증 , 기존 전체 Llama 기록 , 정적 MLP compile, 과거 단일 시간 관측을 구분해 인용합니다 .

근거 : docs/current/12_operations_limits.md, docs/examples/README.md, d_compiler/toolchain.lock.json

TVM 기반 LLM NPU COMPILER

계산의 의미 → 실행 배치 → 수명과 주소 → 명령과 runtime

각 결정은 실제 프로그램의 (DMA bytes, ISA words, SRAM peak) 로 확인하고 ,
수치는 HF / LLVM 기준으로 따로 검증합니다 .

문서 : d_compiler/docs/README.md (13 장) · 예제 : docs/examples/ (실제 IR· 명령 · 실행 결과) · 근거 : docs/evidence/ (source hash 와 함께 보존)
코드 : d_compiler/npu_compiler/, npu_codegen/ · 검증 : d_compiler/run_gate.sh, docs/examples/reproduce.py

2026-09-28 작업 트리 · TVM 0.19.0 · v09 backend · FP16